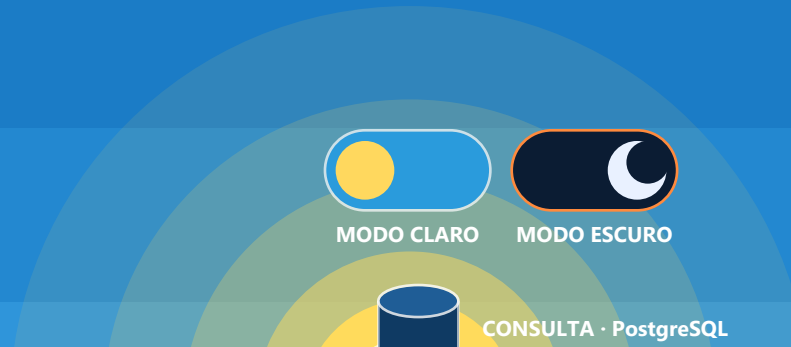


ROYAL NAVY · SITE EDUCATIVO

Documentação Técnica Completa

Desempenho, dois temas, consulta segura ao banco e publicação online — dossiê de construção do site, versão 4.4



Repositório

github.com/AlanSouzaDev7/Naval_Design-WEB-V.1

Versão

Versão 4.4 · 20 de outubro de 2026 · tag v4.4.0 · GitHub Pages + Render + Neon

Sumário

1	**Resumo executivo**	resultados, métricas e o que foi entregue (v1.1, v2.0 e v3.0)
2	**Dossiê técnico: como o site foi construído**	arquitetura, arquivos e o papel de cada linguagem
3	**O motor 3D em detalhe**	Three.js, WebGL, ciclo de vida das cenas e temas
4	**O hero animado e as demais animações**	Canvas 2D, controlador de cenas, reveal, contadores, modal
5	**Modo escuro: guerra em alto mar (v2.0)**	botão, transição, batalha, mergulho, naufrágios e desempenho
6	**Diagnóstico de desempenho**	o que deixava o site lento e como foi resolvido
7	**Erros e bugs corrigidos**	catálogo completo com causa e solução
8	**Paletas: dia ensolarado e noite de guerra**	cores, aplicação e acessibilidade
9	**Como foi verificado**	metodologia, medições e limitações
10	**Pendências e recomendações**	o que ficou de fora e próximos passos
11	**Operação: executar, versionar e publicar**	comandos, variáveis, versões e GitHub
12	**Publicação, domínio e segurança**	GitHub Pages, royalnavy.com.br, CSP e checklist
13	**Consulta de navios: banco de dados e API (v3.0)**	PostgreSQL, API Flask, página de consulta, segurança e testes
14	**Disponibilidade online (v4.0)**	Render + Neon, o certificado TLS, dados no ar e o que ficou pendente
15	**Tema claro “Mar em Paz” (v4.1)**	hero em perspectiva, mergulho, recifes de coral, teste de fluidez e barra de navegação
16	**Glossário**	termos técnicos usados neste documento

Como ler este documento. As seções 1 a 4 explicam *o que é o site e como ele funciona*; a seção 5 é o dossiê do **modo escuro** (versão 2.0), a seção 13 o da **consulta ao banco de dados** (versão 3.0) a seção 14 o da **publicação online** (versão 4.0) e a seção 15 o do **tema claro “Mar em Paz”** (versão 4.1); as seções 6 e 7 registram *o que estava errado e o que foi feito*; a seção 8 documenta as duas identidades visuais; as seções 9 a 12 cobrem verificação, pendências, operação e a publicação na internet (incluindo o domínio próprio). Todos os números citados foram medidos ou extraídos do código-fonte durante o trabalho — nada é estimativa sem indicação.

O que mudou nesta edição (v4.4). PDFs para download da ficha de cada navio e do resultado da busca — seção 19 (página 93). Na v4.3, sugestões de navios e painel do administrador — seção 18 (página 89); na v4.2, revisão de segurança — seção 17 (página 85). O tema claro “Mar em Paz” (v4.1) está na seção 15; a consulta online (v4.0), na seção 14.

1. Resumo executivo

O projeto **Royal Navy** é um site educativo sobre a Marinha Real Britânica, com linha do tempo, navios lendários em **3D interativo**, batalhas, frota atual e galeria. É servido por Flask (Python) e escrito em HTML, CSS e JavaScript puros, com o Three.js para os modelos 3D. A partir da **versão 2.0** ele tem **dois modos**, escolhidos por um botão no canto superior direito: o **modo claro** (um dia ensolarado de navegação nas versões 1.1 e 2.0; a partir da **versão 4.1** o "Mar em Paz", seção 15) e o **modo escuro** ("guerra em alto mar": batalha naval no topo, mergulho ao descer a página, combates submarinos e navios naufragados no fim). A **versão 3.0** acrescenta uma página de **consulta de navios**: o visitante digita o nome e recebe a ficha completa, lida de um **banco PostgreSQL** por uma API somente-leitura, construída para poder ser publicada com segurança.

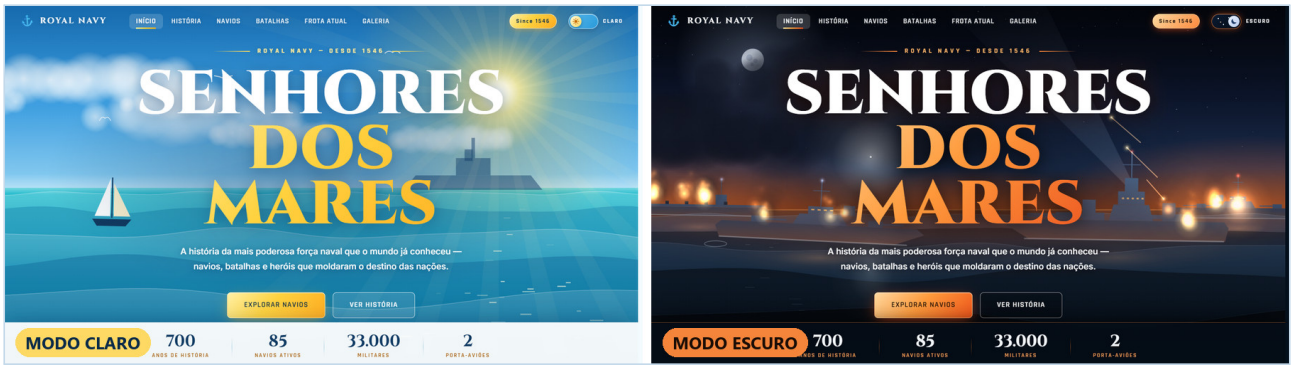


Figura 1 — O mesmo hero nos dois modos (1366×768). Esquerda: modo claro da v2.0 (o hero claro foi redesenhado na v4.1, Figura 30), com o botão de tema. Direita: modo escuro, com a batalha naval em curso.

Foram solicitadas as entregas abaixo (a publicação online da v4.0 está na seção 14), todas concluídas:

- **Versão 1.x — auditoria e nova identidade:** leitura completa do código; diagnóstico de lentidão e de animações pouco fluidas (medido no navegador); correções de desempenho, bugs funcionais, servidor e acessibilidade; paleta "dia ensolarado de navegação"; pré-visualização; publicação no GitHub e **site no ar pelo GitHub Pages** (HTTPS), com preparação segura para o domínio royalnavy.com.br (seção 12).
- **Versão 2.0 — modo escuro "Guerra em alto mar":** botão de tema com animação de transição; hero em batalha naval; mergulho até o leito do mar com medidor de profundidade; faixas de guerra submarina e de navios naufragados; cenas 3D noturnas — tudo **animado e otimizado**, sem alterar o modo claro (seção 5).
- **Versão 3.0 — consulta de navios ligada ao banco:** página `/consulta` (claro, escuro e celular), API `/api/navios` somente-leitura, banco com *view* e papel de privilégios mínimos, e uma **bateria de defesas** — consultas parametrizadas, validação estrita, limite de requisições, segredos fora do Git e da nuvem — verificada por 88 testes automáticos, Bandit e pip-audit (seção 13).
- **Versão 4.1 — tema claro "Mar em Paz":** o modo claro virou o contraponto pacífico do escuro, com o mesmo estilo de profundidade: comboio de navios em perspectiva no hero, mergulho em águas claras com medidor 0–40 m, zona de patrulha (minas, submarino, tubarões, cardumes) e naufrágios com recifes de coral; teste de fluidez e correção da barra de navegação (seção 15).
- **Versão 4.2 — revisão de segurança e robustez:** suíte de 99 testes executada contra um banco descartável (antes, 88 ignorados); fim do 503 intermitente da consulta depois de o banco ficar ocioso; CSP sem 'unsafe-inline' e sem innerHTML; hostname do banco fora do repositório (seção 17).
- **Versão 4.3 — sugestões de navios:** o visitante sugere navios que faltam, o site confere o acervo e um painel do administrador avalia (seção 18).
- **Versão 4.4 — PDFs para download:** ficha de cada navio e resultado da busca, gerados na hora pelo servidor (seção 19).

Resultados medidos — versões 1.0 → 1.1

Indicador	Antes (v1.0)	Depois (v1.1)
FPS medido no painel do navegador (DPR 1,25; WebGL por software)	1,7	56,7 no topo · 60 com os navios visíveis
Contextos WebGL criados ao carregar a página	13, todos renderizando em 60 fps continuamente	0 (criados sob demanda ao chegar perto da tela)
Draw calls do 3D com o hero na tela (nenhum navio visível)	Contínuos (todas as cenas rodavam sempre)	0
Draw calls por frame com o modal aberto	Fundo + modal	≈ 17 (só o modal; fundo pausado)
Redimensionamento de canvas 3D	A cada frame em telas com DPR ≠ 1	Somente quando o tamanho muda
Pesos de fonte baixados do Google Fonts	12	9
Hover dos cards	Não funcionava	Funciona (translateY −8 px)
Barras da seção Frota	Ficavam em 0 para sempre	Animam até o valor correto
Conteúdo do modal (Armamento, História)	Cortado, sem rolagem	Rolável (1192 px de conteúdo em 691 px)
Erros no console do navegador	—	Nenhum, em desktop e celular
Acesso ao site	Só em localhost:5000, na máquina do autor	Público, em HTTPS: alansouzadev7.github.io/Naval_Design-WEB-V.1

Sobre essa medição: o painel do navegador do aplicativo renderiza WebGL por software (sem placa de vídeo), então os valores absolutos são pessimistas — o mesmo teste, mesma máquina e mesma configuração, foi usado antes e depois, o que torna a *comparação* válida. Em uma GPU real o ganho tende a ser menor em FPS, mas continua significativo em uso de CPU/GPU e bateria, porque o trabalho contínuo deixou de existir.

Resultados medidos — versão 2.0

Medidas feitas em uma máquina modesta (Intel Core i3-1215U, gráficos integrados Intel UHD, 7,7 GB de RAM), com o Edge acelerado por GPU (D3D11), janela de 1366×768 e DPR 1. Cada valor é a **mediana de 3 rodadas**, contando quadros do *requestAnimationFrame* durante 3 s. O monitor da máquina atualiza a 120 Hz; uma página sem nenhum canvas desenhando chega a esse teto (≈ 120 fps).

Cenário (fps · ms por quadro)	v1.1 · claro	v2.0 · claro	v2.0 · escuro
Topo: hero animado (canvas 2D em tela cheia)	62 · 16,2	61 · 16,3	61 · 16,3 (batalha naval)
Grade de navios (6 cenas WebGL visíveis)	93 · 10,8	82 · 12,2	3D noturno: mesma ordem de grandeza
Seção sem nenhum canvas (Batalhas)	120 · 8,3	120 · 8,3	120 · 8,3 (só o mar profundo)
Faixa de guerra submarina	— (não existe)	— (oculta)	120 · 8,3
Faixa de navios naufragados	— (não existe)	— (oculta)	90 · 11,1

Como ler: no modo claro a v2.0 fica na mesma faixa da v1.1 (as diferenças entre versões ficam dentro da variação entre rodadas, que chega a ±10% nas cenas 3D nesta máquina). No modo escuro, o hero em batalha custa o mesmo que o hero claro; ao descer, os trechos sem 3D chegam ao teto do monitor porque o fundo do mar roda em meia resolução e as faixas só desenham quando aparecem na tela (seção 5.8). Detalhes, rolagem contínua e limitações na seção 9.

Outros resultados da v2.0

Indicador	Resultado
Modo claro: diferença de pixels contra a v1.1 publicada (corpo das seções História, Linha do tempo e Batalhas)	**0 pixel** (comparação pixel a pixel, mesmas condições) — a única diferença visível é o novo botão na barra de navegação
Modo claro: alterações no CSS original (style.css)	**0 linhas alteradas**; 116 linhas *acrescentadas* (botão, transição e ocultação dos elementos do escuro)
Canvases que desenham no modo claro	Somente o do hero. Fundo do mar, faixas submarinas e medidor têm `display:none`: não medem layout nem executam laço
Peso adicional (5 arquivos novos)	≈ 100 KB brutos · ≈ 29 KB com gzip (o GitHub Pages comprime); scripts com `defer`, só o CSS escuro bloqueia a pintura (4 KB gzip)
Erros de console e violações de CSP	Nenhum, nos dois modos, no Flask local, na versão estática e no site publicado
Persistência e troca	Tema salvo e reaplicado antes da 1ª pintura (sem "piscar"); cliques repetidos durante a animação são ignorados sem quebrar o estado
Publicação	Commit `ae6b9ea` + tag `v2.0.0`; GitHub Pages atualizado em ≈ 15 s e verificado no ar

Resultados da v3.0

Indicador	Resultado
Consulta ao banco	Busca por nome, classe ou designação do casco (ex.: BB-39), sem acento, tolerante a erro de digitação; filtros por marinha, guerra e tipo; ficha completa de cada navio
Dados	64 navios de 8 marinhas nas duas guerras mundiais (tabela original preservada; backup feito antes de qualquer alteração)
Testes automatizados	**88 aprovados**: funcionamento, injeção de SQL e XSS (22 cargas hostis), validação, limite de requisições, cabeçalhos, privilégios do banco e vazamento de segredos
Análise estática e dependências	Bandit: 0 achados · pip-audit: 0 vulnerabilidades (após atualizar o Flask 3.0.3 → 3.1.3, CVE-2026-27205)
Privilégios do banco	A API usa um papel que só lê uma *view*: não lê a tabela original, não escreve, não cria nada, consulta lenta é cancelada em 3 s
Latência (Flask + PostgreSQL, sem rede)	busca ≈ 8 ms (mediana) · ficha ≈ 5 ms · filtros ≈ 2 ms; p95 abaixo de 11 ms
Site publicado (GitHub Pages)	Continua no ar; a página de consulta mostra um aviso claro, pois o Pages não executa servidor nem acessa banco
Erros de console	Nenhum, nos dois modos e no celular

Tamanho do código

Arquivo	v1.0 (original)	v1.1	v2.0	Observação (v2.0)
static/css/style.css	1569	1579	1695	Só acréscimos (116 linhas): botão, transição, ocultar itens do escuro
static/css/dark.css	—	—	406	Novo: todo o visual do modo escuro
static/js/main.js	464	617	668	Controlador RNHero + cena clara registrada + ganchos de tema
static/js/ships-3d.js	1034	867	915	Temas claro/escuro para o 3D
static/js/ships-data.js	182	182	182	Sem alterações
static/js/theme-init.js	—	—	11	Novo: aplica o tema antes da 1ª pintura
static/js/theme.js	—	—	140	Novo: botão, transição, desempenho adaptativo
static/js/war-hero.js	—	—	741	Novo: batalha naval do hero
static/js/deep-sea.js	—	—	791	Novo: mergulho, guerra submarina, naufrágios, medidor
templates/index.html	545	557	609	Botão, canvases, faixas, scripts
app.py	41	70	70	Sem alterações na v2.0
static/js/consulta.js	—	—	326	**v3.0** · página de consulta
static/css/consulta.css	—	—	267	**v3.0** · visual da consulta (claro e escuro)
templates/consulta.html	—	—	131	**v3.0** · página de consulta
consulta_api.py · config_env.py · db/ · tests/	—	—	≈ 1000	**v3.0** · API, segredos, SQL/aplicador do banco e 88 testes (Python/SQL)
Total de JavaScript	**1680**	**1666**	**3774**	v2.0 +1700 (modo escuro) · v3.0 +326 (consulta)

Linhas contadas com `wc -l` nos commits da v1.1 (`d006295`) e da v2.0 (`ae6b9ea`); a coluna "v2.0" não inclui os arquivos novos da v3.0, listados nas últimas linhas. Na v1.1 o `app.py` já estava com os cabeçalhos de segurança (70 linhas). **Mudaram na v3.0:** app.py 70 → 156, index.html 609 → 614, style.css 1695 → 1713 e dark.css 406 → 411 (link e atalho de busca na barra); os demais ficaram como na v2.0.

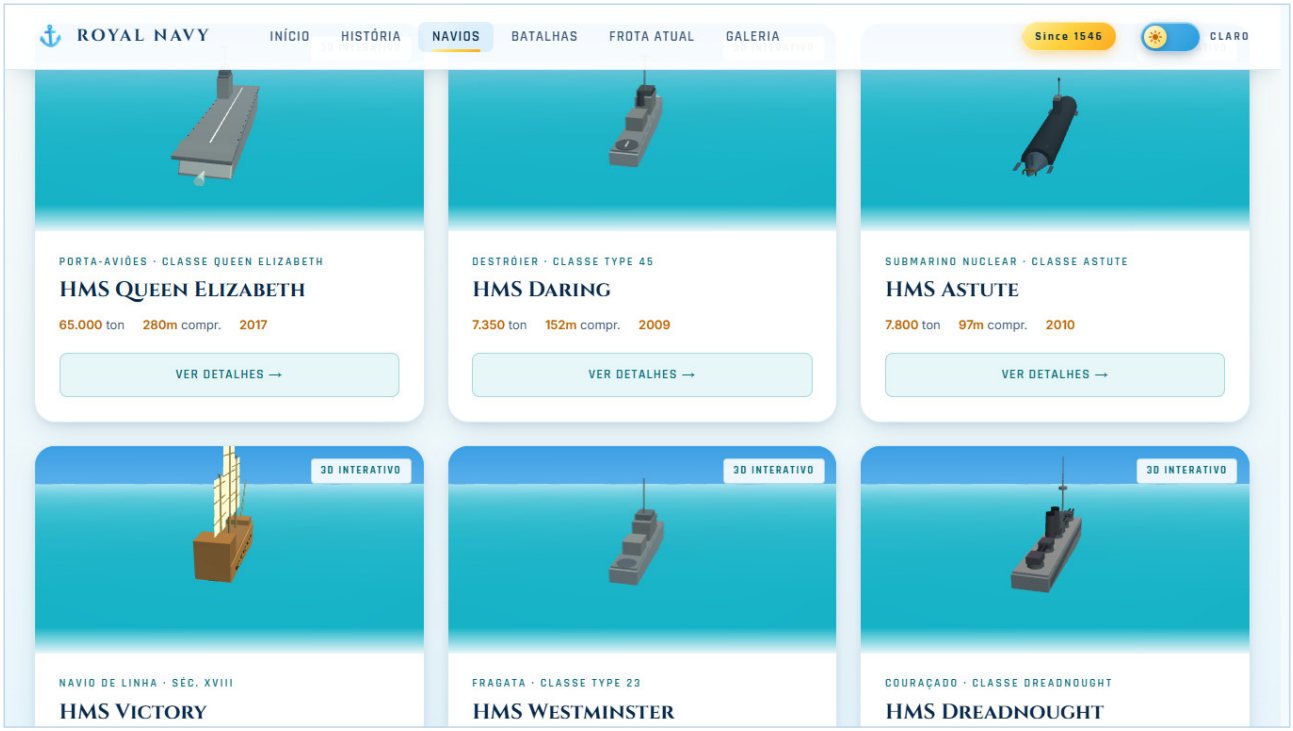
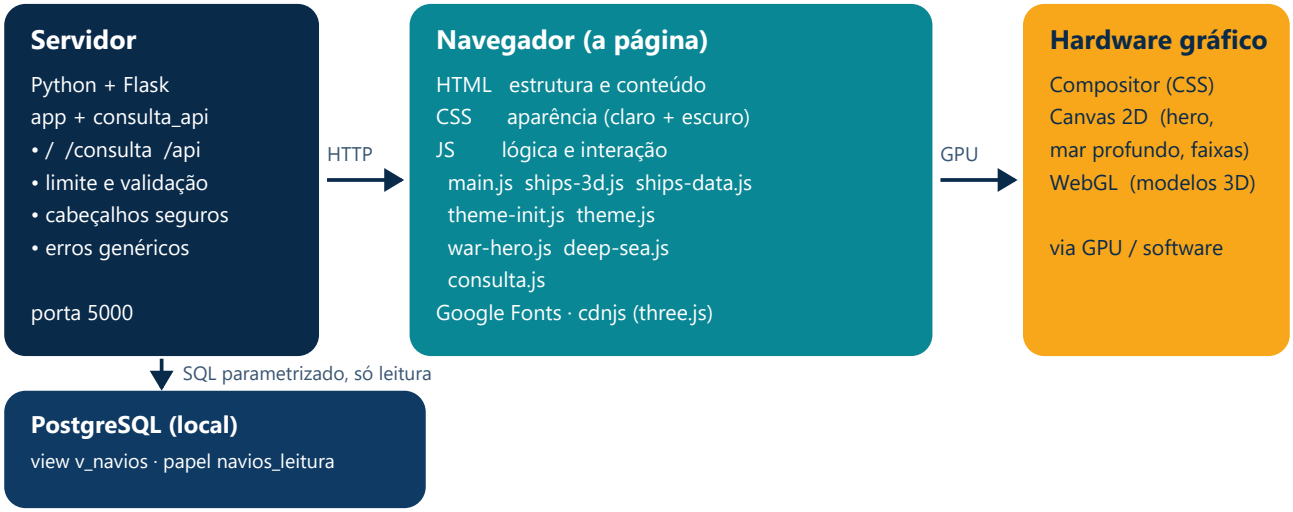


Figura 2 — Modo claro, seção Navios: cards com cenas 3D de mar ensolarado, iluminadas por sol e céu (idêntica à v1.1).

2. Dossiê técnico: como o site foi construído

2.1 Visão geral da arquitetura

O site é uma **página única** (single page) servida por um pequeno servidor Python. O servidor entrega o HTML e os arquivos estáticos; **toda a interatividade acontece no navegador**, em três camadas que trabalham juntas: estrutura (HTML), aparência (CSS) e comportamento (JavaScript). O JavaScript se divide em responsabilidades distintas — interface, hero em canvas 2D, 3D em WebGL, dados e, desde a v2.0, **tema** (botão, transição e cenas do modo escuro).



Fluxo: o navegador pede "/" → Flask devolve index.html → o navegador baixa CSS, fontes e scripts → o JavaScript monta a interação e pede à GPU para desenhar. Depois do primeiro carregamento, o servidor não participa mais. O tema escolhido fica no navegador do visitante (localStorage) e é aplicado antes da primeira pintura.

2.2 Quem faz o quê: o papel de cada tecnologia

Tecnologia	Responsabilidade no projeto	Onde vive
Python 3 + Flask	Servidor web. Entrega a página em `/`, os arquivos de `/static`, define cache, trata 404 e decide se o modo debug pode ficar ligado. **Não** gera conteúdo dinâmico: o template não tem lógica Jinja. Serve os arquivos do modo escuro sem nenhuma alteração. **Na v3.0** também hospeda a **API de consulta**: valida o cabeçalho Host, envia cabeçalhos de segurança e CSP, limita requisições (Flask-Limiter), fala com o PostgreSQL por um pool de conexões (psycopg) e devolve erros genéricos.	app.py · consulta_api.py · config_env.py
PostgreSQL + SQL	**v3.0.** Guarda os dados dos navios (tabela `navios_historicos`) e faz a busca: extensões `unaccent` e `pg_trgm`, funções de normalização, índices, a *view* `v_navios` e o papel de leitura `navios_leitura`, criados por scripts SQL versionados. Só o servidor Flask fala com ele.	db/01_busca.sql · 02_papel_leitura.sql · aplicar.py
HTML5	Estrutura semântica e todo o conteúdo em português: navegação, hero, linha do tempo, cards de navios, batalhas, frota, galeria, rodapé e o modal. Define os `<canvas>` onde o 3D e as cenas 2D são desenhados, ganchos de dados (`data-*`) e atributos de acessibilidade. Na v2.0: o **botão de tema** (`role="switch"`), o canvas do mar profundo, o medidor de profundidade e as duas faixas submarinas. **v3.0:** a página `consulta.html` (busca, filtros, ficha) e o link "Consulta" na barra.	templates/index.html · consulta.html
CSS3	Toda a aparência: paleta (variáveis CSS), tipografia, grid/flex, cards, botões, ondas divisórias, responsividade (4 breakpoints), estados de hover/foco, transições e animações de entrada. Na v2.0 há **dois arquivos**: `style.css` (modo claro + botão + transição) e `dark.css` (tudo que só vale com `data-theme="dark"`). **v3.0:** `consulta.css` (visual da página de consulta nos dois modos).	static/css/style.css · dark.css · consulta.css
JavaScript — interface	Comportamento da página: navbar que muda ao rolar, link ativo, parallax do hero, contadores, revelação dos elementos ao rolar, filtro de navios, barras da frota, modal (abrir/fechar/foco/ESC), brilho do cursor.	static/js/main.js
JavaScript — tema	Aplica o tema salvo antes da 1ª pintura, liga o botão, executa a animação circular de troca, avisa o resto do site (evento `rn:themechange`) e mede o desempenho para reduzir efeitos em máquinas lentas (`RNPerf`).	theme-init.js · theme.js
JavaScript — Canvas 2D	Desenha o hero (controlador `RNHero` com uma cena por tema: dia ensolarado ou batalha naval), o fundo fixo do mar profundo, a guerra submarina, os naufrágios e o medidor de profundidade, sempre com animação baseada em tempo.	main.js · war-hero.js · deep-sea.js
JavaScript + Three.js (WebGL)	Constrói e renderiza os modelos 3D: 6 tipos de navio, oceano, luz, câmera e interação de arrastar para girar. Gerencia o ciclo de vida das 12 cenas + modal e agora troca céu, névoa, luzes e água entre o cenário diurno e o noturno.	static/js/ships-3d.js
JavaScript — consulta	**v3.0.** Página de consulta: busca com *debounce* e cancelamento da requisição anterior, filtros, paginação, ficha completa, estado na URL, modo estático. Todo texto vindo da API entra por `textContent` (nunca `innerHTML`).	static/js/consulta.js
JavaScript — dados	Base de dados dos navios (nome, classe, descrição, especificações, armamento, história) que alimenta o modal.	static/js/ships-data.js
SVG (embutido)	Ícone do site (favicon), a máscara em onda usada nos divisores de seção e os ícones de sol e lua do botão de tema — como `data:` URI ou SVG em linha, sem requisições extras.	index.html, style.css

Tecnologia	Responsabilidade no projeto	Onde vive
Serviços externos	**Google Fonts** (Cinzel, Inter, Rajdhani) e **cdnjs** (three.js r128, com verificação SRI). O modo escuro **não** adiciona nenhuma origem externa.	CDN

2.3 Estrutura de arquivos

```
site/
├── app.py                  servidor Flask + cabeçalhos de segurança (156 linhas)
├── consulta_api.py         API de consulta ao banco (337 linhas)            [v3.0]
├── config_env.py           onde ficam os segredos (71 linhas)                [v3.0]
├── requirements*.txt       versões fixas; -lock (fechamento) e -dev (testes)
├── README.md               uso, modos claro/escuro, consulta, publicação, domínio
├── build_static.py         gera docs/ (versão estática)
├── enable_custom_domain.py ativa/remove domínio (trava de DNS)
├── db/                     SQL da busca, papel de leitura e aplicar.py       [v3.0]
│   ├── 01_busca.sql · 02_papel_leitura.sql · aplicar.py · README.md
├── tests/                  88 testes de funcionamento e segurança             [v3.0]
├── docs/                   site estático do GitHub Pages (14 arquivos)
├── documentacao/           este PDF
├── .env.example · .gitignore
├── templates/
│   ├── index.html          a página inicial (614 linhas)
│   └── consulta.html       página de consulta de navios (131)              [v3.0]
└── static/
    ├── css/  style.css (1713) · dark.css (411) · consulta.css (267)
    └── js/   main.js · ships-3d.js · ships-data.js · theme-init.js · theme.js
              war-hero.js · deep-sea.js · consulta.js (326)
```

2.4 Seções da página e o que cada uma usa

Seção (id)	Conteúdo	Tecnologias envolvidas
Navbar	Marca, links âncora, selo "Since 1546" e **botão de tema**	CSS (fixa, troca de estilo ao rolar) · JS (classe *scrolled*, link ativo) · theme.js (botão)
Hero (#hero)	Título, chamada, botões, estatísticas	Canvas 2D (claro: dia de sol · escuro: batalha naval) · CSS (raios de sol, vinheta, animações de entrada) · JS (parallax, contadores)
História (#history)	Linha do tempo com 7 marcos	CSS (grid/flex, cards) · JS (reveal ao rolar) · *escuro:* fundo = canvas fixo do mar profundo
Navios (#ships)	Filtro + 6 cards com 3D	Three.js/WebGL (cenário diurno ou noturno) · CSS · JS (filtro, drag, modal)
Guerra Submarina *(só no escuro)*	Submarinos, torpedos, sonares, cargas de profundidade	Canvas 2D (#sub-canvas) · deep-sea.js
Batalhas (#battles)	6 batalhas, uma em destaque	CSS (grid, ondas em máscara SVG; no escuro as ondas somem e o fundo é o mar)
Frota (#fleet)	Cena 3D em formação + 6 categorias	WebGL · CSS (barras animadas por transform)
Galeria (#gallery)	5 cenas 3D (1 grande)	WebGL · CSS grid
Navios Naufragados *(só no escuro)*	Cascos no leito, algas, cardumes, baliza	Canvas 2D (#wreck-canvas) · deep-sea.js
Modal	Ficha técnica + 3D em tela cheia	HTML (dialog) · JS (foco, ESC) · WebGL (renderer reutilizado)
Rodapé	Links e créditos	CSS (onda divisória no claro; "leito do mar" no escuro)
Consulta de navios *(página própria, v3.0)*	Busca, filtros e ficha completa lidas do PostgreSQL	HTML/CSS/JS (consulta.js) · API Flask · PostgreSQL (view somente-leitura)
Medidor de profundidade *(só no escuro)*	Metros e zona oceânica, à direita, sob a navbar	HTML + CSS (HUD fixo) · deep-sea.js (Gauge)

2.5 Ordem de carregamento (o que acontece ao abrir a página)

- **1.** O navegador pede `/`; o Flask devolve `index.html` (≈ 29 KB).
- **2.** Ele lê o `<head>`: primeiro o `theme-init.js` (síncrono, 0,5 KB), que define `data-theme` a partir do que ficou salvo — assim a página já *nasce* no tema certo, sem piscar. Depois vêm o ícone embutido, *preconnect* para Google Fonts e cdnjs, a folha de fontes (9 pesos) e os dois CSS (`style.css` ≈ 49 KB e `dark.css` ≈ 18 KB). O CSS é bloqueante de renderização — por isso a página já nasce estilizada.
- **3.** Os sete scripts têm `defer`: baixam em paralelo *sem bloquear* a pintura e executam em ordem — theme.js → three.js → ships-data.js → ships-3d.js → main.js → war-hero.js → deep-sea.js.
- **4.** `theme.js` liga o botão; `main.js` liga os módulos (scroll, hero, contadores, reveal, filtro, modal...), cria o controlador `RNHero` com a cena clara e chama `ShipEngine.init()`; `war-hero.js` registra a cena escura no controlador; `deep-sea.js` prepara o fundo do mar, as faixas e o medidor. No modo claro tudo isso fica **dormente**.
- **5.** `init()` apenas *registra* os 12 canvases 3D. Nenhuma cena é criada ainda; isso só ocorre quando um canvas chega a 120 px da tela.

2.6 Decisões de projeto

Decisão	Motivo
Manter Flask, HTML/CSS/JS puros e Three.js r128	Preservar a base do projeto; r128 é a última série com o build global `three.min.js`, que o site usa via CDN.
Paleta em variáveis CSS (`:root`)	Trocar o tema inteiro editando um bloco; os tokens têm nomes semânticos (sky, sea, sun, navy, coral, sand).
Só `transform` e `opacity` em animações	Essas duas propriedades são animadas pelo compositor da GPU, sem recalcular layout nem repintar.
Estado visual por **classes**, não por `style` inline	Evita brigas de especificidade (causa de dois bugs originais) e mantém o CSS como fonte única da aparência.
Um único loop de animação 3D	Um ponto de controle para pausar/retomar tudo (aba oculta, modal aberto, fora da tela).
Servidor só em `127.0.0.1` por padrão	O debugger do Werkzeug executa código no servidor; nunca deve ficar exposto na rede.
v2.0 · Tema como atributo no `<html>` + arquivo CSS separado (`dark.css`)	O modo claro **não foi tocado** (só acréscimos no style.css). Isso garante que o claro continue idêntico ao aprovado — e foi provado por comparação de pixels.
v2.0 · Um único canvas de hero, com cenas registradas por tema	Nada de segundo canvas nem de segundo laço de animação: trocar de tema é trocar qual cena o controlador desenha.
v2.0 · Fundo do mar em *um* canvas fixo, de meia resolução	A cor da água, os raios de luz e as partículas custam ¼ dos pixels; o conteúdo da página (cards quase opacos) fica por cima.
v2.0 · Faixas submarinas com canvas próprio, só ativas quando visíveis	As duas cenas mais pesadas (submarinos e naufrágios) não custam nada enquanto estão fora da tela ou no modo claro.
v2.0 · Sprites pré-renderizados e *pools* de partículas	Nenhuma alocação por quadro: sem pausas do coletor de lixo e sem criar gradientes a cada frame.
v2.0 · Qualidade adaptativa (`RNPerf`)	Em máquinas lentas o site reduz partículas e efeitos sozinho, em vez de travar; o visitante não precisa configurar nada.
v2.0 · `theme-init.js` externo e síncrono	Aplica o tema antes da 1ª pintura e é compatível com a CSP (que proíbe scripts inline).
v2.0 · View Transitions API com cortina de reserva	Melhoria progressiva: navegadores modernos ganham a revelação circular nativa; os demais, uma cortina equivalente.
v2.0 · Tema padrão sempre *claro*	O modo claro é o visual atual do site; a preferência de cores do sistema operacional não é consultada — vale a escolha salva do visitante.
v3.0 · O navegador **nunca** fala com o banco	Só existe a API; o banco fica atrás dela, com um papel de leitura, e a API não aceita SQL nem nomes de tabela do cliente.
v3.0 · Uma *view* em vez da tabela	O papel de leitura enxerga só `v_navios` (campos públicos): colunas internas e a tabela original ficam inacessíveis, e o desenho da API não depende do da tabela.
v3.0 · Validar por *lista de permitidos*	Só letras, dígitos, espaço e `. ' ( ) / -`, 2 a 60 caracteres e parâmetros conhecidos: o que não está na lista é recusado (400), em vez de tentar "limpar" o texto.
v3.0 · Segredos fora do projeto	A senha do banco fica em `~/.royalnavy` (Windows: `C:\Users\<você>\.royalnavy`), com permissão só do usuário: nem o Git nem a sincronização do OneDrive a levam para fora da máquina. Não fica em AppData, que alguns aplicativos do Windows redirecionam.
v3.0 · A versão estática avisa, não falha	O GitHub Pages não executa servidor: a página de consulta detecta isso (meta `rn-mode`) e explica, sem disparar requisições que dariam erro.

Decisão	Motivo
v3.0 · O site principal não depende do banco	Se faltar dependência, configuração ou o banco cair, só a consulta fica indisponível (503 com mensagem genérica); o restante do site continua no ar.

3. O motor 3D em detalhe

Todo o 3D está em `ships-3d.js`, dentro de uma função anônima que expõe apenas `window.ShipEngine = { init, openModal, closeModal, setTheme }`. Se o three.js não carregar (CDN indisponível), o motor vira um *no-op* e o restante do site continua funcionando.

3.1 Modelos procedurais

Nenhum arquivo de modelo (.glb, .obj) é usado: cada navio é **montado por código** a partir de primitivas (caixas, cilindros, cones, esferas, planos e uma extrusão para o casco do porta-aviões).

Função	Modelo	Peças notáveis
buildCarrier()	Porta-aviões (Queen Elizabeth)	Casco extrudado, convés, rampa *ski-jump*, ilha com 3 níveis, radar, 26 luzes de borda
buildDestroyer()	Destróier (Type 45)	Casco + proa, superestrutura, ponte, canhão, VLS, helipad
buildFrigate()	Fragata (Type 23)	Casco, superestrutura, canhão, mastro, helipad
buildSubmarine()	Submarino (Astute)	Casco cilíndrico, cúpula, cone de popa, vela, periscópio, 4 tubos, lemes, hélice
buildGalleon()	Galeão (HMS Victory)	Casco, 3 mastros com 9 velas/vergas, 16 portinholas, 8 canhões, gurupés, bandeira
buildBattleship()	Couraçado (Dreadnought)	Casco, 5 torres de canhões gêmeos, 2 chaminés, mastro com cofa

3.2 Cena, luz e cenário

- **Céu:** uma textura de gradiente (2×256 px, criada em um canvas 2D) usada como `scene.background` e **compartilhada** por todas as cenas do mesmo tema (uma textura por tema, guardada em cache).
- **Mar:** plano de 80×80 com 30×30 segmentos e ondulação determinística (gerador pseudoaleatório com semente fixa); material opaco. A linha d'água fica em y = −0,32, de modo que os cascos aparecem parcialmente submersos.
- **Luz:** apenas duas — *HemisphereLight* (céu/reflexo do mar) e *DirectionalLight* (o sol, ou o luar no modo escuro). Antes eram quatro luzes, e cada luz aumenta o custo de cada pixel.
- **Névoa:** exponencial na cor do horizonte, com densidade dividida pelo afastamento da câmera para não "lavar" navios distantes.
- **Sombras:** somente no modal e na imagem grande da galeria (mapa de 1024^2). Nos demais quadros, desligadas.

3.3 Ciclo de vida das cenas (o coração da otimização)

Cada `<canvas>` vira uma *entrada* no gerenciador, que decide **quando criar**, **quando desenhar** e **quando parar**:

Evento	Mecanismo	Efeito
Canvas chega a 120 px da tela	IntersectionObserver	A cena entra na fila de criação (no máximo **uma por frame**, para não travar a rolagem)

Evento	Mecanismo	Efeito
Canvas sai da tela	IntersectionObserver	Deixa de ser renderizada (o último quadro permanece visível)
Nenhuma cena visível	Loop verifica *active = 0*	O `requestAnimationFrame` é cancelado — zero trabalho
Aba do navegador oculta	visibilitychange	Loop parado; retoma ao voltar
Modal aberto	Flag *backgroundPaused*	Só a cena do modal renderiza; o fundo (coberto) pausa
Card esmaecido pelo filtro	Classe *is-dimmed*	A cena daquele card é pulada
Tamanho do quadro muda	ResizeObserver	`setSize` e projeção da câmera ajustados uma vez
v2.0 · Tema muda	Evento `rn:themechange`	`setTheme()` atualiza as cenas já criadas e redesenha as visíveis; as ainda não criadas já nascem no tema certo

```
function frame(now) {
  rafId = requestAnimationFrame(frame);
  const dt = Math.min((now - lastTime) / 1000, 0.05);   // tempo real, limitado a 50 ms
  if (initQueue.length) initEntry(initQueue.shift());    // 1 cena nova por frame
  let active = 0;
  for (const e of entries) {
    if (!e.ready || !e.visible) continue;
    if (backgroundPaused && !e.isModal) continue;
    if (e.card && e.card.classList.contains('is-dimmed')) continue;
    if (e.resizePending && !applyResize(e)) continue;
    e.update(dt);  e.renderer.render(e.scene, e.camera);  active++;
  }
  if (!active && !initQueue.length) stop();              // nada a desenhar -> para o loop
}
```

3.4 Reaproveitamento de recursos

- **Cache de geometrias e materiais** (`G.box(...)`, `mat(color, opts)`): um mesmo par forma+material é criado *uma vez* e usado por todas as cenas e navios. Isso reduz memória e evita recompilar shaders. O material da água de cada tema também vem desse cache.
- **InstancedMesh**: peças repetidas (26 luzes do convés, 16 portinholas, 8 canhões, 4 tubos de torpedo, lemes, pás da hélice) viram *um* desenho cada, em vez de dezenas.
- **Renderer do modal criado uma vez** e reutilizado: ao trocar de navio, apenas o grupo 3D é substituído.
- **Canvas opaco** (`alpha:false`): o compositor do navegador não precisa misturar o canvas com o fundo da página.

3.5 Interação

Arrastar para girar usa **Pointer Events** com *pointer capture* (funciona igual para mouse, toque e caneta e não depende de listeners em `window`). Os canvases têm `touch-action: pan-y`: o gesto vertical continua rolando a página; o horizontal gira o navio. Depois de 2,5 s sem tocar, a rotação automática retorna. Toda a suavização usa $k = 1 - e^{-5 \cdot dt}$, portanto independe da taxa de quadros do monitor.

3.6 Enquadramento automático

Quadros "em pé" (como o painel esquerdo do modal, com proporção ≈ 0,85) cortavam o navio. A câmera agora se afasta por `k = √(1,6 / aspecto)` sempre que a proporção fica abaixo de 1,6, mantendo o navio inteiro visível.

3.7 Temas do 3D: dia de sol e noite de guerra (v2.0)

O que muda entre os modos não são os modelos, e sim o **cenário**. Um objeto `THEMES` descreve os dois, e cada cena guarda referências à luz hemisférica, à luz direcional e ao oceano para poder trocá-los sem recriar nada:

Parâmetro	Claro (dia de sol)	Escuro (noite de guerra)
Céu (4 paradas do gradiente)	#3d9fe4 → #86cdf3 → #cdeefb → #cdeefb	#02050b → #0b1c33 → #5a2a14 → #1a0d08 (horizonte em brasa)
Névoa (cor · fator de densidade)	0xcdeefb · ×1	0x140c10 · ×1,25
Água (cor · rugosidade · metalicidade)	0x0e8aa8 · 0,28 · 0,05	0x082733 · 0,22 · 0,25
Luz hemisférica (céu / "chão" · intensidade)	0xeaf6ff / 0xb4d2d6 · 0,66	0x5f80b0 / 0x8a3a14 · 0,95 — o chão alaranjado imita o reflexo dos incêndios
Luz direcional	0xfff0c2 · 1,0 (sol)	0xb4ceff · 1,15 (luar)

- `setTheme(mode)` grava o tema corrente, aplica `applyTheme()` às cenas *já criadas* (fundo, cor da névoa, cores e intensidades das luzes, material da água) e chama `start()` para redesenhar as visíveis. Cenas ainda não criadas nascem no tema certo porque `createScene()` lê o tema corrente.
- A densidade da névoa é recalculada com `fogBase / fogScale`: o enquadramento automático (seção 3.6) afasta a câmera, e a névoa precisa acompanhar essa distância também no tema escuro (fator 1,25).
- As duas texturas de céu são criadas uma única vez (cache por tema): a segunda troca de tema não aloca nada.

4. O hero animado e as demais animações

4.1 Hero em Canvas 2D: um controlador, uma cena por tema

O fundo do hero é um `<canvas id="ocean-canvas">` 2D. Na v1.1 ele era desenhado por uma função única (`initHeroSea()`). Na v2.0 essa função foi dividida em duas partes, **sem mudar uma linha do desenho**: um **controlador** (`window.RNHero`) e a **cena clara**, registrada nele. A cena escura (`war-hero.js`) se registra do mesmo modo. O controlador cuida do que é comum; cada cena só sabe desenhar.

Responsabilidade	Quem faz	Como
Tamanho, DPR (limitado a 1,5) e canvas opaco	RNHero	`ResizeObserver`; ao mudar o tamanho marca as cenas como "sujas" (só a ativa é recalculada na hora)
Pausa fora da tela e com a aba oculta	RNHero	`IntersectionObserver` no hero + `visibilitychange`; sem loop, o quadro fica estático
Movimento reduzido	RNHero	Com `prefers-reduced-motion` o hero é desenhado uma vez, sem laço
Trocar de tema	RNHero.setMode	Escuta `rn:themechange`, troca a cena e pinta um quadro na hora (mesmo pausado)
Preparar o outro tema	RNHero	Quando o ponteiro entra no botão (`rn:prewarm`), a cena do outro tema já monta sprites e gradientes: a troca não engasga
Contrato de uma cena	cada cena	`{ resize(ctx, W, H, dpr), render(ctx, dt) }` — sem laço próprio, sem observers próprios

Atualização v4.1: a cena clara descrita abaixo (função `registerSunnyScene` em main.js) foi **substituída** pelo comboio em perspectiva de `convoy-hero.js` (seção 15). O texto permanece como registro da v1.1/v2.0; o controlador RNHero e o contrato de cena continuam valendo.

A cena clara da v1.1/v2.0 — o dia ensolarado (substituída na v4.1)

O princípio é: **pré-renderizar tudo que não muda** e desenhar por frame só o que se move.

Elemento	Como é feito	Custo por frame
Céu + mar + névoa do horizonte	Pré-renderizados em um canvas fora da tela (refeito só ao redimensionar)	1 `drawImage`
Sol	Sprite radial pré-renderizado; leve pulsar por escala	1 `drawImage`
Nuvens (7)	3 sprites pré-renderizados, deriva horizontal por tempo	7 `drawImage`
Gaivotas (4)	Duas curvas quadráticas; batida de asa por seno	4 traçados curtos
Navio ao fundo e veleiro	Polígonos simples com balanço leve	Poucos preenchimentos
Ondas (5 camadas)	Senoides somadas, gradientes **pré-calculados**, brilho na crista	5 preenchimentos
Brilhos do sol na água (34)	Retângulos finos com cintilação individual	34 retângulos

- **Pausa inteligente:** a animação para quando o hero sai da tela ou a aba fica oculta (agora feito pelo RNHero, para as duas cenas).
- **Tempo real:** o movimento usa *dt* em segundos — um monitor de 144 Hz não anima mais 2,4× mais rápido que um de 60 Hz (como acontecia com `t += 0.016` por frame).
- **Resolução:** DPR limitado a 1,5 e canvas opaco (`alpha:false`).
- **Quadro estático:** com `prefers-reduced-motion`, o hero é desenhado uma vez, sem loop.

4.2 Demais animações e como cada uma é feita

Animação	Implementação	Por que é fluida
Entrada do hero	CSS `@keyframes heroFadeIn` com atrasos escalonados	opacity + transform
Parallax do título	Um handler de scroll (rAF) escreve `translate3d` e opacidade	Sem leitura de layout no scroll; só roda com o hero visível
Navbar ao rolar	Classe *scrolled* (fundo, sombra e cores)	Transição só de cor/sombra — sem blur
Link ativo	IntersectionObserver com faixa no meio da tela	Nenhum trabalho no scroll
Contadores	rAF + easeOutCubic, 1,8 s, formato pt-BR	Baseado em tempo
Revelação ao rolar	IntersectionObserver adiciona *is-visible*; ao fim da transição as classes são **removidas**	opacity/transform; o elemento volta ao CSS normal e o hover funciona
Hover dos cards	CSS `transform` + sombra	Compositor
Filtro de navios	Classe *is-dimmed* + atributo `inert`	Transição no CSS; itens fora do filtro saem do foco
Barras da frota	`scaleX(var(--fill))` com atraso escalonado	Compositor (antes animava *width*)
Modal	Fade do overlay + escala do contêiner; `visibility` escondido quando fechado	Sem blur; some do fluxo de acessibilidade ao fechar
Brilho do cursor	Um `div` movido por `translate3d`; loop para ao alcançar o cursor	Só com mouse; sem blend-mode
v2.0 · Botão de tema	Trilho e botão em CSS: `transform` do "knob" (sol ↔ lua), estrelas que surgem no trilho	transform/opacity; sem JavaScript por quadro
v2.0 · Troca de tema	Revelação circular (View Transitions + clip-path) ou cortina circular de reserva	Feita pelo compositor; ver seção 5.3
v2.0 · Medidor de profundidade	Marcador por `translate3d`; texto atualizado só quando o valor (a cada 10 m) muda	Sem layout por quadro

4.3 Acessibilidade incorporada

- `prefers-reduced-motion`: animações e transições praticamente desligadas; hero estático; sem parallax nem brilho; a troca de tema vira instantânea.
- Modal: foco vai para o botão fechar, **Tab** fica preso dentro do modal, **Esc** fecha e o foco volta ao elemento que abriu.
- Canvases 3D com `role="img"` e `aria-label`; canvas decorativo do hero e o fundo do mar com `aria-hidden`; as faixas submarinas têm `role="region"` e rótulo.
- Foco visível global (contorno âmbar), botões de filtro com `aria-pressed` e link ativo com `aria-current`.
- **Botão de tema:** `role="switch"` com `aria-checked` (verdadeiro no escuro), `aria-label` fixo, `title` que muda conforme o estado e rótulo visível "Claro/Escuro"; alcançável e acionável pelo teclado.

5. Modo escuro: guerra em alto mar (v2.0)

A versão 2.0 acrescenta um segundo modo ao site, escolhido por um botão no canto superior direito. O **modo claro** é o dia ensolarado de sempre — e permanece **exatamente** como era. O **modo escuro** transforma a página numa viagem da superfície ao leito do oceano, em uma noite de guerra. Esta seção documenta como isso foi construído, o que cada arquivo faz e por que a animação continua fluida.

5.1 A jornada: da superfície ao leito do mar

A ideia narrativa é que **rolar a página é afundar**. No topo, duas frotas trocam disparos sob uma tempestade; ao descer, a água escurece, surgem raios de luz, neve marinha e cardumes; no meio do caminho há uma faixa de combate submarino; e, no fim, o visitante chega aos navios que a guerra levou. Um medidor de profundidade acompanha a descida.

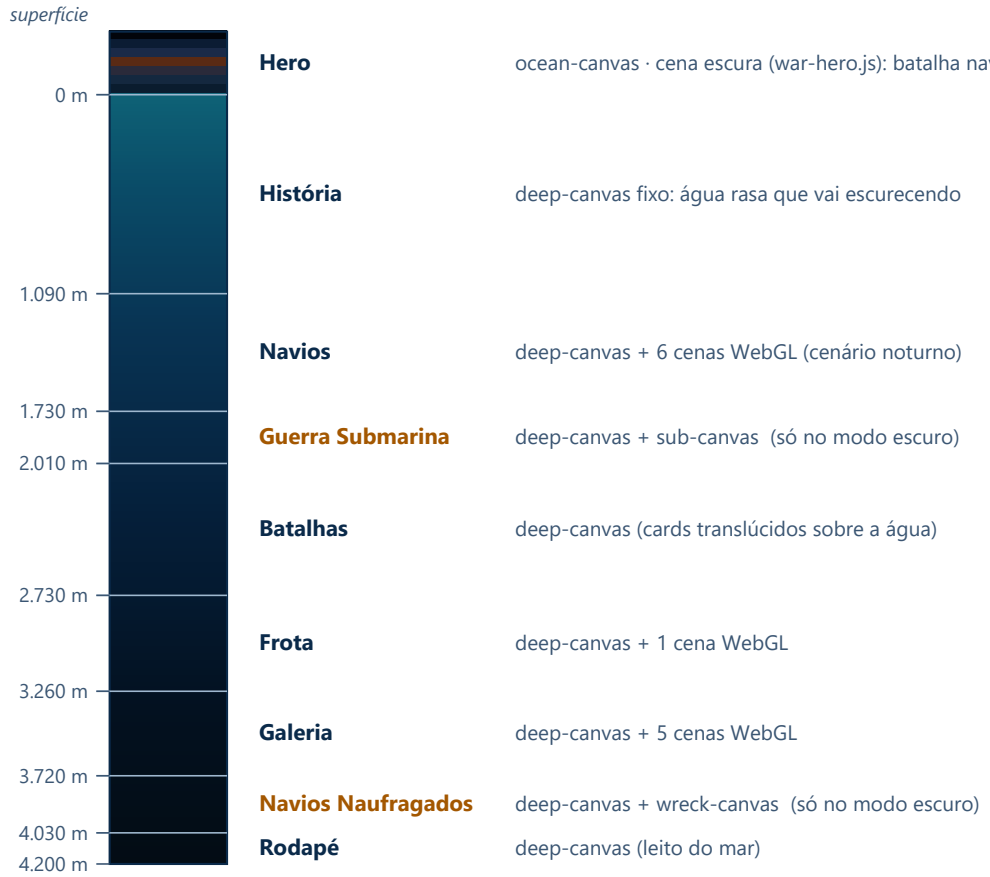


Figura 7 — Mapa da página no modo escuro (proporções reais, medidas a 1366×768: 10.294 px de altura; 8.941 px no modo claro). A cor da barra é a cor da água que o canvas fixo desenha em cada profundidade. Os valores à esquerda são a profundidade mostrada pelo medidor quando o *centro da tela* está naquele ponto. Em laranja, as duas faixas que só existem no modo escuro.

Trecho	Profundidade mostrada	O que o visitante vê	Onde está no código
Hero	— (medidor oculto)	Batalha naval noturna: 4 navios trocam tiros; explosões, incêndios, relâmpagos, holofotes; navios afundam e voltam	war-hero.js
História	≈ 170 → 920 m	Água turquesa-escura com raios de luz, neve marinha, bolhas, cardumes e submarinos distantes	deep-sea.js (Deep)
Navios	≈ 1.250 → 1.560 m	Os 6 modelos 3D sob luar; a água já é azul-marinho	ships-3d.js + Deep

Trecho	Profundidade mostrada	O que o visitante vê	Onde está no código
Guerra Submarina	≈ 1.870 m	Dois submarinos (verde-água × vermelho) trocam torpedos; sonares; cargas de profundidade; um terceiro observa ao fundo	deep-sea.js (subWar)
Batalhas	≈ 2.180 → 2.560 m	Cards translúcidos sobre a água; sem as ondas claras do modo claro	dark.css + Deep
Frota · Galeria	≈ 2.900 → 3.550 m	Cenas 3D em formação; criaturas luminosas surgem nas profundezas	ships-3d.js + Deep
Navios Naufragados	≈ 3.870 m	Destroieres partidos, um galeão, um submarino e um porta-aviões no leito; algas, corais, cardumes, baliza de socorro	deep-sea.js (wrecks)
Rodapé	até ≈ 4.030 m	Leito do mar	dark.css

A profundidade é **narrativa**: vai de 0 a 4.200 m em proporção à rolagem (o medidor usa o centro da tela; no fim da página marca ≈ 4.030 m). As zonas seguem os limites oceanográficos usuais (epipelágica até 200 m, mesopelágica até 1.000 m, batipelágica até 4.000 m, abissal a partir daí).

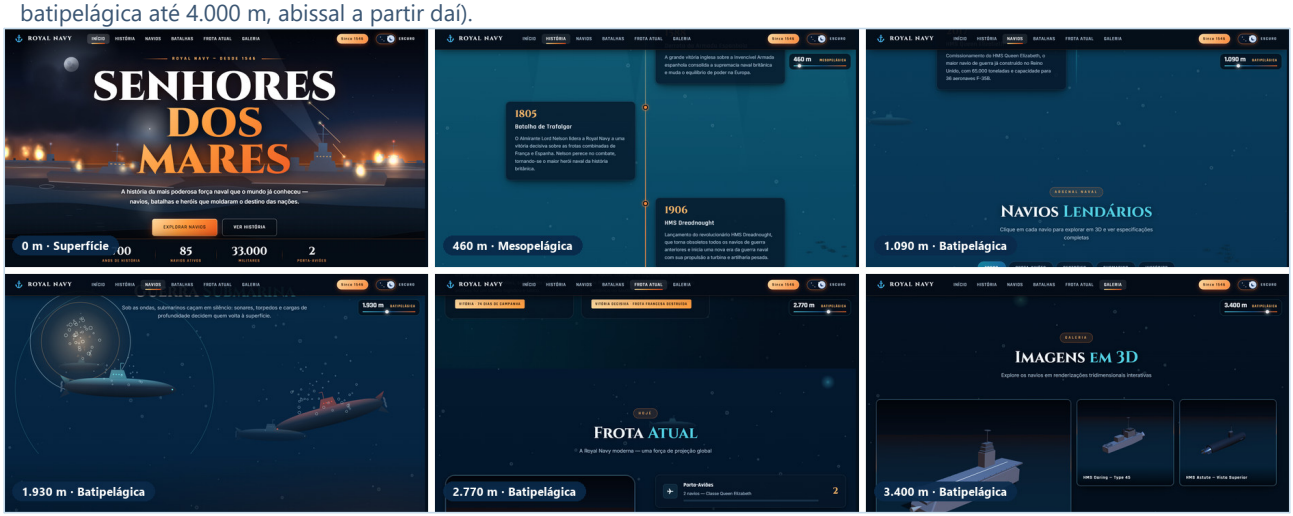


Figura 8 — O mergulho em seis pontos da página (modo escuro, 1366×768): da batalha na superfície até 3.400 m (batipelágica). O medidor à direita mostra metros e zona.

5.2 Como o tema funciona

O tema é um **atributo no elemento raiz**: `<html data-theme="light">` ou `<html data-theme="dark">`. Tudo que é específico do escuro se ativa por esse atributo — no CSS, com seletores `[data-theme="dark"] …`; no JavaScript, lendo o atributo ou escutando um evento. Quatro peças cooperam:

Peça	Arquivo	Papel
Atributo `data-theme`	<html>	Fonte única da verdade sobre o tema atual. O padrão é `light`.
Aplicação inicial	theme-init.js (11 linhas)	Script *síncrono* no <head>: lê `localStorage['rn-theme']`, define `data-theme` e a cor da barra do navegador (`meta theme-color`) **antes da primeira pintura** — sem "piscar" claro→escuro. É um arquivo externo (e não inline) por causa da CSP.
Controle	theme.js (140 linhas)	`window.RNTheme = { get, set, toggle }`: valida, anima a troca, atualiza botão e armazenamento e dispara o evento `rn:themechange`. Também abriga o `RNPerf` (seção 5.8).
Reações	main.js · deep-sea.js · ships-3d.js · CSS	Cada módulo escuta o evento (ou apenas o atributo) e faz só o seu trabalho — não há um "gerente de tema" que conheça todos.

O que acontece ao clicar

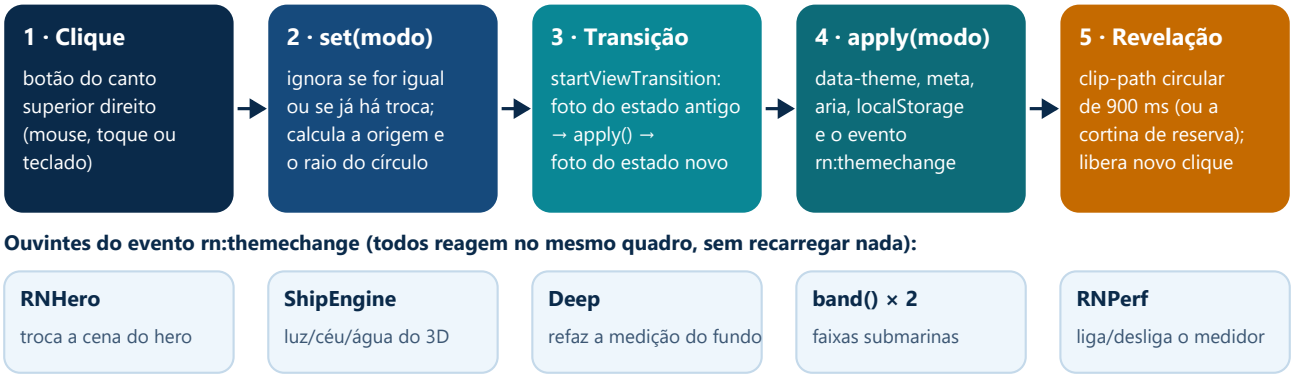


Figura 9 — Sequência de uma troca de tema. O mesmo caminho vale para o clique, o teclado e para chamadas programáticas (`RNTheme.set('dark')`).

Ouvinte	Arquivo	O que faz ao receber `rn:themechange`
RNHero.setMode	main.js	Troca a cena do hero (clara ↔ escura) e pinta um quadro na hora, mesmo com o hero pausado
ShipEngine.setTheme	ships-3d.js	Atualiza céu, névoa, luzes e água das cenas 3D já criadas e redesenha as visíveis
Deep (remeasure)	deep-sea.js	No escuro: mede a altura da página e do hero e liga o laço do fundo se houver rolagem. No claro: só desliga
band() × 2	deep-sea.js	Recalcula o tamanho dos canvases das faixas (que passam de `display:none` a visíveis) e liga/desliga o laço
RNPerf (syncPerf)	theme.js	Liga o medidor de tempo de quadro só enquanto o tema é escuro
CSS	style.css · dark.css	Seletores por atributo: cores, sombras, fundos, botões, medidor, faixas — tudo muda sem uma linha de JavaScript

Persistência e padrões

- A escolha fica em `localStorage['rn-theme']` (`'light'` ou `'dark'`). Toda leitura/gravação está em `try/catch`: em janela privada ou com o armazenamento bloqueado o site simplesmente abre no claro.
- O padrão é **sempre o claro**; a preferência do sistema operacional não é consultada (ver decisões, seção 2.6).
- A cor da barra do navegador no celular (`theme-color`) acompanha: #2a9bdc no claro, #050b14 no escuro.
- O botão reflete o estado em `aria-checked`, no `title` (“Ativar o modo escuro (guerra no mar)” / “Voltar ao modo claro”) e no rótulo “Claro/Escuro”.

```
// theme.js — RNTheme.apply(): o único lugar que muda o tema
function apply(mode) {
  root.setAttribute('data-theme', mode);                    // 1) CSS reage sozinho
  if (meta) meta.setAttribute('content', COLORS[mode]);     // 2) cor da barra do navegador
  if (btn) { btn.setAttribute('aria-checked', String(mode === 'dark')); btn.title = /* ... */; }
  if (label) label.textContent = mode === 'dark' ? 'Escuro' : 'Claro';
  try { localStorage.setItem(KEY, mode); } catch (e) { /* ignora */ }
  document.dispatchEvent(new CustomEvent('rn:themechange', { detail: { theme: mode } }));  // 3) avisa os módulos
}
```

5.3 A animação de transição e o botão

Revelação circular. Ao clicar, o novo tema "nasce" do botão e se expande em círculo até cobrir a tela. Isso usa a **View Transitions API** do navegador: ele tira uma "foto" da página no estado antigo, executa a troca de tema e tira uma "foto" do estado novo; as duas ficam empilhadas como pseudo-elementos `::view-transition-old(root)` e `::view-transition-new(root)`. O site desliga a animação padrão (um esmaecimento) e anima só a camada nova com `clip-path: circle()` — de raio 0, no centro do botão, até o raio que alcança o canto mais distante da tela — durante **900 ms** com `cubic-bezier(.4, 0, .2, 1)`. Quem faz o trabalho é o compositor do navegador; o JavaScript apenas dispara a animação.



Figura 10 — A revelação circular congelada em quatro instantes (animação de 900 ms; o tempo é o da animação, com o easing aplicado). O modo escuro nasce no canto do botão e a batalha já anima dentro do círculo.

```
// theme.js — RNTheme.set() (trecho simplificado)
function set(mode, opts = {}) {
  if (mode !== 'dark' && mode !== 'light') return;
  if (mode === get() || busy) return;                    // igual ao atual, ou troca em andamento
  if (opts.instant || REDUCED || !btn) { apply(mode); return; }

  const r = btn.getBoundingClientRect();
  const x = r.left + r.width / 2, y = r.top + r.height / 2;                    // origem = centro do botão
  const radius = Math.hypot(Math.max(x, innerWidth - x), Math.max(y, innerHeight - y));  // até o canto mais longe

  busy = true;
  if (typeof document.startViewTransition === 'function') {
    root.classList.add('theme-switching');
    const vt = document.startViewTransition(() => apply(mode));                // foto antiga → troca → foto nova
    vt.ready.then(() => root.animate(
      { clipPath: [`circle(0px at ${x}px ${y}px)`, `circle(${radius}px at ${x}px ${y}px)`] },
      { duration: 900, easing: 'cubic-bezier(.4,0,.2,1)', pseudoElement: '::view-transition-new(root)' }));
    vt.finished.then(finish, finish);                                           // libera novo clique
  } else {
    veilTransition(mode, x, y, radius, finish);                                 // cortina circular de reserva
  }
}
```

Situação	Comportamento
Navegador com View Transitions	Revelação circular acima. Enquanto dura, `<html>` recebe a classe `theme-switching` e novos cliques são ignorados (flag `busy`).
Navegador **sem** a API	**Cortina de reserva:** um `<div class="theme-veil">` (cor do tema de destino) cresce em círculo por 520 ms, o tema é aplicado quando a tela está coberta e a cortina se desfaz em 380 ms. Testada simulando a ausência da API.
prefers-reduced-motion	Sem animação: a troca é instantânea.
Chamada programática	`RNTheme.set('dark', { instant: true })` aplica na hora (usado nos testes).
Mouse chega ao botão	Evento único `rn:prewarm` (também com foco por teclado ou toque): o `RNHero` já monta os sprites e gradientes da cena do *outro* tema, para a troca não engasgar.

O botão

O botão é um `<button role="switch">` com trilho e "bolinha" desenhados em CSS: no claro o trilho é azul-céu e a bolinha é um **sol** à esquerda; no escuro o trilho vira azul-noite com borda e brilho cor de brasa, surgem **estrelas** no trilho e a bolinha desliza 32 px para a direita transformando-se em **lua** (os dois ícones SVG trocam de opacidade, giro e escala). Tudo por `transform` e `opacity`. Abaixo de 1100 px de largura o rótulo "Claro/Escuro" some e só o botão fica; no celular ele vai para a extremidade direita da barra.

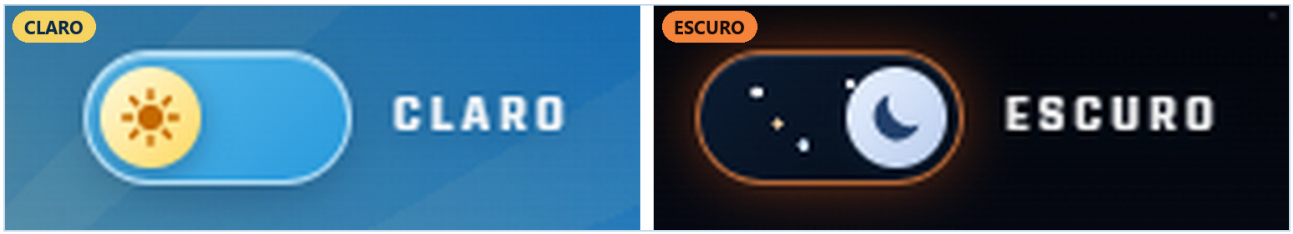


Figura 11 — O botão de tema ampliado 5× nos dois estados (recortes das capturas do hero).

5.4 O CSS do modo escuro (dark.css)

Todo o visual do modo escuro está em **um arquivo**, `dark.css`, cujos seletores começam por `[data-theme="dark"]` (ou são classes que só existem no escuro). O `style.css` ganhou **116 linhas novas e nenhuma alterada** — botão, transição e a regra que oculta, no modo claro, os elementos exclusivos do escuro. É por isso que o claro continua idêntico (comprovado na seção 9.5).

Bloco do dark.css	O que define
Tokens (`:root[data-theme="dark"]`)	Variáveis: `--scene-bg` (fundo das cenas 3D), `--grad-fire` (gradiente de brasa nos títulos), `--d-ember` #ff8a3d, `--d-ember-1` #ffb86b, `--d-cyan` #54d1e4, painéis `--d-panel` translúcidos, textos `--d-tx/-tx2/-tx3`, sombras. Também `color-scheme: dark` (barras de rolagem e controles nativos escuros).
Base e componentes	Corpo, seleção, foco, barra de rolagem, botões, filtro, navbar (rolada), selo, cards de linha do tempo/navios/batalhas/frota/galeria, modal, rodapé, brilho do cursor (vira clarão de incêndio).
Hero	Fundo de noite, vinheta central reduzida (para a batalha aparecer), brilho de incêndio no horizonte (`fireGlow`: só a opacidade anima), título com gradiente de brasa, barra de estatísticas escura.
Seções sobre o mar	História, Navios, Frota e Galeria ficam com `background: transparent`: quem dá a cor é o canvas fixo do mar profundo; os cards usam painéis translúcidos por cima.
Faixas submarinas	`.abyss-band` (altura `clamp(440px, 84vh, 780px)`; naufrágios `clamp(520px, 92vh, 860px)`), legenda sobre o canvas e um degradê que encosta o fundo no rodapé.

Bloco do dark.css	O que define
Medidor de profundidade	HUD fixo (`top: 82px; right: 16px`; 74/10 px no celular), `pointer-events: none`, entra com fade e desliza; marcador movido por `translate3d`.
Batalhas e rodapé	As ondas divisórias claras somem (`display: none`) e o fundo vira um degradê escuro translúcido.

No modo claro, três elementos existem no HTML mas ficam ocultos por `.deep-canvas, .depth-gauge, .abyss-band { display: none }` (regra em style.css): o canvas do mar profundo, o medidor e as duas faixas. Como `display:none` zera o tamanho, os canvases nem chegam a ser dimensionados nem a executar laços.

5.5 O hero em batalha (war-hero.js)

A cena escura do hero é **Canvas 2D** puro (741 linhas). Ela se registra no controlador com `RNHero.register('dark', createWarScene())` e obedece ao contrato `{ resize(ctx, W, H, dpr), render(ctx, dt) }`: não tem laço, observers nem listeners próprios — o controlador cuida de pausa, visibilidade e tamanho.

Os atores

Navio	Lado	Posição (x)	Comprimento	Intervalo entre salvas	Acertos p/ afundar
Porta-aviões	1 (luz azul)	7,5%	22% da largura	6–9 s (1 tiro)	9
Destróier	1 (luz azul)	26,5%	33%	1,2–2,2 s (até 2 tiros)	7
Couraçado	2 (luz vermelha)	73,5%	35%	1,4–2,4 s (até 2 tiros)	8
Cruzador	2 (luz vermelha)	92,5%	21%	2,4–4 s (até 2 tiros)	6

Porta-aviões e cruzador ficam ao *fundo* (desenhados atrás das duas primeiras camadas de onda); destróier e couraçado, à frente. Em telas estreitas (largura < 600 px) os navios são ampliados ×1,34.

O ciclo de combate

- **Disparo.** Cada navio sorteia um inimigo ainda ativo e um ponto do casco; sai um *clarão de boca* (branco + laranja), fumaça e o projétil parte em **arco parabólico** (0,85–1,35 s). Há 55% de chance de acerto; se errar, cai na água perto do alvo.
- **Acerto.** Explosão (35% delas "grandes") com clarões, chamas, faíscas e fumaça; o alvo soma um acerto e ganha um **incêndio a bordo** (até 4 por navio, 7–12 s cada) que continua soltando chamas e fumaça. **Erro:** anel de onda + coluna d'água (gotas) + clarão azulado.
- **Naufrágio.** Ao atingir o limite de acertos o navio *afunda em 9 s* (desce 15% do comprimento, inclina até 0,4 rad, some no fim, soltando fumaça); depois fica *ausente 4,5 s* e **volta ao combate** com um esmaecimento de 2 s. A batalha nunca acaba.
- **Atmosfera.** Relâmpago a cada 4–9 s (traço irregular com ramificação e clarão no céu); *sinalizador de iluminação* a cada 7–11 s (sobe em 1 s e desce em 8 s, iluminando a cena e o mar); 2–3 **holofotes** que varrem o céu; 16 brilhos da lua na água; clarões distantes no horizonte; um "clarão geral" que decai (até 0,3) a cada tiro e explosão.

```
// war-hero.js — máquina de estados de cada navio (trecho simplificado de update())
if (s.state === 'ok') {
  s.cool -= dt;
  if (s.cool <= 0) { s.cool = rnd(s.rate[0], s.rate[1]); fire(s); }            // atira
  if (s.hits >= s.dmgMax) { s.state = 'sinking'; s.sinkT = 0; }
} else if (s.state === 'sinking') {
  s.sinkT = Math.min(1, s.sinkT + dt / 9);                                      // 9 s afundando
  s.sinkY = s.sinkT * s.L * 0.15;  sinkRoll = s.sinkT * 0.4 * s.dir;
  s.alpha = 1 - clamp((s.sinkT - 0.78) / 0.22, 0, 1);                           // some no fim
```

```
    if (s.sinkT >= 1) { s.state = 'gone'; s.gone = 4.5; ring(s.x, s.y); }
  } else if (s.state === 'gone') {
    s.gone -= dt;
    if (s.gone <= 0) { s.state = 'ok'; s.hits = 0; s.alpha = 0; s.fade = 1; }       // volta ao combate
  }
```



Figura 12 — Três quadros da batalha (recortes 1366×440, alguns segundos entre eles): incêndios a bordo, projéteis em arco, traçantes, clarões e holofotes.

Como a cena custa pouco

Técnica	Onde	Efeito
Pré-renderização	céu com 90 estrelas, lua com halo e crateras, incêndios do horizonte, mar; silhuetas dos navios; sprites de brilho, fumaça, feixe e nuvens de tempestade	Feita *uma vez* por redimensionamento; por quadro é só `drawImage`.
Silhuetas com luz	cada navio é um sprite com luar por cima e brilho de incêndio por baixo, aplicados com `source-atop` (só nos pixels já desenhados)	Aparência de iluminação sem custo por quadro.
Brilhos aditivos	explosões, chamas, clarões, projéteis, luzes de navegação (`globalCompositeOperation = 'lighter'`)	Sprites radiais reaproveitados em vez de gradientes criados a cada frame.
Pools de tamanho fixo	partículas 150 · clarões 28 · anéis 10 · projéteis 26 · traçantes 18	Nenhuma alocação por quadro: sem pausas do coletor de lixo.
Ondas	5 camadas com gradientes pré-calculados (um por camada)	Senoides somadas; 5 preenchimentos por quadro.
Tempo real	tudo por `dt` (limitado a 50 ms)	Mesma velocidade em 60 Hz ou 144 Hz.
Escala	`S = clamp(W / 1366, 0.6, 1.35)`	Tamanhos, velocidades e efeitos proporcionais à tela.

```
// war-hero.js — partículas em pool: nunca cria objetos durante a batalha
const CAP = [44, 84, 150];                                    // teto ativo por nível do RNPerf
const P = Array.from({ length: 150 }, () => ({ a: 0, type: 0, x: 0, y: 0, vx: 0, vy: 0, life: 0, max: 1, size: 10, y0: 0 }));
function spawn(type, x, y, vx, vy, life, size, y0) {
  const cap = CAP[level()];
  for (let k = 0; k < cap; k++) {
    pIdx = (pIdx + 1) % cap;
```

```
    const p = P[pIdx];
    if (!p.a) { p.a = 1; p.type = type; p.x = x; p.y = y; /* ... */ return p; }
  }
  return null;                                                  // sem vaga: descarta em vez de alocar
}
```

5.6 O mergulho, a guerra submarina e os naufrágios (deep-sea.js)

O segundo arquivo do modo escuro (791 linhas) tem quatro módulos independentes. Nenhum usa laço próprio quando está invisível.

Módulo	Canvas / elemento	Função
Gauge	#depth-gauge (HTML)	Converte a rolagem em metros e zona; move o marcador
Deep	#deep-canvas (fixo, atrás de tudo)	A água: cor por profundidade, superfície, raios de luz, neve, bolhas, cardumes, submarinos distantes, criaturas luminosas
subWar	#sub-canvas (faixa "Guerra Submarina")	Submarinos, torpedos, sonares, cargas de profundidade, explosões
wrecks	#wreck-canvas (faixa "Navios Naufragados")	Naufrágios no leito, algas, cardumes, bolhas, baliza de socorro

O fundo do mar (Deep)

Um único `<canvas>` **fixo**, atrás do conteúdo, desenhado em **meia resolução** (`SC = 0.5`: um quarto dos pixels) e opaco. A profundidade de cada ponto da página é a fração *(y − altura do hero) ÷ (altura da página − altura do hero)*; a cor da água é interpolada entre seis paradas; cada quadro pinta um degradê vertical que cobre exatamente o trecho da página visível naquele instante.

Profundidade (fração)	Cor da água	Elementos ativos
0 (logo abaixo do hero)	rgb(14, 98, 118) · #0e6276	Linha d'água ondulante e 4 raios de luz (força □ (1 − d)^{1,7}); cardumes (d < 0,55)
0,12	rgb(10, 76, 104) · #0a4c68	Neve marinha (opacidade 0,28 + 0,4·d) e bolhas subindo, com paralaxe
0,30	rgb(8, 52, 84) · #083454	3 submarinos distantes (28% de opacidade) com clarões e ondas de choque de combates ao longe, a cada 5–10 s
0,55	rgb(5, 31, 58) · #051f3a	Criaturas luminosas (7) surgem a partir de d > 0,35, com brilho ciano ou verde
0,80	rgb(3, 17, 32) · #031120	Neve mais densa; quase sem luz
1 (fim da página)	rgb(3, 12, 20) · #030c14	Leito

```
// deep-sea.js — a profundidade é a posição na página
const dAt = y => clamp((y - heroH) / Math.max(1, docH - heroH), 0, 1);   // 0 = abaixo do hero · 1 = fim
const d0 = dAt(scrollY), d1 = dAt(scrollY + innerHeight);
for (let i = 0; i <= 4; i++) g.addColorStop(i / 4, colAt(d0 + (d1 - d0) * (i / 4)));   // água entre topo e base da tela

// só roda quando faz sentido:
const want = isDark() && !document.hidden && window.scrollY > 2;        // no hero (scrollY≈0) ele nem desenha
```

O medidor de profundidade

O valor em metros é `round(d × 4200 / 10) × 10` com d calculado no *centro* da tela; o texto só é reescrito quando muda (a cada 10 m) e o marcador se move por `translate3d`. As zonas do medidor são: 0 m Superfície · 30 m Epipelágica · 200 m Mesopelágica · 1.000 m Batipelágica · 4.000 m Abissal. O HUD só aparece depois que a página desce mais de 40% da altura da tela.

Faixas com canvas próprio: o utilitário band()

As duas faixas usam a mesma função auxiliar, que garante que o canvas **só anima quando precisa**: com tema escuro, faixa a menos de 80 px da tela, aba visível e sem *reduced-motion*. O canvas é transparente (a água é a do fundo fixo) e usa DPR limitado a 1,25.

```
// deep-sea.js — band(): o laço só existe enquanto o canvas está visível no modo escuro
function sync() {
  const want = isDark() && visible && !document.hidden && !REDUCED;
  if (want && !raf) { last = performance.now(); raf = requestAnimationFrame(frame); }
  else if (!want && raf) { cancelAnimationFrame(raf); raf = 0; }
}
new IntersectionObserver(([e]) => { visible = e.isIntersecting; sync(); }, { rootMargin: '80px 0px' }).observe(box);
new ResizeObserver(resize).observe(canvas);                    // no claro o canvas tem tamanho 0: resize() sai logo
```

Guerra Submarina (subWar)

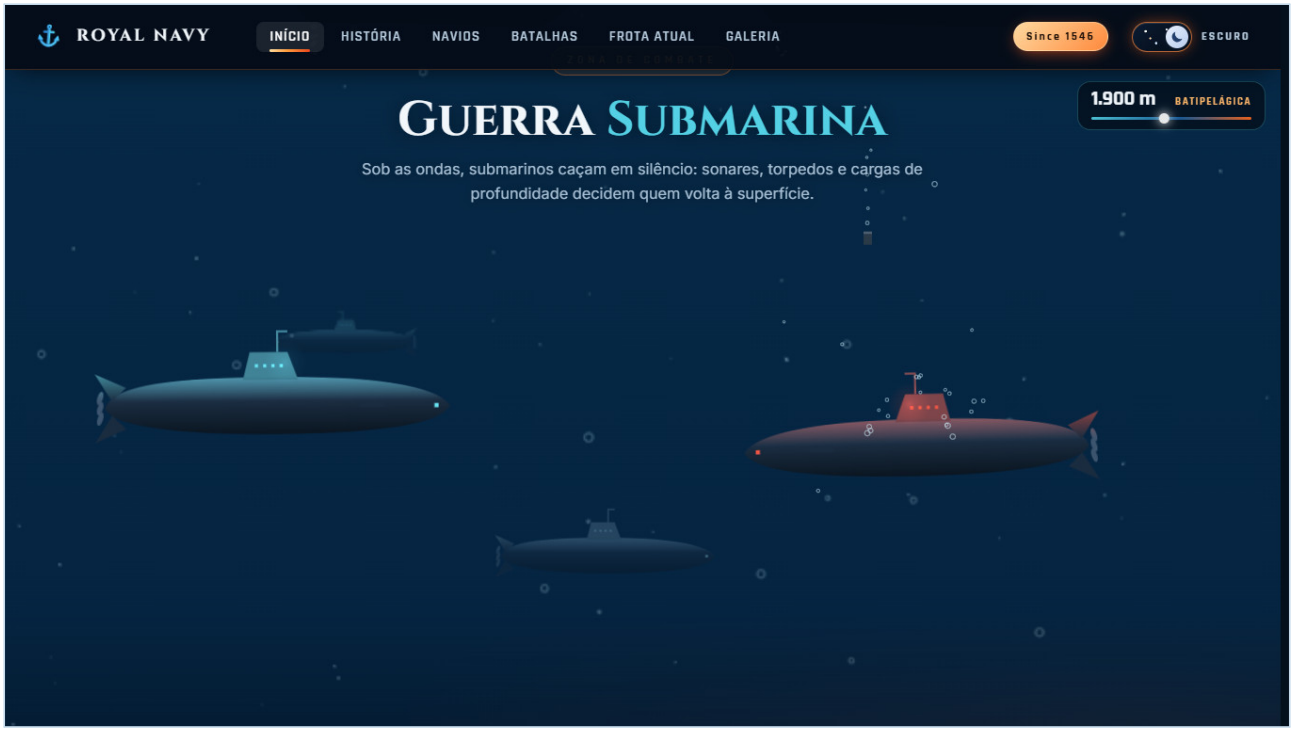


Figura 13 — Faixa "Guerra Submarina": submarino verde-água (esquerda) e vermelho (direita) em combate, um terceiro ao fundo; bolhas, sonar e o medidor em 1.900 m.

Elemento	Regra
Submarinos	2 combatentes (verde-água, sentido → · vermelho, sentido ←) e 1 observador distante (42% de opacidade, não combate); cada um oscila lentamente e tem 2 pontos de vida
Torpedos	Saem a cada 2,3–3,8 s de cada combatente, a 0,55 largura/s, com esteira de bolhas; 55% acertam; erros passam perto e dão clarão + anel
Sonar	A cada 3,2–4,4 s cada combatente emite um anel que cresce até 42% da largura
Cargas de profundidade	A cada 3,2–5,5 s uma carga cai de um dos 2 cascos de superfície (70–100 px/s) e explode entre 42% e 74% da altura: clarão, 2 ondas de choque, bolhas, destroços e **tremor** da cena
Avaria e naufrágio	Um acerto faz o submarino vazar bolhas por 6 s; o segundo o afunda (3,4 s, inclinando e esmaecendo); após 2,6 s ausente ele **reaparece** — como na batalha da superfície, o combate é contínuo
Pools	torpedos 8 · bolhas 150 (teto 60/100/150 pelo RNPerf) · anéis 18 · clarões 10 · cargas 6 · destroços 30

Navios Naufragados (wrecks)



Figura 14 — Faixa “Navios Naufragados”: cascos partidos no leito, algas, cardumes e a baliza de socorro (ponto vermelho); o medidor marca 3.890 m (zona abissal).

Elemento	Como é feito
Naufrágios (5 sprites)	Desenhados por polígonos quebrados, com cavernas, ferrugem e corais aplicados por `source-atop` e “névoa” de profundidade: porta-aviões (ao fundo, 50% de opacidade), proa e popa de destróier, galeão (inclinado 0,34 rad, mastros e velas rasgadas) e submarino
Aleatoriedade repetível	Um gerador pseudoaleatório com *semente fixa* por sprite: o desenho é sempre o mesmo a cada carregamento
Leito	Areia com pedras e conchas, uma âncora e canhões espalhados, pré-renderizados uma vez e desenhados *na frente* da base dos cascos
Vida	20+ algas (2 traçados no total, um por cor) que balançam; 2 cardumes; bolhas saindo de 3 pontos dos cascos; neve marinha; 3 raios de luz muito fracos
Baliza	Luz vermelha piscando no destróier, com um anel de sonar a cada 4,2 s

5.7 O 3D em cenário noturno

Os modelos são os mesmos; o que muda é o cenário (luar, névoa escura, água escura, reflexo alaranjado dos incêndios), como detalhado na seção 3.7. Os painéis, o modal e os rótulos ganham o visual escuro por CSS.

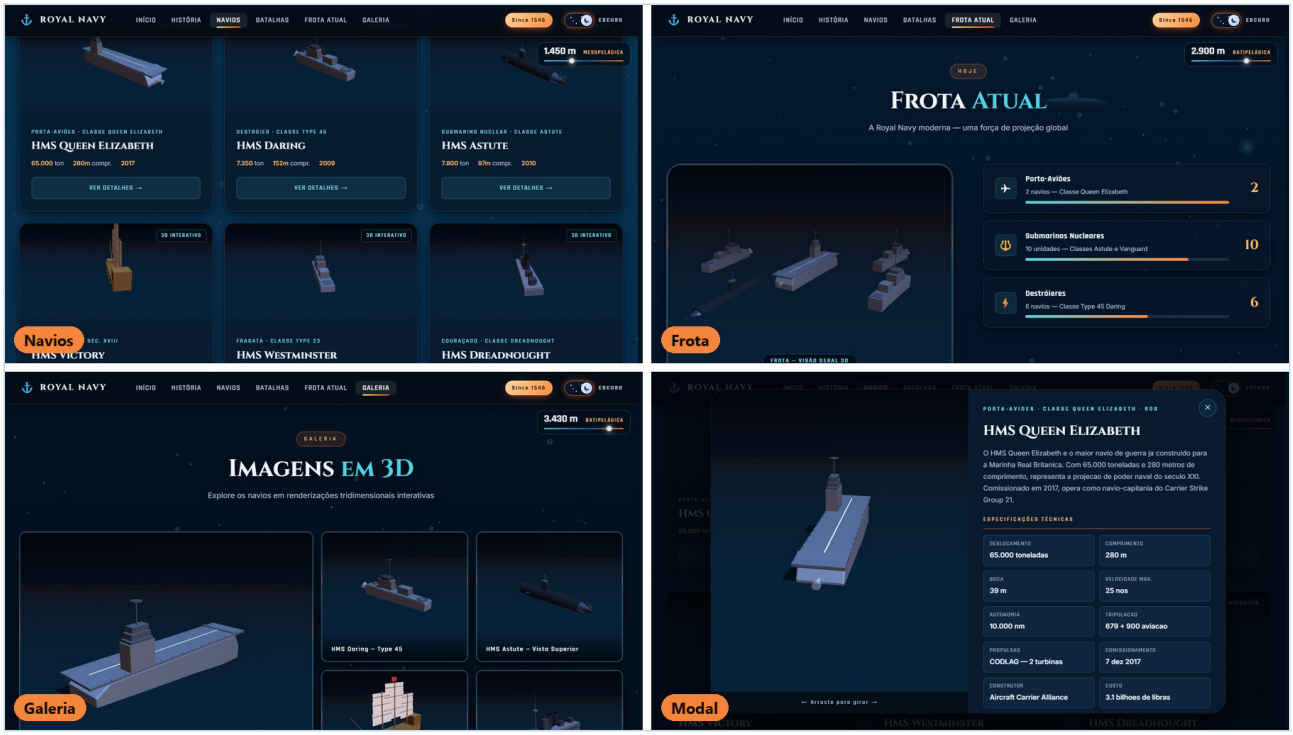


Figura 15 — Demais seções no modo escuro (1366×768, reduzidas): grade de navios, frota, galeria e o modal do HMS Queen Elizabeth.

5.8 Desempenho do modo escuro

O modo escuro tem muito mais coisas em movimento que o claro — e mesmo assim precisa continuar fluido. A regra de ouro é: **nada roda se não estiver sendo visto**.

Canvas	Roda quando…	Resolução	Observação
ocean-canvas (hero)	hero visível · aba visível · sem reduced-motion (a cena depende do tema)	CSS × DPR (≤ 1,5)	Um só laço para as duas cenas
deep-canvas (fundo)	**tema escuro** · aba visível · rolagem > 2 px	50% (meia resolução)	No claro nem mede o layout (`display:none`)
sub-canvas	tema escuro · faixa a ≤ 80 px da tela · aba visível	CSS × DPR (≤ 1,25)	Transparente sobre o fundo
wreck-canvas	tema escuro · faixa a ≤ 80 px da tela · aba visível	CSS × DPR (≤ 1,25)	Transparente sobre o fundo
Amostrador RNPerf	tema escuro · aba visível	—	Um `requestAnimationFrame` leve
Cenas 3D	regras da seção 3.3 (inalteradas)	—	Trocar de tema não recria nada

Técnicas usadas

- **Pré-renderizar o que não muda** (céu, mar, silhuetas, brilhos, leito, corais) e desenhar por quadro só o que se move.
- **Sprites em vez de gradientes por quadro:** brilhos, fumaça e feixes são imagens radiais criadas uma vez e reutilizadas (com mistura aditiva).
- **Pools de tamanho fixo** para partículas, clarões, anéis, torpedos, bolhas e destroços: zero alocação por quadro.
- **Meia resolução** para o fundo fixo: a cor e as partículas suaves não precisam de todos os pixels.
- **Só o necessário no modo claro:** elementos do escuro com `display:none`

- **Tempo real (dt)** e teto de 50 ms por quadro; DPR limitado; canvases opacos quando possível; um único traçado para todas as bolhas e outro por cor de alga.
- **Compositor:** o medidor e o botão só animam `transform` e `opacity`; o brilho de incêndio do hero anima só a opacidade.
- **Pré-aquecimento:** a cena do outro tema é preparada quando o mouse chega ao botão.

Qualidade adaptativa: RNPerf

Máquinas diferentes têm capacidades diferentes. O `RNPerf` (em theme.js) mede o tempo médio de quadro em janelas de 45 quadros, **somente no modo escuro**, e ajusta um nível de qualidade `RNPerf.level` (2 alto, 1 médio, 0 baixo) que as cenas consultam a cada quadro:

Regra	Valor
Descer um nível	média da janela > 26 ms (menos de ≈ 38 fps), nível > 0 e fora do período de espera (2,5 s)
Subir um nível	média < 18,5 ms (≈ 54+ fps) por 14 s seguidos, nível < 2 e fora da espera (4 s)
Ignorado	quadros > 250 ms (aba voltou do segundo plano)

Efeito	Nível 2 (alto)	Nível 1 (médio)	Nível 0 (baixo)
Hero: partículas ativas	150	84	44
Hero: explosões e respingos (fator)	1,0	0,8	0,5
Hero: fogo e fumaça a bordo (fator)	1,0	0,65	0,35
Hero: traçantes · sinalizador de iluminação	sim · sim	sim · sim	não · não
Hero: holofotes · brilhos da lua · clarão geral	sim	sim	não
Hero: nuvens de tempestade	5	5	3
Mar profundo / naufrágios: neve e bolhas	100%	70%	40%
Mar profundo / naufrágios: raios de luz e cardumes	sim	sim	não
Submarinos: teto de bolhas · esteira dos cascos	150 · sim	100 · sim	60 · não

```
// theme.js — RNPerf (simplificado): janelas de 45 quadros, histerese e período de espera
function sample(now) {
  raf = requestAnimationFrame(sample);
  const dt = now - last; last = now;
  if (dt > 250) return;                                   // aba voltou do segundo plano: ignora
  acc += dt; if (++n < 45) return;
  const avg = acc / n; acc = 0; n = 0;
  cooldown = Math.max(0, cooldown - 45 * avg);
  if (avg > 26 && Perf.level > 0 && cooldown === 0) { Perf.level--; cooldown = 2500; calmMs = 0; }     // lento
  else if (avg < 18.5) { calmMs += 45 * avg;                                                            // folgado
    if (calmMs > 14000 && Perf.level < 2 && cooldown === 0) { Perf.level++; cooldown = 4000; calmMs = 0; } }
  else calmMs = 0;
}
```

Custo de carga

Arquivo novo	Bruto	Com gzip*	Como carrega
theme-init.js	0,5 KB	0,4 KB	síncrono, no head (11 linhas)
dark.css	18,4 KB	4,2 KB	no head; bloqueia a pintura, como todo CSS
theme.js	6,1 KB	2,4 KB	defer
war-hero.js	33,7 KB	9,7 KB	defer
deep-sea.js	43,3 KB	12,6 KB	defer
Total dos 5 arquivos	**102,0 KB**	**29,3 KB**	
Crescimento dos arquivos existentes	+ 12,4 KB	+ 3,3 KB	style.css, main.js, ships-3d.js e index.html

* Estimado com `gzip -9` local; o GitHub Pages usa a sua própria compressão. Soma dos arquivos servidos (sem o three.js, que vem do CDN): ≈ 34,7 KB → ≈ 67,3 KB com gzip. Esse custo existe mesmo para quem nunca ativa o modo escuro. Resultados medidos de FPS: seção 9.3.

5.9 Acessibilidade, segurança e compatibilidade

- **CSP intacta:** a política não foi afrouxada. Como ela proíbe scripts inline, o aplicador de tema é um arquivo externo (`theme-init.js`); nenhuma origem externa nova foi adicionada; a validação com o domínio Log do DevTools não registrou violações.
- **Botão acessível:** `<button role="switch">` com `aria-checked`, foco visível (contorno âmbar) e acionável por teclado.
- **Canvases decorativos** com `aria-hidden` (fundo do mar, medidor); faixas com `role="region"` e `aria-label`; canvases das faixas com `role="img"` e descrição.
- **Movimento reduzido:** com `prefers-reduced-motion` não há laços (o hero e o fundo são desenhados como quadro estático), a troca de tema é instantânea e as animações CSS ficam praticamente desligadas.
- **Contraste no escuro:** textos entre 6,5:1 e 17,4:1 sobre o pior painel (medido, seção 8).
- **Compatibilidade:** o suporte à View Transitions API varia por navegador e versão; por isso existe a cortina de reserva. Os testes foram feitos em Chromium/Edge; Firefox e Safari não foram executados (seção 9.2).
- **Sem novas dependências:** nada de bibliotecas — tudo é JavaScript e CSS puros, na mesma linha do restante do projeto.

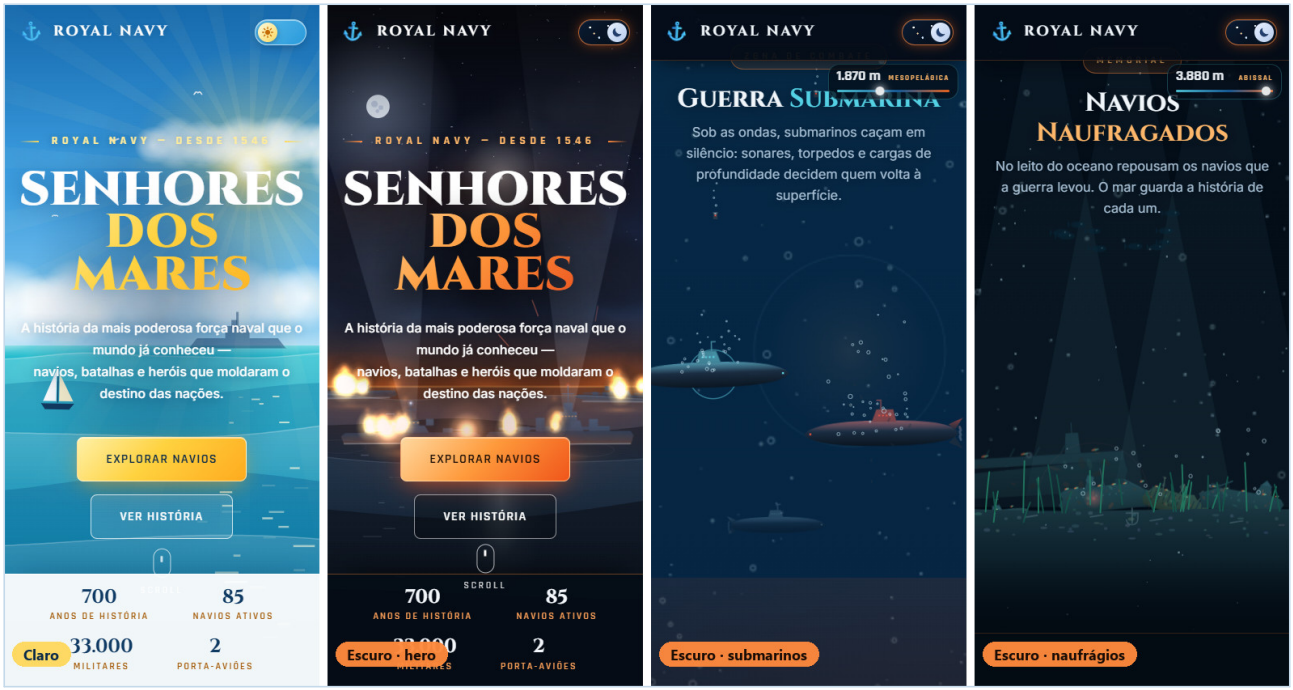


Figura 16 — No celular (390×844, emulação): modo claro e três telas do escuro; o botão fica na extremidade direita da barra, sem o rótulo.

6. Diagnóstico de desempenho

A causa da lentidão não era um único defeito, e sim a **soma de trabalho contínuo e invisível**: enquanto o usuário olhava o hero, o navegador também renderizava 12 cenas 3D fora da tela, repintava um grid animado, borrava fundos sobre canvases em movimento e recalculava layout a cada evento de rolagem. Cada item abaixo foi identificado por leitura do código; os mais graves foram confirmados por medição no navegador.

6.1 Problemas de maior impacto

#	Problema encontrado	Consequência	Solução aplicada
P1	**13 contextos WebGL** criados no carregamento, cada um com seu *loop* a 60 fps, mesmo fora da tela	GPU/CPU ocupadas o tempo todo; pico de travamento no *load* (13 renderers e compilações de shader); risco de perder contextos (limite ≈ 16)	Criação sob demanda + um loop único que só desenha o que está visível
P2	Verificação de resize comparava `canvas.width` (que já inclui o DPR) com `clientWidth`	Em telas com escala ≠ 100% (ex.: 125%), `setSize` rodava **em todo frame**, realocando o buffer de desenho	ResizeObserver: ajusta só quando o tamanho realmente muda
P3	Modal: o loop continuava após fechar; a cada abertura novos listeners em `window` e um novo renderer eram criados	Trabalho invisível permanente que crescia a cada uso; vazamento de memória	Renderer criado uma vez; loop para ao fechar; listeners por Pointer Events no próprio canvas
P4	12 mapas de sombra (1024^2) com PCF suave, antialias e *tone mapping* em todas as cenas	Cada cena pequena pagava por sombras que mal se vêem	Sombras só no modal e na imagem grande; tone mapping removido
P5	Materiais e geometrias novos para cada peça; 26 esferas de luz + 16 portinholas + canhões como malhas separadas	Centenas de draw calls e de objetos por cena	Cache compartilhado + InstancedMesh
P6	`backdrop-filter` (blur 4–20 px) na navbar, na barra de estatísticas, nos selos 3D e no overlay do modal	O blur é reprocessado a cada mudança do que está atrás — e atrás havia canvases animando	Removido; fundos quase opacos
P7	`.hero::before` animava `background-position` em tela cheia, para sempre	Repintura da viewport inteira a cada frame (não é acelerável pelo compositor)	Substituído por raios de sol estáticos (uma única pintura)
P8	Brilho do cursor: loop infinito, alterando `left/top` e com `mix-blend-mode: screen` sobre tudo	Layout a cada frame + mistura de cores da página inteira	`translate3d`, sem blend-mode, loop para ao convergir, só com mouse

6.2 Problemas de impacto médio

#	Problema	Solução
P9	Hero 2D: 6 `createLinearGradient` por frame, 120 estrelas, partículas, sem pausa fora da tela; tempo por frame (`t += 0.016`)	Gradientes/sprites pré-renderizados; pausa por visibilidade; tempo em segundos (*dt*)
P10	Handler de scroll lia `offsetTop` de todas as seções (layout forçado) e escrevia estilos a cada evento	Um handler com rAF que só lê `scrollY`; link ativo por IntersectionObserver
P11	`transition: all` em ~20 componentes	Listas explícitas de propriedades (transform, box-shadow, cores...)
P12	Barras da frota animavam `width` (layout)	`transform: scaleX`
P13	Contadores com `setInterval(16 ms)`	rAF com easing por tempo
P14	Animação infinita de escala em uma camada de seção inteira (batalhas), quase invisível	Removida
P15	Scripts sem `defer` e sem verificação de integridade; 12 pesos de fonte; sem preconnect para o cdnjs	`defer` + SRI (sha512), 9 pesos, preconnect
P16	Canvases WebGL com `alpha:true`; oceano transparente; 4 luzes por cena	Canvas opaco; oceano opaco; 2 luzes
P17	Sem `prefers-reduced-motion`	Suporte completo (CSS e JS)

6.3 Exemplos de código (antes → depois)

P2 — resize a cada frame:

```
// ANTES (todo frame): em DPR 1,25 a condição é SEMPRE verdadeira
if (renderer.domElement.width !== cw || renderer.domElement.height !== ch) {
  renderer.setSize(cw, ch, false);          // realoca o buffer de desenho
}
// DEPOIS: ResizeObserver marca "resizePending" só quando o tamanho muda
const ro = new ResizeObserver(list => list.forEach(r => byCanvas.get(r.target).resizePending = true));
```

P7 — grid animado do hero:

```
/* ANTES: repinta a tela inteira a cada frame, para sempre */
.hero::before { background-size: 60px 60px; animation: gridDrift 20s linear infinite; }
@keyframes gridDrift { to { background-position: 60px 60px; } }

/* DEPOIS: raios de sol estáticos, uma única pintura */
.hero::before { background: repeating-conic-gradient(from 195deg at 78% 24%, ...); }
```

7. Erros e bugs corrigidos

Esta seção separa **bugs funcionais** (comportamento errado) dos problemas de desempenho da seção anterior. Os itens marcados com **[R]** foram **reproduzidos no navegador** antes de serem corrigidos; os demais foram identificados por leitura do código e verificados depois da correção.

7.1 Interface e animações

#	Bug	Causa raiz	Correção
B1 [R]	O "levantar" dos cards no hover **nunca acontecia** (ships, batalhas, linha do tempo, frota)	`.fade-in-up.visible` e `.ship-card:hover` têm especificidade (0,2,0) e a primeira vem depois no arquivo, então `translateY(0)` vencia	Classes de revelação são **removidas** no fim da transição; o elemento volta ao CSS normal. Hover verificado: `matrix(1,0,0,1,0,-8)`
B2 [R]	Hover lento (0,7 s) e sem transição de sombra/borda	`.fade-in-up` definia `transition: opacity 0.7s, transform 0.7s` e sobrescrevia a do componente	Mesma correção de B1
B3 [R]	Barras da seção Frota **ficavam em 0 px para sempre**	O JS zerava o `style` inline *antes* de ler a largura original; a regex depois só encontrava "0px"	Largura passa a vir de `--fill` no HTML; o JS só adiciona uma classe; a barra anima por `scaleX`
B4	Parallax do título não funcionava	A animação CSS com `forwards` tem precedência sobre estilos inline e travava `transform/opacity`	Animação removida do contêiner (os filhos animam); parallax via rAF
B5	Erro no console ao clicar nos links do rodapé ("'#' is not a valid selector")	Links `href="#"` passavam pelo handler de scroll suave, que fazia `querySelector('#')`	Links usam `data-open-ship`; scroll suave passou para o CSS
B6	Atributos `data-aos` sem biblioteca AOS	Código morto (7 elementos)	Removidos
B7	Cards filtrados ainda recebiam foco por teclado	Só se usava `pointer-events: none`	Atributo `inert` + `aria-pressed` nos botões
B8	Em telas baixas o conteúdo do hero passava por trás da barra de estatísticas (botões escondidos)	Barra absoluta sem reserva de espaço	Padding reservado no hero; título limitado por `17vh`
B9	Card de destaque (Trafalgar) com título e selo empurrados para o canto	Grid com coluna *auto* e itens sem posição explícita: o texto longo alargava a coluna	Posições explícitas de coluna/linha; versão de coluna única no celular

7.2 Modal

#	Bug	Causa raiz	Correção
B10	Conteúdo cortado: **Armamento e História** não apareciam e não rolavam	Linha do grid *auto* crescia com o conteúdo além do `max-height: 90vh`; `overflow:hidden` cortava	`grid-template-rows: minmax(0,1fr)`: o painel direito rola (1192 px em 691 px, medido)
B11	Navio deslocado para baixo e fora do quadro	O canvas seguia a altura inflada da linha do grid; câmera fixa para um quadro largo	Correção do grid + câmera com enquadramento automático
B12	Sem gerenciamento de foco, ESC ou armadilha de Tab; rolagem do fundo saltava ao abrir	Ausentes	Foco no botão fechar, Tab preso, Esc fecha, foco devolvido, `scrollbar-gutter: stable`
B13	Renderização continuava após fechar; memória crescia a cada abertura	Loop nunca cancelado; renderer e listeners recriados	Loop pausado ao fechar; renderer único
B14	Botão fechar com `onclick` inline e handlers globais	Estilo do código original	Delegação de eventos (`data-open-ship`, `.modal-close`)

7.3 Servidor (app.py)

#	Bug	Causa raiz	Correção
B15 [R]	**O servidor não iniciava** quando a saída não era um console (UnicodeEncodeError)	Banner com caracteres Unicode (traço duplo e emojis de âncora e globo) em cp1252 (Windows)	Banner só ASCII. Verificado: sobe sem `-X utf8`
B16	**Risco de segurança:** `debug=True` com `host="0.0.0.0"`	O debugger do Werkzeug executa código Python; ficava acessível para toda a rede local	Host padrão 127.0.0.1; debug só é permitido em host local (desligado automaticamente caso contrário)
B17	Rota `/static/<path>` duplicada	O Flask já registra a rota estática automaticamente	Rota removida
B18 [R]	Arquivos inexistentes (ex.: `/favicon.ico`) recebiam a página inteira (≈ 25 KB) com status 404	O handler 404 sempre renderizava o index	404 real de 9 bytes para `/static/*` e URLs com extensão; favicon embutido evita a requisição
B19	Sem cache para arquivos estáticos	Configuração padrão	1 h fora do debug; sem cache em debug

7.4 Erros de julgamento meus, corrigidos durante o trabalho

Registro honesto de ajustes feitos depois da primeira versão, encontrados nas capturas de tela:

- Mar 3D “estourado” em ciano neon: a soma das luzes passava de 1,0. Reduzidas as intensidades e escurecida a água.
- Velas do galeão esverdeadas: o hemisfério refletia o mar ciano nas superfícies verticais. Cor de solo neutralizada.
- Ícone das espadas cruzadas invisível nas Batalhas: glifo de texto herdava cor escura sobre fundo azul. Definida cor clara.
- Barra de estatísticas abaixo da dobra em 1366×768: padding do hero reduzido.
- Comentário/commit dizia “12 → 8 fontes”; o correto é 12 → 9 pesos. Corrigido antes do envio.

7.5 Ajustes e correções durante o desenvolvimento do modo escuro (v2.0)

Achados nas capturas de tela e nas medições, corrigidos antes da publicação:

#	Achado	Causa	Correção
D1 [R]	A batalha do hero ficava pouco visível	Cascos escuros demais, quase da cor do fundo, e a vinheta CSS do `.hero-overlay` escurecia o centro da tela	Cascos mais claros (#1b2d45 / #243a56) com luar por cima e brilho de incêndio por baixo; faixa quente refletida no mar; navios, explosões e fogo maiores; mais tiros por segundo; vinheta central reduzida
D2 [R]	O medidor de profundidade cobria o conteúdo	Primeira versão: coluna vertical sobre as seções	HUD horizontal fixo logo abaixo da navbar (`top: 82px`; 74 px no celular), sem capturar cliques
D3	Emenda visível entre Guerra Submarina e Batalhas	O fundo da seção Batalhas começava com um brilho opaco	Degradê que começa transparente (0 → 42% aos 220 px) e brilhos radiais mais discretos (0,22 → 0,12); depois, uma máscara de 320 px no topo de `.battles-bg` eliminou a borda que ainda restava
D8	Zonas do medidor com limites diferentes dos oceanográficos (2.180 m aparecia como "Mesopelágica")	Limites simplificados (150 / 900 / 2.600 / 3.700 m)	Limites usuais: epipelágica 0–200 m, mesopelágica 200–1.000 m, batipelágica 1.000–4.000 m, abissal 4.000 m+ (`ZONES` em deep-sea.js)
D4	No celular os navios da batalha ficavam pequenos	Tamanhos proporcionais à largura (390 px)	Fator ×1,34 quando a largura é menor que 600 px
D5	O modo claro pagava por trabalho do modo escuro	O fundo do mar media o layout a cada redimensionamento mesmo com o tema claro	`remeasure()` sai sem medir enquanto o tema é claro; `deep-canvas`, medidor e faixas com `display:none` no claro
D6	Medição de FPS travava	O painel do navegador do aplicativo, quando oculto, não dispara `requestAnimationFrame`	Medições passaram a usar o Edge headless via DevTools Protocol (primeiro por software; depois com a GPU — ver seção 9)
D7	Comparação de pixels com um artefato (barra de navegação duplicada)	Comparava capturas de sessões diferentes; o Edge às vezes captura elementos fixos duplicados	Comparação refeita com a v1.1 publicada (via git) e a v2.0 servidas nas mesmas condições: 0 pixel de diferença no corpo

Dica de operação descoberta no processo: ao encerrar o Flask em modo debug, o recarregador deixa um segundo processo Python segurando a porta 5000; é preciso encerrar os dois.

8. Paletas: dia ensolarado e noite de guerra

v4.1: o modo claro ganhou uma paleta própria de "mar calmo" (fundo `#beeef7`, água turquesa, areia `#dcc28c`) em `light.css` e nas cores das cenas; as cores desta seção são as da v1.1/v2.0 e seguem valendo para os elementos gerais do site (botões, textos, cartões).

A identidade anterior era noturna (navy profundo, aço e ouro escuro, estrelas). A nova é **diurna**: o que se vê num dia de sol em alto-mar — céu azul, água turquesa, luz dourada, espuma branca e o vermelho-coral das boias. A mudança é global e vive em **variáveis CSS** (`:root`), o que permite trocar o tema editando um único bloco.

Céu

sky-50	sky-100	sky-200	sky-300	sky-400	sky-500	sky-600
#F3FBFF	#E3F4FD	#C4E6F9	#92D2F3	#57B8EA	#2A9BDC	#1A7DBB

Mar turquesa

sea-300	sea-400	sea-500	sea-600	sea-700
#6FE3D8	#2EC9C2	#0FA9B1	#0A8795	#076A78

Sol

sun-100	sun-200	sun-300	sun-400	sun-500	sun-600	sun-700	sun-800
#FFF7D9	#FFEC9F	#FFDB5C	#FFC62E	#F9A71A	#E58A0B	#B96500	#A35700

Azul-marinho (texto e rodapé)

navy-900	navy-800	navy-700	navy-600
#0A2A4A	#0F3A63	#164A7C	#1F5D96

Coral e areia

coral-400	coral-500	sand-50	sand-100
#FF8060	#F2593A	#FFFDF6	#FDF3DC

Onde cada cor é usada

Papel	Cores	Aplicação
Texto principal / títulos	navy-900, navy-800	Títulos de seção, nomes de navios, números
Texto secundário	#3b5673 (7,7:1 sobre branco)	Parágrafos e legendas
Destaques de seção	sea-600 / sea-700	Palavra em destaque nos títulos, classes dos navios, botão "Ver detalhes"
Ação e ênfase	sun-200 … sun-500	Botão principal, selos, anos da linha do tempo, resultado das batalhas, sublinhado do menu
Rótulos pequenos sobre claro	sun-800, sea-700	Escolhidos para atingir ≥ 4,5:1 em textos pequenos
Interação/alerta	coral-400/500	Ponto da linha do tempo no hover; botão fechar no hover
Superfícies	branco, sky-50/100, sand-50	Cards e fundos de seção, alternando frio e quente
Seção de contraste	gradiente #0e4d86 → #0a6aa0 → #0a8a9c	Batalhas: mar profundo iluminado por um brilho dourado no canto
Rodapé	navy-900	Âncora visual final; separado por uma onda

Elementos de identidade novos

- **Hero diurno:** sol com halo e raios, nuvens à deriva, gaivotas, veleiro, silhueta de um navio de guerra ao horizonte, ondas com crista brilhante e cintilação do sol na água.
- **Cenas 3D ensolaradas:** céu em gradiente, mar turquesa, luz quente do sol, hulls em cinza-naval claro, velas cor de creme.
- **Ondas divisórias:** máscara SVG reutilizável (colorida por `background`) entre Navios → Batalhas → Frota e antes do rodapé.
- **Vinheta de contraste no hero:** garante leitura do texto branco sobre o céu e o mar.

Acessibilidade de cor

Texto secundário e rótulos pequenos usam tons escuros (sun-800, sea-700, #3b5673) para atingir contraste mínimo de 4,5:1 sobre fundo claro. Sobre o hero (imagem viva), a leitura é apoiada por vinheta escura suave e *text-shadow*. O contraste do texto grande do hero é adequado, mas o parágrafo sobre o mar tem contraste mais modesto; ficou registrado como ponto de atenção na seção 10.



Figura 17 — Batalhas: seção de contraste em azul profundo, card de destaque corrigido e ícone claro.

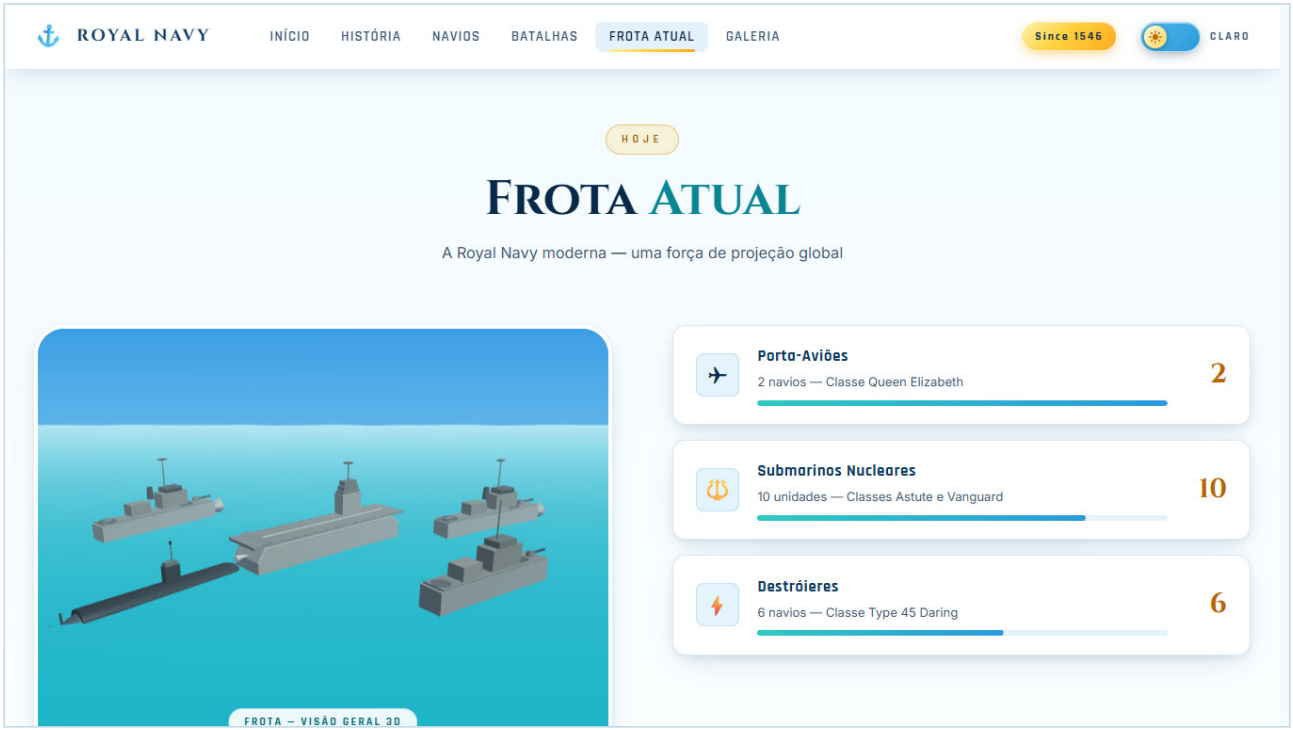


Figura 18 — Frota Atual: cena 3D em formação e barras animadas até seus valores (2, 10 e 6 unidades).

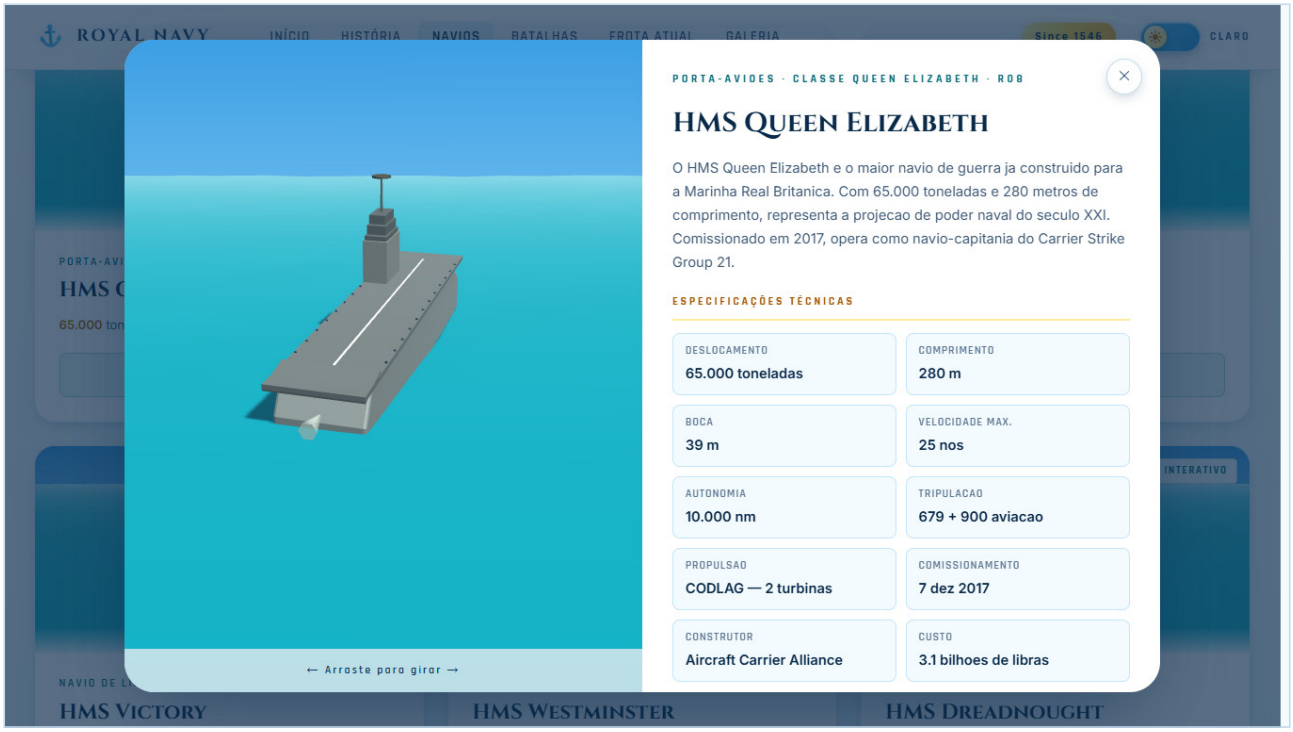


Figura 19 — Modal do HMS Queen Elizabeth: navio enquadrado, ficha técnica e botão fechar.

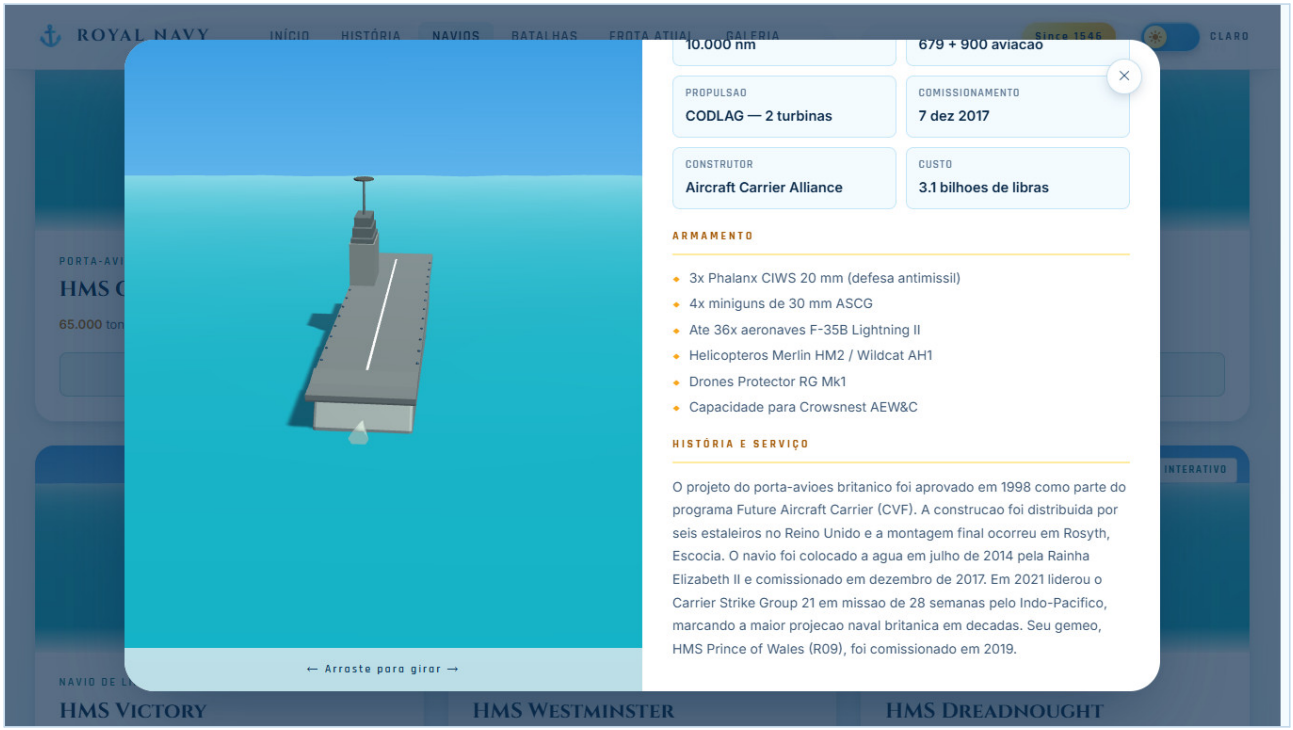


Figura 20 — O painel do modal rolado: Armamento e História agora acessíveis.

Paleta do modo escuro: noite de guerra

O modo escuro troca o dia por uma noite de tempestade: **noite e aço** no fundo, **brasa** (laranja) para o que é ação e ênfase — fogo, títulos, botões, anos — e **ciano** para o que é água e informação. As cores vivem em variáveis CSS no bloco `:root[data-theme="dark"]` de dark.css e, para as cenas, em constantes dos arquivos JavaScript.

Noite e aço

night	body	steel	sky-mid	hull	super
#050B14	#04101C	#0A1A2E	#0B1C33	#1B2D45	#243A56

Brasa

ember-l	ember	fire	fire-d	glow
#FFB86B	#FF8A3D	#FF6A1F	#E8431B	#FFE2A8

Ciano e água (do raso ao fundo)

cyan	cyan-l	d 0	d 0,12	d 0,30	d 0,55	d 1
#54D1E4	#7FDCEC	#0E6276	#0A4C68	#083454	#051F3A	#030C14

Textos

tx	tx2	tx3
#EEF5FB	#A9C1D6	#8FA9C0

Contraste (WCAG) no modo escuro

Texto	Sobre o fundo do corpo (#04101c)	Sobre o painel translúcido no pior caso (água rasa)
--d-tx #eef5fb (texto principal)	17,4 : 1	14,3 : 1
--d-tx2 #a9c1d6 (secundário)	10,3 : 1	8,5 : 1
--d-tx3 #8fa9c0 (rótulos)	7,9 : 1	6,5 : 1
ember-l #ffb86b (anos, destaques)	11,2 : 1	9,3 : 1
ember #ff8a3d	8,2 : 1	6,7 : 1
cyan-l #7fdcec (classes dos navios)	12,2 : 1	10,0 : 1
Botão principal: #1a0d05 sobre #ffd9a0 → #ff9a3c → #f0561b	14,2 : 1 → 9,0 : 1 → 5,5 : 1	—

Todos os pares ficam acima de 4,5:1 (nível AA). O pior caso do painel considera o painel `rgba(6,22,40,0.8)` sobre a água mais clara (#0e6276).

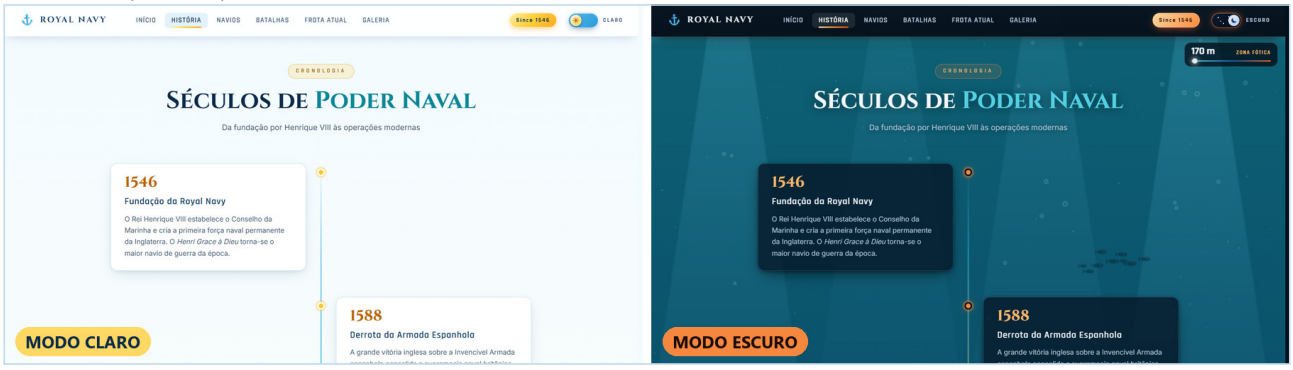


Figura 21 — Seção História nos dois modos: mesmo conteúdo e layout; no escuro, a água do canvas fixo aparece atrás dos cards translúcidos.

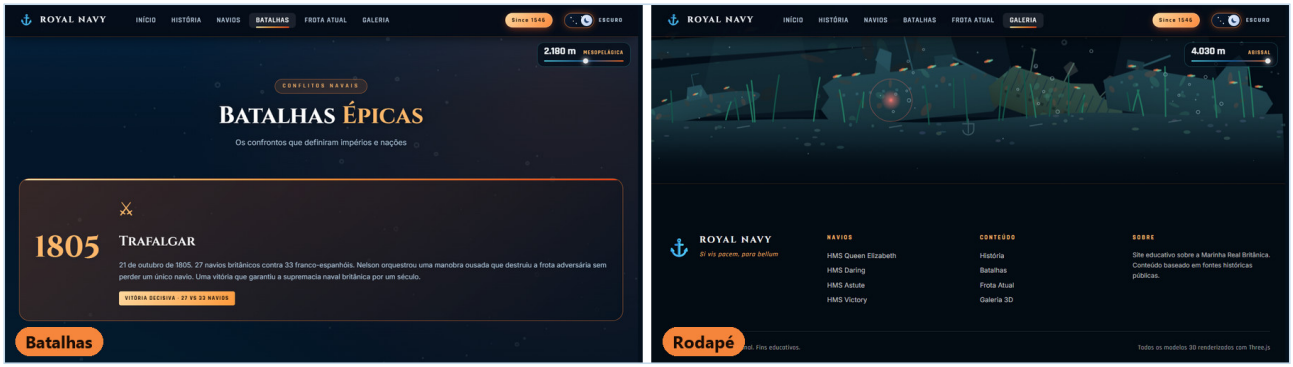


Figura 22 — Modo escuro: Batalhas (sem as ondas claras) e rodapé (leito do mar).

9. Como foi verificado

9.1 Métodos

Método	Ferramenta	O que confirmou
Leitura completa do código	Revisão manual (6 arquivos)	Lista de problemas P1–P17 e B1–B19
Medição de FPS	Navegador do aplicativo (rAF durante 3 s)	1,7 FPS (original) → 56,7 (topo) / 60 (navios visíveis)
Inspeção do CSS calculado	`document.styleSheets` + `getComputedStyle`	Conflito .fade-in-up × :hover (B1/B2); barras em 0 px (B3)
Contagem de draw calls	Interceptação de `drawElements/drawArrays` em WebGL/WebGL2	≈ 77/frame com navios; ≈ 17/frame só o modal; 0 no topo
Capturas por seção	Edge headless via Chrome DevTools Protocol (script próprio)	Revisão visual de todas as seções em 1366×768 e 390×844
Hover real	Evento de mouse sintético do DevTools	Transform do card = translateY(−8 px)
Rolagem do modal	Medição de scrollHeight × clientHeight (desktop)	1192 px de conteúdo em 691 px visíveis; overflow-y: auto
Console	Captura de erros e avisos em cada execução	Nenhum erro ou aviso
Rotas do servidor	curl	/ = 200; /static/* = 200; arquivo inexistente = 404 (9 bytes); URL desconhecida = 404 + página
Sintaxe	`node --check`, `py_compile`	Todos os arquivos válidos
Versão estática num subcaminho	http.server + Edge headless	Site idêntico ao do Flask, sem erros
Política CSP com teste de controle	Domínio *Log* do DevTools	Política real: 0 violações. Política propositalmente errada: bloqueio detectado (o teste enxerga violações)
Site publicado	curl no endereço do Pages	index.html e os arquivos de static respondendo HTTP 200 (20/09/2026)
Disponibilidade do domínio	RDAP do Registro.br + nslookup	royalnavy.com.br não registrado (404 no RDAP; controle google.com.br = 200; DNS inexistente)
Trava do domínio	Execução de enable_custom_domain.py	Recusou domínio inexistente e domínio de outro servidor; nada foi gravado
v2.0 · Comparação de pixels	Servir a v1.1 (git) e a v2.0 e comparar capturas com PIL	Modo claro: 0 pixel de diferença no corpo (seção 9.6)
v2.0 · Desempenho por estado e por rolagem	Edge headless com GPU; contagem de quadros do rAF; medianas de rodadas	Seção 9.3
v2.0 · Canvases ativos	Instrumentação das chamadas de desenho por canvas	Só o que está visível desenha (seção 9.4)
v2.0 · Sistema de temas	Cliques, recarga, API desativada, sondagem da animação	Seção 9.5
v2.0 · Quadros da revelação	Animação congelada em 4 pontos (Web Animations API)	Figura 10

Método	Ferramenta	O que confirmou
v2.0 · Site publicado	curl + Edge headless no endereço do Pages	Seção 9.7
v3.0 · Testes automatizados	pytest contra o banco real, com o papel de leitura (88 testes)	Todos aprovados (seção 13.7)
v3.0 · Segurança do código e das dependências	Bandit e pip-audit	0 achados; 0 vulnerabilidades (após atualizar o Flask)
v3.0 · Interface da consulta	Edge + DevTools em claro, escuro e celular; resposta hostil injetada	Sem erros; nada injetado (seção 13.7)

9.2 Limitações honestas

- As medições de FPS foram feitas em **WebGL por software**; não foram feitas na sua máquina com GPU real.
- Não houve teste em navegadores diferentes do Chromium/Edge (Firefox e Safari não foram executados).
- Não há testes automatizados no projeto; a verificação foi por medição, captura e inspeção.
- O celular foi verificado por emulação (390×844), não em dispositivo físico; toque e gestos não foram exercitados em hardware real.
- O código original não estava em um repositório git, então não há histórico anterior; as versões antigas podem ser recuperadas pelo histórico do OneDrive.
- **v2.0:** as medições de FPS foram feitas em um único computador (gráficos integrados Intel UHD); as de rolagem têm apenas 2 rodadas, e a variação entre rodadas é de até ±10%.
- **v3.0:** a consulta foi testada no computador do projeto e em Chromium/Edge; não houve teste de invasão feito por terceiros nem teste com um banco remoto (seção 13.7).
- **v2.0:** o modo escuro foi testado em Chromium/Edge (desktop e emulação de celular 390×844); Firefox e Safari não foram executados, nem aparelhos móveis reais.

9.3 Desempenho medido na v2.0

Máquina: Intel Core i3-1215U, gráficos integrados Intel UHD, 7,7 GB de RAM; Edge com GPU (D3D11), janela 1366×768, DPR 1; monitor a 120 Hz (uma página sem canvas desenhando chega a ≈ 120 fps). Cada valor é a **mediana de 3 rodadas** contando quadros do `requestAnimationFrame` por 3 s. A v1.1 foi servida a partir do commit `d006295`, nas mesmas condições.

Estado (fps · ms por quadro)	v1.1 claro	v2.0 claro	v2.0 escuro
Topo: hero animado	62 · 16,2	61 · 16,3	61 · 16,3
Grade de navios (6 cenas WebGL)	93 · 10,8	82 · 12,2	61 · 16,5
Galeria (5 cenas WebGL)	85 · 11,8	73 · 13,7	58 · 17,2
Seção sem canvas (Batalhas) / só mar profundo (História)	120 · 8,3	120 · 8,3	120 · 8,3
Faixa Guerra Submarina	—	—	120 · 8,3
Faixa Navios Naufragados	—	—	90 · 11,1

Rolagem contínua do topo ao fim da página em 18 s (mediana de 2 rodadas):

Métrica	v1.1 claro	v2.0 claro	v2.0 escuro
Altura da página	8.941 px	8.941 px	10.294 px (mais duas faixas)
FPS médio	81,1	78,1	68,6
Tempo de quadro: mediana · p95 · p99 (ms)	8,4 · 24,9 · 83	8,4 · 25,2 · 79	8,4 · 29,1 · 104
Quadros acima de 33 ms · acima de 50 ms	36 · 20	36 · 20	40 · 22
Nível do RNPerf ao final	—	— (desligado)	0 nas duas rodadas (o monitor reduziu os efeitos)

Leitura: no modo claro a v2.0 fica na mesma faixa da v1.1 — a variação entre rodadas nas cenas 3D chega a ±10% nesta máquina. No modo escuro o hero em batalha custa o mesmo que o hero claro; trechos com só o mar profundo e a guerra submarina chegam ao teto do monitor; nas seções com 3D o modo escuro fica ≈ 60 fps (o fundo do mar é desenhado junto com as cenas WebGL). Na rolagem contínua o escuro tem ≈ 12% menos FPS médio e o RNPerf reduziu a qualidade ao mínimo; a mediana de quadro é a mesma nos três (8,4 ms).
Uma medição anterior, com o Edge renderizando por *software* (SwiftShader), deu ≈ 6 fps já no hero **claro original**; por isso os valores absolutos daquela configuração não foram usados.

9.4 Quais canvases desenham em cada trecho

Contagem de chamadas de desenho por canvas em cada posição da página (canvases com mais de 1,5 chamada/s):

Modo · posição	Canvases 2D desenhando	WebGL
Claro · topo	ocean-canvas (hero)	—
Claro · grade de navios / galeria	nenhum	sim (cenas visíveis)
Claro · Batalhas	nenhum (após voltar do escuro: nenhum também)	—
Escuro · topo	ocean-canvas (batalha)	—
Escuro · rolado 400 px	ocean-canvas + deep-canvas	—
Escuro · História	deep-canvas	—
Escuro · grade de navios	deep-canvas	sim
Escuro · Guerra Submarina	deep-canvas + sub-canvas	—
Escuro · naufrágios / rodapé	deep-canvas + wreck-canvas	—

9.5 Sistema de temas: testes

Teste	Resultado
Persistência	Tema escuro salvo em `localStorage`; após recarregar: `data-theme=dark`, `theme-color` #050b14, `aria-checked=true`, rótulo "Escuro", fundo do mar visível
6 cliques rápidos (120 ms entre eles)	Só o primeiro foi aceito; estado final consistente (tema, `aria-checked` e armazenamento iguais) e a classe `theme-switching` removida ao fim
Cortina de reserva (API desativada)	Cortina presente durante a troca, removida depois; tema aplicado; sem erros
Parâmetros da revelação	Animação criada em `::view-transition-new(root)` com 900 ms e `cubic-bezier(.4,0,.2,1)`; termina no estado `finished` e limpa `theme-switching`
Console e CSP	Nenhum erro ou violação em todos os testes

9.6 Modo claro sem regressões

A v1.1 publicada (commit `d006295`) e a v2.0 foram servidas nas mesmas condições e as seções História, Linha do tempo e Batalhas foram comparadas pixel a pixel:

Seção	Pixels diferentes no corpo (abaixo dos 70 px da navbar)	Pixels diferentes na navbar
História	**0**	14.375
Linha do tempo	**0**	14.590
Batalhas	**0**	14.602



Figura 23 — Batalhas (modo claro): v1.1, v2.0 e a diferença (em vermelho). A única diferença está na barra de navegação, que ganhou o botão de tema.

Além disso, o `style.css` teve 116 linhas acrescentadas e 0 alteradas (`git diff --stat` entre os commits).

9.7 Verificação no site publicado

- Commit `ae6b9ea` e tag `v2.0.0` enviados; remoto igual ao local.
- O GitHub Pages atualizou em ≈ 15 s; `theme-init.js`, `theme.js`, `war-hero.js`, `deep-sea.js` e `dark.css` responderam HTTP 200 e o HTML publicado contém o botão.
- No endereço público, o modo escuro foi ligado e percorrido no Edge headless: sem erros de console nem violações de CSP.

10. Pendências e recomendações

Prioridade	Item	Detalhe
Alta	Acentuação em `ships-data.js`	O arquivo de dados está sem acentos (“Porta-Avioes”, “Britanica”, “projecao”), diferente do restante do site. Não foi alterado por estar fora do escopo pedido; é uma correção simples e de alto valor.
Alta	Menu para celular	Abaixo de 768 px os links do menu somem e não há alternativa (hambúrguer). A navegação por âncoras fica indisponível.
Alta	Registrar royalnavy.com.br	Concluir a compra no Registro.br (só o titular pode) e criar os registros DNS; depois rodar `enable_custom_domain.py`. Passo a passo na seção 12. O domínio estava disponível em 20/09/2026.
Concluída	Publicar como site	Feito: versão estática em `docs/` servida pelo GitHub Pages em HTTPS (seção 12).
Concluída	Publicar a consulta ao banco na internet	Feito: Render (aplicação) + Neon (PostgreSQL gerenciado), TLS verificado de ponta a ponta, verificado no ar (seção 14).
Concluída	Redesenhar o modo claro (v4.1)	Feito: “Mar em Paz”, com comboio em perspectiva, mergulho, patrulha e recifes (seção 15); barra de navegação corrigida (15.7).
Média	Travadas nos primeiros segundos após carregar	Uma rodada sem aquecimento mostrou quadros de 100–200 ms logo após o carregamento, nos dois temas (seção 15.6, L1). Investigar a construção dos sprites e a compilação inicial.
Média	Testar em aparelho real e em computador modesto	A medição de desempenho é de um PC com GPU dedicada e monitor de 120 Hz; o celular foi só emulado (seção 15.9).
Média	Contraste do parágrafo do hero	O texto branco médio sobre o mar tem contraste modesto; considerar aumentar a vinheta ou o peso da fonte.
Média	Hospedar three.js localmente	Hoje vem do cdnjs (com SRI). Hospedar em `static/` elimina uma dependência externa e funciona offline (≈ 600 KB).
Média	Compressão gzip/brotli	O servidor de desenvolvimento não comprime; em produção use um servidor/host que comprima (o GitHub Pages já faz).
Baixa	Limite de contextos WebGL	O site usa até 13 contextos após rolar tudo (limite ≈ 16). Uma evolução seria um único renderer compartilhado, ou liberar cenas muito distantes.
Baixa	Testes automatizados	Um pequeno teste de fumaça (Playwright) para carregar a página, checar o console e abrir o modal evitaria regressões.
Baixa	Quebra de linha do texto do hero	O ` ` fixo gera linhas curtas no celular.
Alta	Trocar a senha do usuário `postgres`	A senha atual é curta e apareceu em conversa. Só o administrador pode trocá-la (comando na seção 13.6.1); depois, atualizar o pgAdmin. O site não é afetado (usa outro papel).
Alta	Reiniciar o serviço PostgreSQL	`listen_addresses = 'localhost'` já está gravado, mas só vale após reiniciar (exige administrador do Windows; seção 13.6.1).

11. Operação: executar, versionar e publicar

11.1 Executar localmente

```
python -m venv .venv
.venv\Scripts\activate          # Windows  (Linux/macOS: source .venv/bin/activate)
pip install -r requirements.txt
python db/aplicar.py            # 1 vez: prepara o banco da consulta (v3.0)
python app.py                   # abre em http://localhost:5000  (consulta em /consulta)
```

11.2 Variáveis de ambiente

Variável	Padrão	Função
PORT	5000	Porta do servidor
HOST	127.0.0.1	Interface de rede. `0.0.0.0` expõe na rede local; nesse caso o modo debug é desligado automaticamente
FLASK_DEBUG	1	`0` desliga o recarregamento automático e ativa cache de 1 h nos estáticos

11.3 Repositório

Item	Valor
URL	https://github.com/AlanSouzaDev7/Naval_Design-WEB-V.1
Branch	main (rastreia origin/main)
Site publicado	https://alansouzadev7.github.io/Naval_Design-WEB-V.1/ (Pages: branch main, pasta /docs)
Commits	a450870 — otimização e paleta · f26ebc1 — versão estática e segurança · mais o commit desta documentação e do preparo do domínio (ver `git log`)
Arquivos versionados	código, README, .gitignore, requirements, scripts de build/domínio, docs/ e este PDF
Fora do repositório	.venv, .claude/, arquivo `teste`, caches Python

11.4 Guia rápido de manutenção

Quero...	Edite...
Trocar as cores do site	Bloco `:root` no início de `style.css` (e as cores de `C` em `ships-3d.js` para as cenas 3D; e os gradientes do hero em `initHeroSea`)
Adicionar um navio	1) Entrada em `ships-data.js`; 2) card em `index.html` com `data-ship` e `data-type`; 3) se for um tipo novo, uma função `buildX()` e um `case` em `buildShip`
Ajustar o brilho/luz 3D	`createScene()` em `ships-3d.js` (HemisphereLight e DirectionalLight)
Mudar textos e seções	`templates/index.html`
Mudar cache/porta/segurança	`app.py` (Flask local) e a meta CSP em `index.html` (vale também no Pages)
Atualizar o site publicado	Edite templates/static, rode `python build_static.py`, faça commit e push (o Pages atualiza sozinho em ~1 min)
Ativar o domínio próprio	`python enable_custom_domain.py royalnavy.com.br` — só depois do registro e do DNS
Adicionar animação de entrada	Inclua o seletor na lista *groups* de `initReveal()` (main.js)

11.5 Histórico de versões

Versão	Data	Commit(s)	O que mudou
1.0 (original)	—	sem git	Site recebido: Flask + HTML/CSS/JS + Three.js, identidade noturna, 1,7 FPS
1.1	19–20/09/2026	`a450870`, `f26ebc1`, `d006295`	Auditoria, desempenho, correção de bugs, paleta ensolarada, versão estática (docs/), CSP e SRI, preparo do domínio, PDF v1.1
2.0.0	26/09/2026	`ae6b9ea` · tag `v2.0.0`	Modo escuro "Guerra em alto mar": botão e transição, hero em batalha, mergulho, guerra submarina, naufrágios, 3D noturno, RNPerf; 19 arquivos (+4.918 / −177 linhas, incluindo docs/)
2.0.1	26/09/2026	`v2.0.1`	Zonas do medidor de profundidade com limites oceanográficos usuais; emenda visual entre Guerra Submarina e Batalhas eliminada (ver D3 e D8, seção 7.5)
3.0.0	27/09/2026	`450f8ab` + PDF · tag `v3.0.0`	Consulta de navios: página, API somente-leitura, banco com view e papel de privilégios mínimos, segredos fora do Git e do OneDrive, Flask 3.1.3, 88 testes (seção 13)
3.0.1	27/09/2026	correção	Segredos em `~/.royalnavy` (o AppData era redirecionado); dependências instaladas no Python certo; mensagens de diagnóstico em modo local; 3 testes de regressão (88 no total)
4.0.0	28/09/2026	tag `v4.0.0`	Consulta publicada na internet: Render + Neon (PostgreSQL gerenciado), raiz TLS própria para `verify-full`, dados recarregados via INSERT, verificada no ar (seção 14)
4.1.0	29–30/09/2026	`6cf732a`, `ad63f65` · tag `v4.1.0`	Tema claro "Mar em Paz": comboio em perspectiva, mergulho 0–40 m, patrulha, recifes de coral; RNPerf nos dois temas; barra de navegação corrigida até 1280 px; teste de fluidez (seção 15)

11.6 Guia de manutenção do modo escuro

Quero...	Edite...
Trocar as cores do modo escuro	Tokens em `:root[data-theme="dark"]` (dark.css); `THEMES.dark` (ships-3d.js); cascos e mar em war-hero.js; `STOPS` (cor da água) em deep-sea.js
Ajustar a intensidade da batalha	`buildShips()` em war-hero.js (`rate` = intervalo entre salvas, `dmgMax` = acertos para afundar); `CAP` e `POOL` para partículas
Mudar profundidade e zonas do medidor	`ZONES` e a escala 4200 em `Gauge` (deep-sea.js)
Criar outra faixa submarina	Um `<div class="abyss-band">` com canvas em index.html e uma chamada `band('id', onResize, onFrame)` em deep-sea.js
Ajustar a qualidade adaptativa	Limites 26 ms, 18,5 ms e 14 s em `sample()` (theme.js)
Ajustar a animação de troca	`set()` e `veilTransition()` (theme.js): duração 900 ms, easing, cortina 520 + 380 ms
Publicar uma nova versão	`python build_static.py`, commit, `git tag -a vX.Y.Z` e `git push --tags`
Preparar o banco pela primeira vez	`python db/aplicar.py` (pede a senha do administrador; grava a senha do papel de leitura fora do Git)
Trocar a senha do papel de leitura	`python db/aplicar.py --rotacionar-senha` (gera nova senha, atualiza o arquivo de segredos e refaz as verificações)
Mudar campos ou o ranking da busca	A view `v_navios` em 01_busca.sql e a consulta `SQL_BUSCA` em consulta_api.py; depois `python db/aplicar.py` e `pytest tests`
Ajustar limites e validação	`limiter.limit(...)`, `EXTRAS_PERMITIDOS` e as regras de `validar_busca()` em consulta_api.py
Rodar os testes de segurança	`pytest tests` · `bandit -r app.py consulta_api.py config_env.py db` · `pip-audit -r requirements-lock.txt`

Variáveis da consulta (banco, hosts, proxy, CORS, limite) e o checklist completo de publicação estão na seção 13.8.

12. Publicação, domínio e segurança

12.1 Por que o localhost:5000 não funcionava no GitHub

`localhost` significa "este computador". O endereço `http://localhost:5000` só existe na máquina de quem executa `python app.py` e apenas enquanto o servidor estiver aberto. Quem abre esse link em outro computador (ou no celular) tenta falar com a *própria* máquina, onde não há nada rodando.

Afirmação	Realidade
"O GitHub roda o meu site"	O GitHub **armazena** o código. Ele não executa Python nem o Flask.
"localhost é um endereço público"	É um endereço **privado** da máquina local (127.0.0.1); ninguém de fora consegue acessá-lo.
"Preciso de um servidor Flask ligado"	Não. O template não usa lógica Jinja e nenhuma rota devolve dados dinâmicos: o site é 100% estático.

Solução adotada: publicar o site como **estático** no GitHub Pages — gratuito para repositórios públicos, com HTTPS automático e sem nenhum servidor seu ligado.

12.2 Como a publicação funciona

```
templates/index.html  +  static/      (fonte de verdade, usada também pelo Flask local)
          |
          |  python build_static.py    troca "../static/" por "static/" e copia os arquivos
          v
       docs/                           versão estática pronta (index.html, static/, .nojekyll)
          |
          |  git push  ->  GitHub Pages (Settings > Pages: branch main, pasta /docs)
          v
https://alansouzadev7.github.io/Naval_Design-WEB-V.1/     (HTTPS, atualiza em ~1 min)
```

- **Por que reescrever os caminhos:** no Flask a página fica em `/` e `../static/` resolve para `/static/`. No Pages o site vive em um subcaminho (`/Naval_Design-WEB-V.1/`), então caminhos relativos simples (`static/...`) são necessários.
- **.nojekyll:** desliga o processamento Jekyll do Pages, que ignoraria arquivos e pastas iniciados por "_".
- **Uma só fonte:** a pasta `docs/` é *gerada*; edite sempre `templates/` e `static/` e rode o build.
- **Verificação:** a versão estática foi testada num subcaminho local antes do envio e, depois de ativada, o endereço público respondeu HTTP 200 para a página e para CSS e JS.

12.3 Segurança

O maior ganho de segurança vem da própria arquitetura: um site estático **não tem servidor, banco de dados, login, formulários nem credenciais** para atacar. Sobre isso foram aplicadas camadas adicionais:

Medida	Onde	O que protege
HTTPS (TLS) automático	GitHub Pages	Criptografa o tráfego e impede adulteração no caminho
Content-Security-Policy (CSP)	`<meta>` em index.html (vale no Pages e no Flask)	Limita de onde scripts, estilos, fontes e imagens podem vir; reduz o impacto de injeção de código (XSS)
SRI (sha512) no three.js	Tag `<script>`	O navegador recusa o script se o arquivo do CDN for alterado
Referrer-Policy	meta + cabeçalho Flask	Envia o mínimo de informação de origem a terceiros
X-Content-Type-Options, X-Frame-Options, Permissions-Policy	Cabeçalhos do Flask (uso local)	Sem "adivinhação" de tipos, sem iframes e sem câmera/microfone/localização
Debug só em 127.0.0.1	app.py	O debugger do Werkzeug executa código; nunca fica exposto na rede
Repositório sem segredos	Todo o código	Não há chaves, senhas ou tokens no código (o push usou o login do próprio Git)

Política CSP aplicada, diretiva por diretiva

Diretiva	Valor	Motivo
default-src	'self'	Padrão restritivo: só o próprio site
script-src	'self' https://cdnjs.cloudflare.com	Scripts do site e o three.js do cdnjs; nenhum script inline
style-src	'self' 'unsafe-inline' https://fonts.googleapis.com	CSS do site e do Google Fonts; *unsafe-inline* é necessário para os atributos `style` (ex.: `--fill` das barras)
font-src	https://fonts.gstatic.com	Arquivos de fonte do Google
img-src	'self' data:	Ícone e máscara de onda embutidos como `data:` URI
connect-src	'self'	Sem chamadas de rede a terceiros
object-src / base-uri / form-action	'none' / 'self' / 'self'	Bloqueia plugins e evita sequestro de links e formulários

Teste da política: com a CSP real, o navegador não registrou nenhuma violação. Como prova de que o teste enxerga violações, uma cópia com a política propositalmente restritiva foi bloqueada de fato — e o site degradou sem quebrar (o 3D foi desativado com um aviso).

Riscos residuais (transparência)

- **Sem cabeçalhos HTTP no Pages:** o GitHub Pages não permite definir cabeçalhos; por isso a CSP vai em *meta*, e a diretiva `frame-ancestors` (anti-clickjacking) não pode ser usada nesse modo.
- **Dependência de terceiros:** cdnjs e Google Fonts. O SRI protege o script; as fontes revelam o IP do visitante ao Google (hospedar as fontes localmente eliminaria isso).
- **'unsafe-inline' em estilos:** aceito por ser só CSS de apresentação; não permite executar scripts.

12.4 Domínio personalizado: royalnavy.com.br

Resultado da investigação (20/09/2026): o domínio **não está registrado**. Isso foi verificado por vias independentes:

Verificação	Resultado	Interpretação
RDAP do Registro.br (royalnavy.com.br)	HTTP 404	Sem registro no Registro.br
Controle: RDAP de google.com.br	HTTP 200	A consulta funciona e diferencia registrado de livre
DNS (nslookup royalnavy.com.br e www)	Non-existent domain	Não há zona DNS
Variações (royalnavy.br, royal-navy.com.br, royalnavybr.com.br, marinhareal.com.br)	HTTP 404	Também disponíveis na data da consulta

O que não foi feito, e por quê. Registrar um domínio é uma compra em nome de uma pessoa (CPF/CNPJ) com pagamento; só o titular pode fazê-la. Além disso, **o arquivo CNAME não foi publicado**: com o domínio inexistente, o endereço github.io passaria a redirecionar para um endereço que não abre (o site cairia) e, enquanto ninguém o registrasse, qualquer pessoa poderia tomá-lo e receber os visitantes do site.

Trava de segurança criada

O script `enable_custom_domain.py` só grava o CNAME depois de conferir, por DNS, que o domínio já aponta para os quatro IPs do GitHub Pages. Foi testado: recusou `royalnavy.com.br` (não resolve) e `example.com` (resolve para outro servidor), sem gravar nada.

```
python enable_custom_domain.py royalnavy.com.br     # ativa (recusa se o DNS ainda não estiver certo)
python enable_custom_domain.py --remove             # volta ao endereço github.io
```

Passo a passo para ativar

- **1. Registrar** royalnavy.com.br em registro.br (conta, CPF/CNPJ, pagamento da anuidade; confirme o valor vigente no site).
- **2. Criar os registros DNS** (tabela abaixo) na zona do domínio.
- **3. Aguardar a propagação** (minutos a algumas horas) e conferir com `nslookup royalnavy.com.br`.
- **4. Ativar no projeto:** rodar o script, depois `git add -A`, `git commit` e `git push`.
- **5. No GitHub** (Settings → Pages): conferir o domínio e marcar **Enforce HTTPS** quando o certificado for emitido (até ~1 h).
- **6. Verificar o domínio** na conta (Settings → Pages → Add a domain, registro TXT), impedindo que terceiros o usem em outro repositório.

Tipo	Nome	Valor	Obrigatório
A	@	185.199.108.153 · 185.199.109.153 · 185.199.110.153 · 185.199.111.153 (um registro por IP)	Sim
AAAA	@	2606:50c0:8000::153 · 2606:50c0:8001::153 · 2606:50c0:8002::153 · 2606:50c0:8003::153	Opcional (IPv6)
CNAME	www	alansouzadev7.github.io	Sim (para o www funcionar)

IPs conferidos na API oficial do GitHub (api.github.com/meta, campo *pages*) em 20/09/2026.

Solução de problemas

Sintoma	Causa provável	O que fazer
O script responde "BLOQUEADO … não resolve"	Domínio não registrado ou DNS ainda não propagou	Conferir o registro e os registros A; aguardar e repetir
O script responde que resolve para outros IPs	Registros A apontando para outro servidor (ou proxy)	Trocar pelos 4 IPs do GitHub; desligar proxies/CDN que ocultem o IP
Pages mostra "DNS check unsuccessful"	Propagação incompleta ou www sem CNAME	Esperar e usar "Check again"; revisar o CNAME do www
"Enforce HTTPS" desativado	Certificado ainda em emissão	Aguardar até ~1 h; se persistir, remover e recolocar o domínio
O site abre sem estilo/3D depois de ativar o domínio	Cache ou caminhos	Ctrl+F5; os caminhos são relativos e não mudam com o domínio
Quero voltar ao endereço github.io	—	`enable_custom_domain.py --remove`, commit, push e limpar o domínio em Settings → Pages

12.5 Checklist de publicação

Item	Estado (20/09/2026; domínio reconferido em 26/09/2026)
Código no GitHub (repositório público)	Concluído
Versão estática em docs/ + .nojekyll	Concluído
GitHub Pages ativado (main /docs) e respondendo HTTP 200	Concluído (ativado pelo titular; verificado por curl)
HTTPS	Concluído (automático no endereço github.io)
CSP, SRI e cabeçalhos	Concluído e testado
Modo escuro (v2.0.0): commit, tag e site atualizado e verificado no ar	Concluído (26/09/2026)
Registro de royalnavy.com.br	**Pendente — depende do titular** (compra)
Registros DNS (A, AAAA, CNAME www)	Pendente — após o registro
enable_custom_domain.py + push	Pendente — após o DNS propagar
Enforce HTTPS e verificação do domínio	Pendente — após ativar o domínio

12.6 Publicar a consulta: o que o GitHub Pages não faz — e o que foi publicado (v4.0)

O GitHub Pages serve arquivos: **não executa Python nem acessa banco de dados**. Por isso a versão em `docs/` mostra a página de consulta em *modo estático*, com um aviso. Para a consulta funcionar na internet foi preciso hospedar o servidor Flask e um PostgreSQL de verdade — o que foi feito na v4.0 (seção 14) usando os mesmos passos de segurança já descritos na seção 13.8. O site em `docs/` (claro e escuro) continua publicado à parte e nenhuma credencial vai para o repositório.

Situação	Consulta ao banco	Restante do site
Local (python app.py + PostgreSQL)	Funciona	Funciona
GitHub Pages (docs/)	Página com aviso (sem servidor)	Funciona
Publicado (Render + Neon) — royalnavy-consulta.onrender.com	**Funciona** (verificado no ar; seção 14.5)	Funciona

13. Consulta de navios: banco de dados e API (v3.0)

A versão 3.0 acrescenta ao site uma **página de consulta**: o visitante digita o nome de qualquer navio (ou a classe, ou a designação do casco) e recebe **todas as informações** que o banco de dados guarda sobre ele. Os dados ficam em um **PostgreSQL local**, lido por uma API Flask somente-leitura. O pedido foi que isso fosse feito *de forma segura, porque o site será publicado no futuro*: por isso a maior parte desta seção é sobre **como o banco fica protegido** — e como isso foi testado.

13.1 O que foi construído

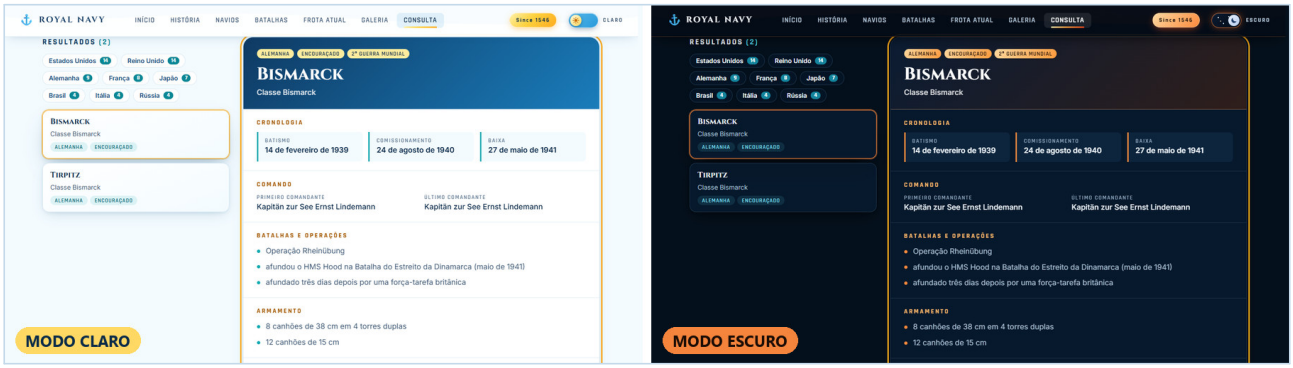


Figura 24 — Página de consulta (1366×768): busca por "bismarck", lista de resultados e ficha completa. Esquerda: modo claro; direita: modo escuro.

Recurso	Como funciona
Busca	Por nome, classe ou designação do casco (ex.: *BB-39*). Ignora acentos e maiúsculas e tolera erro de digitação: "arizonna" encontra o USS Arizona. A busca roda enquanto o visitante digita (após uma pausa de 260 ms).
Explorar por marinha	Botões com a contagem de navios: Estados Unidos 14 · Reino Unido 14 · Alemanha 9 · França 8 · Japão 7 · Brasil 4 · Itália 4 · Rússia 4.
Filtros	Marinha, guerra (1ª, 2ª ou ambas) e tipo, com a contagem de cada opção; combináveis com o texto.
Resultados	10 por página, com paginação; o navio escolhido fica destacado.

Recurso	Como funciona
Ficha do navio	Marca-etiquetas (marinha, tipo, guerra), nome e classe; **cronologia** (batismo, comissionamento, baixa); **comando** (primeiro e último comandante); **batalhas e operações** e **armamento** em listas; **situação atual**; **observações**; e "outros navios da classe".
Endereço compartilhável	A busca e o navio ficam na URL (`?q=bismarck&id;=60`): o link reabre a mesma ficha.
Claro, escuro e celular	Usa o botão de tema, a paleta e a barra de navegação do resto do site; no celular há um atalho de busca (lupa) na barra, pois o menu de links some abaixo de 768 px.
Acessível	Rótulos e `aria-live` para anunciar resultados, foco na ficha ao abrir, link "Ir para o conteúdo", respeita `prefers-reduced-motion`.
Versão publicada (GitHub Pages)	Mostra um aviso explicando que a consulta precisa do servidor; nenhuma requisição inútil é feita.

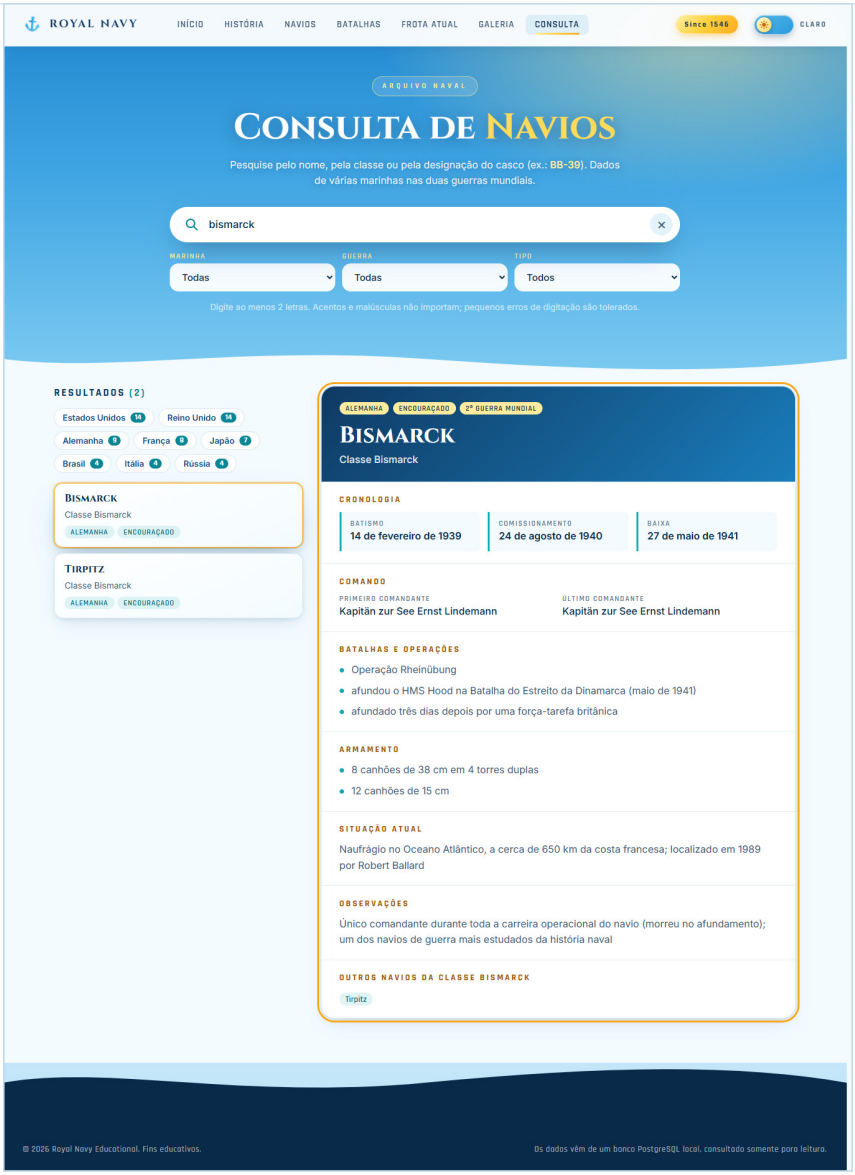


Figura 25 — A página inteira (modo claro): cabeçalho com a busca e os filtros, resultados à esquerda e a ficha de Bismarck à direita.

13.2 Arquitetura e papel de cada tecnologia



Ida: GET /api/navios/busca?q=bismarck → linhas do banco → JSON. Volta: o navegador monta a página com textContent.

O navegador nunca fala com o banco. A porta do PostgreSQL só aceita conexões locais; a API é a única porta de entrada.

Se qualquer etapa falhar (banco fora do ar, dependência ausente), só a consulta responde 503; o restante do site continua no ar.

Figura 26 — O caminho de uma consulta, do navegador ao banco, e o que cada camada controla.

Tecnologia	O que faz na consulta	Arquivos
PostgreSQL 18 + SQL	Guarda os navios; a busca sem acento e por similaridade; a *view* pública; o papel de leitura e seus limites.	db/01_busca.sql · db/02_papel_leitura.sql
Python 3.14 + Flask 3.1.3	Serve a página `/consulta` e a API `/api/navios`; valida o cabeçalho Host; envia CSP e demais cabeçalhos; converte erros em respostas genéricas.	app.py · consulta_api.py
psycopg 3 + pool	Conexão com o banco por um *pool* de até 5 conexões, sempre com consultas **parametrizadas**; recusa usuário administrador.	consulta_api.py
Flask-Limiter	Limite de requisições por IP (janela móvel).	consulta_api.py
python-dotenv + config_env	Lê os segredos de um arquivo fora do Git e da nuvem, com permissão só do usuário.	config_env.py
HTML · CSS · JavaScript	A página: formulário, filtros, resultados, ficha; tudo montado com `textContent`.	templates/consulta.html · static/css/consulta.css · static/js/consulta.js
pytest · Bandit · pip-audit	Testes (88), análise estática de segurança do código Python e auditoria de vulnerabilidades das dependências.	tests/ · requirements-*.txt

13.3 O banco de dados

A tabela original, `public.navios_historicos`, foi **preservada sem nenhuma alteração** (64 navios, 17 colunas). Antes de qualquer mudança foi feito um backup completo (pg_dump, 44 KB, fora do repositório).

Coluna	Tipo	Observação
id	integer	chave primária
nome · designacao_casco	varchar(100) · varchar(50)	nome obrigatório; designação pode ser "N/A"
marinha	varchar(30)	obrigatório; restrito a 8 valores (EUA, Reino Unido, Alemanha, Japão, Itália, Brasil, França, Rússia)
tipo · classe	varchar(50) · varchar(100)	ex.: Encouraçado · Bismarck
guerra	varchar(30)	obrigatório; "1ª Guerra Mundial", "2ª Guerra Mundial" ou "Ambas"

Coluna	Tipo	Observação
data_batismo · data_comissionamento · data_baixa	date	podem ser nulas
primeiro_comandante · ultimo_comandante	varchar(150)	
batalhas · armamento	text	itens separados por ";" (a página os exibe como listas)
estado_atual · fonte_observacoes	varchar(300)	a API expõe a segunda como "observacoes"
criado_em	timestamp	**interna**: não sai pela API

Distribuição: 44 navios da 2ª Guerra, 12 "ambas" e 8 da 1ª; 38 encouraçados, 10 porta-aviões, 4 cruzadores leves, 4 cruzadores de batalha e 8 de outros tipos. Índices já existentes (guerra, marinha, tipo e a chave primária) foram mantidos.

O que foi acrescentado (só acréscimos; nenhum dado foi tocado)

Objeto	Para quê
`unaccent` · `pg_trgm`	Extensões: remover acentos; comparar textos por trigramas (similaridade e busca por trecho)
`rn_norm(texto)`	Função: minúsculas e sem acento ("Encouraçado" → "encouracado"). Declarada IMMUTABLE para poder ser usada em índice
`rn_busca(nome, casco, classe, marinha)`	Função: o texto pesquisável de cada navio (nome + casco + classe + marinha, normalizado)
`idx_navios_busca_trgm`	Índice GIN de trigramas sobre `rn_busca(...)`: acelera a busca por trecho e por similaridade quando a tabela crescer
`idx_navios_nome_norm`	Índice sobre o nome normalizado
`v_navios`	**View pública**: só os campos de consulta (sem `criado_em`), "N/A" vira nulo, `fonte_observacoes` vira `observacoes`. É a única coisa que a API lê
`navios_leitura`	**Papel de leitura**: o único usuário do banco que a API usa (veja abaixo)

```
-- db/01_busca.sql (trecho)
CREATE FUNCTION public.rn_norm(t text) RETURNS text
LANGUAGE sql IMMUTABLE PARALLEL SAFE STRICT
AS $fn$ SELECT lower(public.unaccent('public.unaccent'::regdictionary, t)) $fn$;

CREATE INDEX idx_navios_busca_trgm ON public.navios_historicos
  USING gin (public.rn_busca(nome, designacao_casco, classe, marinha) gin_trgm_ops);

CREATE VIEW public.v_navios AS
SELECT n.id, n.nome, nullif(n.designacao_casco, 'N/A') AS designacao_casco, n.marinha, n.tipo, n.classe, n.guerra,
       n.data_batismo, n.data_comissionamento, n.data_baixa, n.primeiro_comandante, n.ultimo_comandante,
       n.batalhas, n.armamento, n.estado_atual, n.fonte_observacoes AS observacoes,
       public.rn_norm(n.nome) AS nome_norm,
       public.rn_busca(n.nome, n.designacao_casco, n.classe, n.marinha) AS busca_norm
FROM public.navios_historicos n;
```

O papel de leitura: privilégios mínimos

Pode	Não pode (cada item é um teste automatizado)
Conectar (até 10 sessões) e dar SELECT na view `v_navios`	Ler a tabela original · INSERT · UPDATE · DELETE · TRUNCATE · DROP · criar tabelas ou funções · criar papéis · virar superusuário · conceder permissões · ler arquivos do servidor · COPY ... PROGRAM · ler `pg_shadow` · ver colunas internas

Configuração do papel	Valor	Efeito
default_transaction_read_only	on	toda sessão nasce somente-leitura (defesa em camadas)
statement_timeout	3 s	consulta lenta ou maliciosa (ex.: pg_sleep) é cancelada

Configuração do papel	Valor	Efeito
lock_timeout · idle_in_transaction_session_timeout	1 s · 5 s	não segura travas nem transações abertas
CONNECTION LIMIT	10	a API não consegue esgotar as conexões do servidor
search_path	public	evita resolução de nomes em esquemas inesperados
REVOKE … FROM PUBLIC (banco e esquema)	—	só o papel autorizado entra; ninguém herda acesso por padrão

O script `db/aplicar.py` executa os dois SQL, define a senha do papel (43 caracteres aleatórios), grava os segredos e **verifica por dentro**, conectando como o próprio papel, que ele lê a view e não consegue escrever, criar, apagar nem ler a tabela (11 verificações, todas aprovadas). A senha do administrador é pedida no terminal e nunca é gravada.

Com 64 linhas o planejador do PostgreSQL faz uma varredura sequencial (é mais barata que usar índice); forçando o uso de índice, o plano passa a *Bitmap Index Scan on idx_navios_busca_trgm* — confirmando que a expressão do índice coincide com a da view e que ele será usado quando a base crescer.

13.4 A API

Rota (somente GET)	Parâmetros	Resposta	Limite por IP
`/api/navios/busca`	q · marinha · guerra · tipo · pagina	total, página, 10 resultados (id, nome, casco, marinha, tipo, classe, guerra)	30 por minuto
`/api/navios/<id>`	— (id inteiro)	ficha completa + até 6 navios da mesma classe	60 por minuto
`/api/navios/filtros`	—	marinhas, guerras e tipos com contagem (cache de 60 s)	30 por minuto
`/api/saude`	—	funcionamento e estado do banco	geral

Todas as rotas ainda contam para o limite geral de 120 requisições por minuto por IP.

Validação da entrada

Campo	Regra	Se violar
q	2 a 60 caracteres; só letras, dígitos, espaço e `. ' ’ ( ) / -` (após normalização Unicode e colapso de espaços)	400 consulta_invalida
marinha · guerra · tipo	2 a 40/40/60 caracteres, mesma lista de caracteres; comparação exata, parametrizada	400
pagina	inteiro de 1 a 20	400
Parâmetros	só os conhecidos; chave repetida é recusada; URL ≤ 400 bytes; ao menos q ou um filtro	400
id	inteiro positivo (rota `<int>`); sem parâmetros extras	404 / 400
Método e corpo	só GET; corpo ≤ 4 KB	405 / 413

A consulta de busca

```
-- consulta_api.py: parâmetros SEMPRE separados do SQL (psycopg envia o texto como parâmetro)
WITH p AS (SELECT public.rn_norm(%(q)s::text) AS q,
                  replace(replace(replace(public.rn_norm(%(q)s::text), '\', '\\'), '%%', '\%%'), '_', '\_') AS qlike)
SELECT v.id, v.nome, v.designacao_casco, v.marinha, v.tipo, v.classe, v.guerra, count(*) OVER () AS total
FROM public.v_navios v, p
WHERE (%(q)s::text IS NULL OR v.busca_norm LIKE '%%' || p.qlike || '%%' OR word_similarity(p.q, v.busca_norm) >= 0.5)
  AND (%(marinha)s::text IS NULL OR v.marinha = %(marinha)s::text)      -- filtros opcionais, sempre com parâmetro
  AND (%(guerra)s::text  IS NULL OR v.guerra  = %(guerra)s::text)
  AND (%(tipo)s::text    IS NULL OR v.tipo    = %(tipo)s::text)
ORDER BY coalesce(v.nome_norm = p.q, false) DESC,                       -- 1º nome exato
         coalesce(v.nome_norm LIKE p.qlike || '%%', false) DESC,        -- 2º nome que começa com o texto
         coalesce(word_similarity(p.q, v.busca_norm), 0) DESC, v.nome   -- 3º mais parecido
LIMIT %(lim)s OFFSET %(off)s
```

Por que é seguro: o texto do visitante só chega ao SQL como *parâmetro* (nunca concatenado); além disso já passou pela lista de caracteres permitidos e os curingas do `LIKE` são escapados. **Ordem dos resultados:** nome exato, depois nome que começa com o texto, depois o mais parecido (similaridade de trigramas ≥ 0,5, o que permite erros de digitação).

Respostas de erro (sempre JSON, sempre genéricas)

HTTP	erro	Quando	O que o cliente vê
400	consulta_invalida · host_nao_permitido	entrada fora das regras; Host inesperado	mensagem curta e segura (sem eco do texto enviado)
404	nao_encontrado	navio inexistente, id inválido, rota desconhecida	"Navio não encontrado."
405 · 413	metodo_nao_permitido · requisicao_grande	método diferente de GET; corpo grande	mensagem genérica
429	muitas_requisicoes	limite por IP excedido	cabeçalhos `Retry-After` e `X-RateLimit-*`
503	servico_indisponivel	banco fora do ar, sem configuração ou usuário administrador no .env	"O serviço de consulta está indisponível no momento."
500	erro_interno	falha inesperada	"Erro interno. Tente novamente." — o traceback fica só no log do servidor

Desempenho da API

Operação (200 requisições seguidas)	Mediana	p95	Máximo
busca por nome	8,6 ms	10,6 ms	15,8 ms
busca com erro de digitação	8,0 ms	9,8 ms	13,9 ms
busca só com filtro	4,9 ms	6,1 ms	7,6 ms
ficha	4,7 ms	6,4 ms	7,3 ms
filtros (cache)	2,3 ms	3,6 ms	7,2 ms

Medido dentro do processo (Flask + PostgreSQL, sem rede) no computador do projeto; consultas de busca incluem o pool de conexões e a similaridade por trigramas.

13.5 A página de consulta

- **Renderização segura:** toda a interface é montada com `document.createElement` e `textContent` — nunca `innerHTML`. Um texto como `<img src=x onerror=…>` vindo da API aparece como texto.

- **Requisições comportadas:** *debounce* de 260 ms; cada nova busca cancela a anterior (`AbortController`); sem cookies (`credentials: 'omit'`); o id só é aceito se for inteiro positivo.
- **Validação também no navegador:** a mesma lista de caracteres do servidor evita requisições que seriam recusadas (o servidor continua sendo a autoridade).
- **Estados tratados:** carregando, sem resultado, entrada inválida, limite excedido (429), serviço indisponível (503), navio não encontrado e versão estática.
- **Estado na URL:** `history.replaceState` grava `q`, filtros e `id`; ao abrir o link, a busca e a ficha são restauradas.
- **Integração ao site:** link "Consulta" na barra e no rodapé; no celular, uma lupa ao lado do botão de tema (o espaçamento da barra foi reduzido para caber).

```
// consulta.js — todo texto da API entra na página assim (nunca por innerHTML)
function cria(tag, classe, texto) {
  const e = document.createElement(tag);
  if (classe) e.className = classe;
  if (texto !== undefined && texto !== null) e.textContent = texto;     // sempre textContent
  return e;
}
const TEXTO_OK = /^[\p{L}\p{N} .'’()/-]*$/u;                          // mesma regra do servidor
```

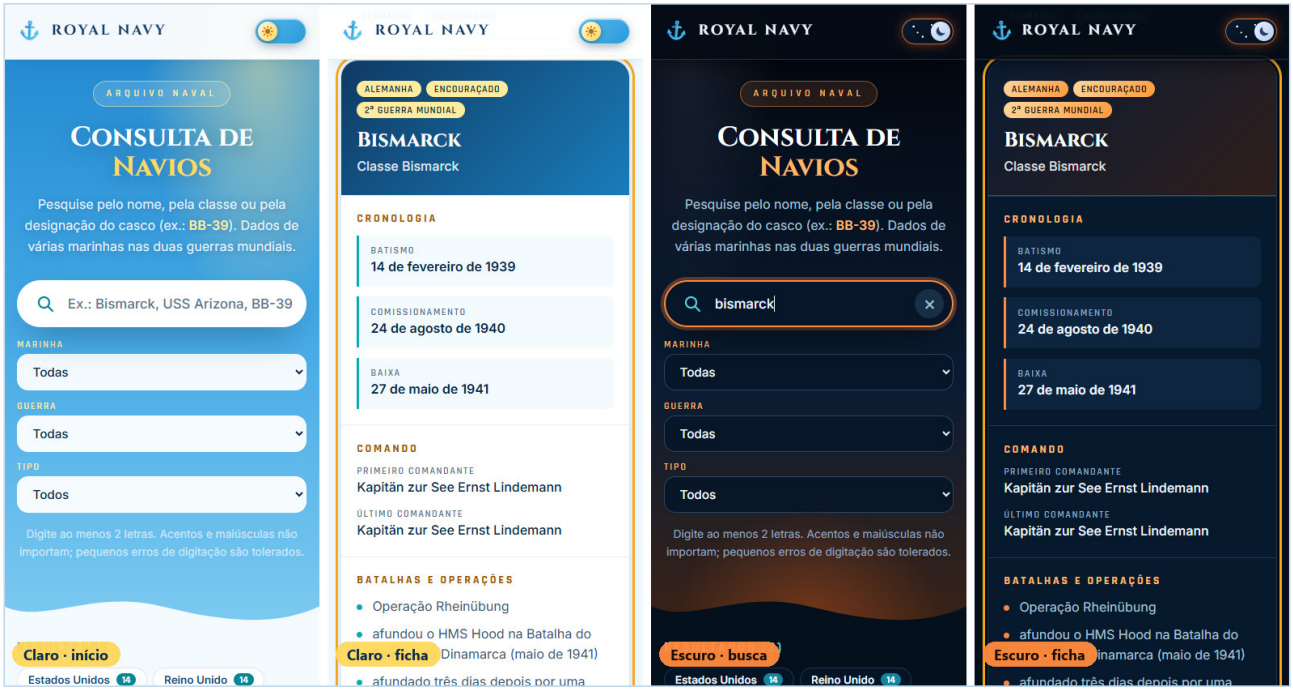


Figura 27 — A consulta no celular (390×844): modo claro (início e ficha) e modo escuro (busca e ficha).



Figura 28 — Versão estática publicada no GitHub Pages (modo escuro): a página explica que a consulta precisa do servidor e não dispara requisições.

13.6 Segurança: modelo de ameaças e defesas

Ponto de partida: o banco estava *local*, protegido só por senha, e a intenção é publicar o site. Foram levantadas as ameaças abaixo e cada uma tem pelo menos uma defesa **testada**.

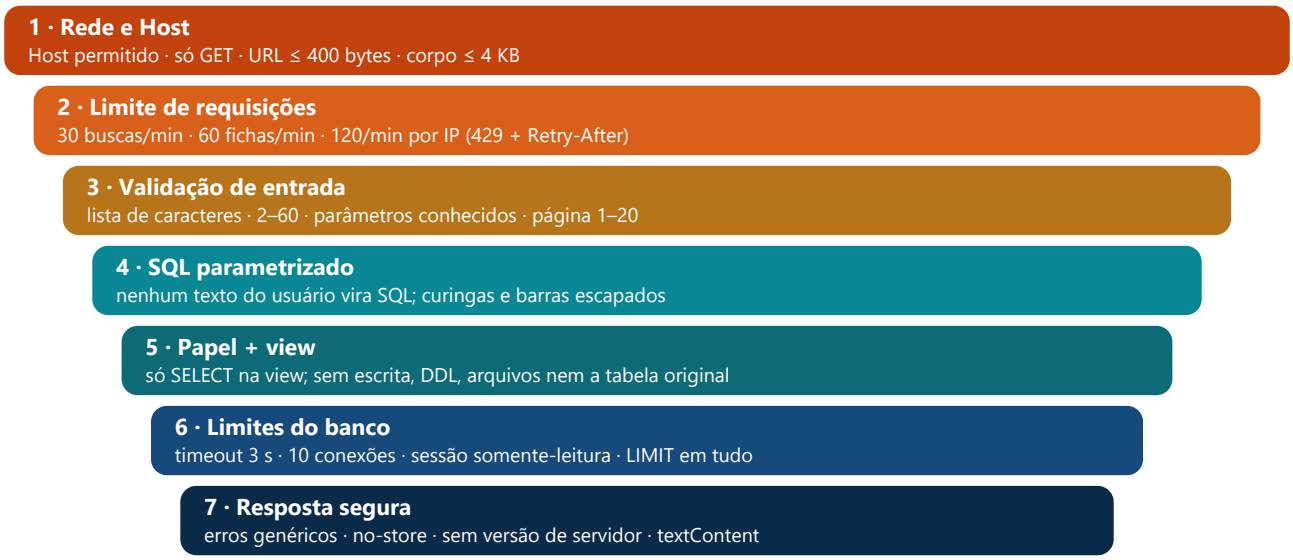


Figura 29 — Defesa em camadas: uma requisição hostil precisa vencer as sete, de fora para dentro; cada camada funciona mesmo se a anterior falhar.

Ameaça	Defesa	Onde está	Como foi verificada
Injeção de SQL	consultas parametrizadas; validação por lista de permitidos; papel sem escrita	consulta_api.py · 02_papel_leitura.sql	22 cargas hostis → nenhuma 500, nenhum vazamento; dados intactos; 15 comandos proibidos ao papel

Ameaça	Defesa	Onde está	Como foi verificada
Banco exposto à rede/internet	a API é a única entrada; PostgreSQL só em localhost; pg_hba só aceita 127.0.0.1/::1 com scram-sha-256	db/README.md · postgresql.auto.conf	porta 5432 com escuta em localhost (após reiniciar o serviço — ação do administrador)
Uso da conta administrativa	a API usa só `navios_leitura` e recusa iniciar com `postgres`/root/admin	consulta_api.py	teste que força `DB_USER=postgres` → 503
XSS (dados ou resposta maliciosos)	`textContent` em toda a interface; CSP sem `unsafe-eval` e sem scripts inline	consulta.js · CSP	resposta hostil injetada no navegador → 0 elementos criados, texto literal
Abuso, raspagem, negação de serviço	limite por IP; LIMIT em toda consulta; ≤ 20 páginas; pool de 5; timeout de 3 s; corpo ≤ 4 KB	consulta_api.py · papel	35 buscas seguidas → 429 a partir da 31ª; pg_sleep(6) cancelado em 3 s
Vazamento de informações	erros genéricos; sem versão do servidor; view sem colunas internas; `Cache-Control: no-store`	app.py · consulta_api.py	testes de cabeçalho e de mensagem de erro
Vazamento de segredos	senha aleatória de 43 caracteres fora do Git e do OneDrive, com permissão só do usuário	config_env.py · aplicar.py	teste varre todos os arquivos versionados atrás da senha e de URIs com senha
DNS rebinding · Host forjado	lista de hosts permitidos (`ALLOWED_HOSTS`)	app.py	Host `evil.example` → 400
Clickjacking	`X-Frame-Options: DENY` e CSP `frame-ancestors 'none'`	app.py	cabeçalhos conferidos em `/` e `/consulta`
Uso indevido entre origens	CORS só por lista exata (nunca *); `Cross-Origin-Resource-Policy: same-origin`	consulta_api.py · app.py	origem não listada não recebe o cabeçalho CORS
Dependências vulneráveis	versões fixas (lock); pip-audit; Bandit	requirements-*.txt	pip-audit: 0 vulnerabilidades após atualizar o Flask; Bandit: 0 achados
CSRF	não se aplica: só GET, sem cookies nem sessão	—	teste de métodos (POST/PUT/DELETE/PATCH → 405)

O que a revisão de segurança encontrou e o que foi feito

#	Achado	Risco	Correção
A1	Flask 3.0.3 tem a vulnerabilidade CVE-2026-27205 (cabeçalho `Vary: Cookie` omitido em certos acessos à sessão), apontada pelo pip-audit	Baixo aqui (o site não usa sessão), mas seria um achado em qualquer auditoria	Atualizado para **Flask 3.1.3**; 88 testes reexecutados; pip-audit: 0 vulnerabilidades
A2	O primeiro arquivo de segredos foi criado dentro da pasta do projeto, que é **sincronizada pelo OneDrive**	A senha do papel de leitura iria para a nuvem	Segredos movidos para fora do projeto (local final: `~/.royalnavy`, ver A10), com permissão só do usuário; arquivo antigo removido; a senha foi **rotacionada**, invalidando a antiga
A3	O PostgreSQL escutava em **todas** as interfaces (`listen_addresses = '*'`), protegido apenas pelo `pg_hba.conf`	Um erro futuro em uma regra abriria o banco à rede	Gravado `listen_addresses = 'localhost'` (`postgresql.auto.conf`); **vale após reiniciar o serviço** (exige administrador — seção 13.6.1)
A4	Conta `postgres` (superusuário) com senha curta, exposta em conversa	Controle total do banco	O site nunca usa essa conta (papel de leitura); troca da senha é tarefa do administrador (seção 13.6.1)
A5	O servidor de desenvolvimento respondia `Server: Werkzeug/3.1.8 Python/3.14.7`	Revela versões a quem procura falhas conhecidas	Cabeçalho `Server` genérico
A6	A CSP existia só como `<meta>`, que não aceita `frame-ancestors`	Clickjacking	CSP também como cabeçalho no Flask (idêntica à meta — um teste garante), com `frame-ancestors 'none'`
A7	Nenhuma validação do cabeçalho Host, nem limite de requisições	DNS rebinding; abuso	Lista de hosts permitidos; Flask-Limiter
A8	Atributo `hidden` ignorado pelo CSS da página (paginação visível com 1 página; bloco vazio na versão estática)	Defeito visual	Regra `[hidden] { display: none !important }` na página de consulta
A9	Com o novo atalho, a barra de navegação estourava no celular (espaçamento de 3 rem)	Botão de tema cortado	Espaçamento reduzido a 0,8 rem em telas estreitas
A10	**Em uso real**, a consulta mostrava "O serviço de consulta está indisponível": o servidor iniciado no VS Code não achava a senha do banco. O arquivo estava em `%LOCALAPPDATA%`, mas o aplicativo que o criou (empacotado, MSIX) **redireciona** o AppData para uma pasta privada; para o servidor do usuário a pasta `RoyalNavy` não existia (erro "caminho não encontrado")	Consulta fora do ar em uso normal, sem o banco ter qualquer defeito	Segredos passaram para `~/.royalnavy/consulta.env` (fora do AppData, do Git e do OneDrive); o arquivo antigo foi removido; confirmado por um processo *fora* do pacote; teste de regressão
A11	O servidor era executado pelo Python global, que **não tinha** as bibliotecas novas (psycopg, Flask-Limiter): a API se desligava em silêncio (503 genérico)	Consulta fora do ar; causa difícil de descobrir	Dependências instaladas nesse Python (Flask 3.1.3 incluído); em modo local a mensagem passou a mostrar o motivo e a dica `python -m pip install -r requirements.txt`; teste garante que o site continua no ar sem as bibliotecas

13.6.1 Tarefas de segurança que dependem do administrador

Duas ações só você pode fazer (exigem a senha do administrador ou permissão de administrador do Windows):
1. Trocar a senha do usuário postgres (a atual é curta e apareceu em conversa). No terminal, digite a nova senha quando o psql pedir: `psql -h localhost -U postgres -c "\password postgres"`. Depois atualize a senha salva no pgAdmin. O site não é afetado.
2. Reiniciar o serviço do PostgreSQL para que ele passe a escutar só em localhost. No PowerShell como administrador: `Restart-Service postgresql-x64-18`. Conferir: `Get-NetTCPConnection -State Listen -LocalPort 5432` deve mostrar só 127.0.0.1 e ::1.

13.7 Testes e resultados

Todos os testes estão em `tests/` e rodam com `pytest tests`: **88 aprovados** em ≈ 7,5 s, contra o banco real e com o papel de leitura.

Grupo	Testes	O que verifica
Funcionamento	7	busca sem acento e maiúscula, por designação do casco (BB-39), tolerância a erro de digitação, filtros e paginação, ficha completa (sem colunas internas), filtros, saúde
Validação de entrada	28	14 entradas inválidas (vazia, 1 letra, 61 caracteres, parâmetro repetido, página 0/21/abc…), limite de 60 caracteres, apóstrofo legítimo, 7 ids/rotas inexistentes, ficha com parâmetro, 4 métodos proibidos
Cargas hostis	23	22 ataques (SQL, XSS, CRLF, Log4Shell, template, traversal, Unicode invisível, entrada gigante…) nunca geram erro 500 nem vazam pistas; depois deles os dados continuam intactos
Cabeçalhos e rede	8	cabeçalhos da API, sem versão de servidor, CSP nas páginas e igual à da meta, Host forjado, CORS por origem, limite de requisições (429 + Retry-After), recusa do usuário administrador
Privilégios do banco	17	15 comandos proibidos ao papel (escrita, DDL, ler a tabela, arquivos, superusuário…), papel sem poderes e view sem colunas internas, timeout cancela consulta de 6 s
Segredos	2	.env no .gitignore; nenhum arquivo versionado contém a senha do banco nem URI com senha
Robustez da configuração	3	segredos em ~/.royalnavy (nunca no AppData); o site continua no ar sem as bibliotecas da consulta (503 com dica só em modo local); o diagnóstico de segredos não vaza o conteúdo
Total	**88**	

Matriz de ataques (resultado real, contra a API)

Ataque	HTTP	Resposta
SQL injection clássica	400	recusado (consulta_invalida)
SQL injection destrutiva	400	recusado (consulta_invalida)
SQL injection (aspas duplas)	400	recusado (consulta_invalida)
Empilhar comando / atraso	400	recusado (consulta_invalida)
UNION para ler senhas	400	recusado (consulta_invalida)
XSS (script)	400	recusado (consulta_invalida)
XSS (evento)	400	recusado (consulta_invalida)
Curinga LIKE	400	recusado (consulta_invalida)
Curinga LIKE	400	recusado (consulta_invalida)
Barra invertida	400	recusado (consulta_invalida)
Byte nulo	400	recusado (consulta_invalida)
Caractere de controle bidirecional	400	recusado (consulta_invalida)
Path traversal	200	0 resultado(s)
Log4Shell	400	recusado (consulta_invalida)
Injeção de template	400	recusado (consulta_invalida)
Bypass de login	200	3 resultado(s)
BOM invisível	400	recusado (consulta_invalida)
Espaço de largura zero	400	recusado (consulta_invalida)
Entrada gigante	400	recusado (consulta_invalida)
Injeção de cabeçalho (CRLF)	400	recusado (consulta_invalida)
Apóstrofo legítimo (deve passar)	200	0 resultado(s)
Consulta normal	200	2 resultado(s)

Os dois casos aceitos (`../../etc/passwd` e `admin'--`) só usam caracteres permitidos: são tratados como *texto de busca*, vão ao banco como parâmetro e não têm nenhum efeito (0 e 3 resultados comuns). Nenhum caminho de arquivo nem comando é alcançável.

Verificações no servidor em execução (HTTP real)

Verificação	Resultado
Páginas e arquivos	/ · /consulta · consulta.js · consulta.css → 200; /api/saude → 200 JSON
Busca real	"yamato" → Yamato e navio da mesma classe (JSON)
Injeção (' OR 1=1 --)	400
Host: evil.example	400 host_nao_permitido
POST na busca	405 metodo_nao_permitido
Cabeçalhos de /consulta	CSP com frame-ancestors 'none', X-Frame-Options DENY, nosniff, Referrer-Policy, Permissions-Policy, COOP e CORP same-origin
35 buscas em sequência	as 28 primeiras (2 já haviam sido gastas) → 200; as demais → 429 com Retry-After e X-RateLimit-*

Análise estática, dependências e interface

- **Bandit** (análise de segurança do código Python): 0 achados. Os 3 avisos de baixa severidade sobre a chamada ao `icacls` foram tratados (caminho completo, sem shell, argumentos próprios) e justificados no código.
- **pip-audit**: 1 vulnerabilidade no Flask 3.0.3 (CVE-2026-27205) → corrigida com a atualização para 3.1.3; após isso, **0 vulnerabilidades conhecidas**.
- **Interface** (Edge, GPU): busca, ficha, filtro por marinha e URL compartilhável funcionam em claro, escuro e celular; sem erros de console; a resposta hostil injetada não criou nenhum elemento (0 imagens, 0 scripts).
- **Sem regressões:** a página inicial (claro e escuro) segue sem erros, agora com o link "Consulta"; a versão estática também.

Limitações

- As medições de latência foram feitas em um só computador e dentro do processo (sem rede); o limitador usa memória de um processo — em produção com vários processos é preciso Redis.
- O reinício do serviço PostgreSQL (escuta só em localhost) e a troca da senha do administrador dependem do administrador (seção 13.6.1).
- Os testes de interface foram feitos em Chromium/Edge; não houve teste de invasão feito por terceiros.
- Os textos pesquisados aparecem na URL e, portanto, nos registros de acesso do servidor: em produção é preciso definir o prazo de retenção (LGPD).

13.8 Como executar e como publicar

```
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
python db/aplicar.py        # 1 vez: prepara o banco; pede a senha do administrador (não é gravada)
python app.py               # http://localhost:5000  ·  consulta em /consulta

pip install -r requirements-dev.txt
pytest tests                # 88 testes
bandit -r app.py consulta_api.py config_env.py db build_static.py
pip-audit -r requirements-lock.txt
```

Variável	Padrão	Função
DB_HOST · DB_PORT · DB_NAME	localhost · 5432 · postgres	onde está o banco
DB_USER · DB_PASSWORD	navios_leitura · (aleatória)	sempre o papel de leitura; gravadas em config_env (fora do Git)
DB_SSLMODE · DB_POOL_MAX	prefer · 5	TLS com o banco (use verify-full em banco remoto) · tamanho do pool
ALLOWED_HOSTS	localhost,127.0.0.1,[::1]	hosts aceitos; ao publicar, o seu domínio
TRUST_PROXY_HOPS · RN_HTTPS	0 · desligado	confiar em N proxies (nginx/Cloudflare) · enviar HSTS
CORS_ORIGINS	vazio	origens externas autorizadas (lista exata)
RATELIMIT_STORAGE_URI	memory://	use Redis com mais de um processo
RN_ENV_FILE	~/.royalnavy/consulta.env	outro caminho para o arquivo de segredos
RN_API_ORIGIN (no build)	vazio	ao gerar o site estático, aponta a consulta para uma API publicada (https)

Para publicar de verdade (checklist)

#	Item	Por quê
1	Hospedar o servidor (Render, Fly.io, Railway ou VPS) atrás de gunicorn/waitress e de HTTPS; nunca `app.run` nem debug	o GitHub Pages não executa Python nem acessa banco; o servidor de desenvolvimento não é para produção
2	PostgreSQL dedicado ao site, com os dois SQL aplicados; porta 5432 liberada só para o IP do servidor; `hostssl` no pg_hba; `DB_SSLMODE=verify-full`	o banco nunca fica aberto à internet e o tráfego é criptografado
3	`ALLOWED_HOSTS`=seu domínio · `TRUST_PROXY_HOPS=1` · `RN_HTTPS=1` · `FLASK_DEBUG=0`	Host validado, IP real para o limite, HSTS, sem depurador
4	Limitador em Redis (`RATELIMIT_STORAGE_URI`) se houver mais de um processo	o contador em memória não é compartilhado
5	Site estático em outro domínio: `RN_API_ORIGIN=https://api… python build_static.py` e `CORS_ORIGINS` no servidor	libera só a origem certa na CSP e no CORS
6	Senha longa e aleatória, backups automáticos testados; trocar a senha do administrador	resiliência e menor dano em caso de vazamento
7	Definir retenção de logs (a busca aparece na URL); rodar pytest, Bandit e pip-audit a cada atualização	privacidade (LGPD) e manutenção da segurança

Solução de problemas: "O serviço de consulta está indisponível"

Em modo local (`python app.py`) a própria mensagem da página mostra o motivo entre parênteses. Só nesse modo: em produção a mensagem é sempre genérica.

Motivo mostrado	Causa	O que fazer
`No module named 'psycopg'` (ou Flask-Limiter)	As dependências não estão no **mesmo Python** que roda o site	`python -m pip install -r requirements.txt` com esse Python e reiniciar o servidor
`DB_USER/DB_PASSWORD nao configurados` (+ lista dos locais procurados)	O arquivo de segredos não foi encontrado (ou foi criado onde o servidor não enxerga, como o AppData redirecionado)	`python db/aplicar.py` (grava em `~/.royalnavy/consulta.env` e lista os caminhos); não usar AppData
Erro de conexão ou de autenticação	PostgreSQL parado ou senha do papel diferente da do arquivo	`Get-Service postgresql-x64-18`; `python db/aplicar.py` refaz a senha e as verificações

14. Disponibilidade online (v4.0)

Até a v3.0, a consulta ao banco só funcionava com `python app.py` rodando no computador de quem a desenvolveu (seção 12.6). Esta seção documenta como ela passou a funcionar de **qualquer aparelho**, na internet, em `https://royalnavy-consulta.onrender.com` — incluindo dois problemas reais encontrados no caminho e como foram corrigidos.

14.1 Por que não uma VM na nuvem

A primeira ideia foi manter a mesma arquitetura local (PostgreSQL e Flask na mesma máquina), só que numa VM gratuita na nuvem, atrás de um túnel. Duas tentativas de conta esbarraram no cadastro do provedor, antes mesmo de chegar a criar a VM:

Provedor	O que ofereceria	Resultado do cadastro
Oracle Cloud (Always Free, VM Ampere A1)	Instância Arm gratuita permanente	Não concluído (verificação de identidade)
Google Cloud (Always Free, e2-micro)	Instância x86 gratuita permanente	Não concluído (mesma etapa)

Diante disso, a arquitetura foi trocada para dois serviços gerenciados com cadastro só por e-mail ou GitHub, sem cartão: **Render** para a aplicação e **Neon** para o PostgreSQL. Nenhum servidor próprio para administrar.

14.2 Arquitetura publicada

Componente	Onde	O que muda em relação ao local (seção 13)
Aplicação (Flask)	Render, plano gratuito	Mesmo código; roda com `gunicorn` (1 processo, 4 threads) em vez do servidor de desenvolvimento
Banco (PostgreSQL)	Neon, plano gratuito	Mesma view `v_navios`, mesmo papel `navios_leitura` e as mesmas regras (seção 13.5); a conexão passa a exigir `DB_SSLMODE=verify-full` de verdade
Site (claro e escuro)	GitHub Pages, inalterado	Continua em modo estático (seção 12); nesta versão não foi ligado à API publicada — ver 14.8

Nesta v4.0 o próprio Render serve o site inteiro (página e API), não só a API: mais simples de configurar do que separar a interface no GitHub Pages e apontá-la para a API com `RN_API_ORIGIN` (esse recurso já existia desde a seção 13.8, mas não foi usado aqui).

14.3 Carregar os dados no Neon: uma pegadinha de protocolo

O SQL Editor do Neon roda no navegador: envia comandos SQL e lê o resultado por HTTP. O comando `COPY ... FROM stdin ... \.` usado para carregar os 64 navios (seção 13, backup verificado) não é SQL padrão — é um recurso do **protocolo** do psql/libpq, em que o cliente intercepta o comando e passa a transmitir as linhas seguintes numa fase separada da conversa com o servidor. O editor aceitou o `TRUNCATE` que vinha antes, mas travou no `COPY`.

No painel do Neon, o resultado apareceu em duas abas: `1: TRUNCATE` (bem-sucedida) e `2: ERROR` — o script parou ali, sem chegar às linhas seguintes. A correção foi gerar um `INSERT INTO navios_historicos (...) VALUES (...), (...), ...;` equivalente, a partir do mesmo backup já conferido (os mesmos 64 navios, com escape automático de aspas simples nos nomes de comandantes) — SQL padrão, sem depender de nenhum recurso de protocolo. Rodou em um único comando, com o resultado `INSERT 64` confirmando a contagem.

Um segundo tropeço, menor: a primeira instrução deste documento usou `SEU_BANCO` como espaço reservado para "o nome do seu banco no Neon", mas foi interpretado como um nome literal a digitar — o papel de leitura chegou a ser criado sem

permissão de conectar no banco real (neondb), porque as instruções `REVOKE/GRANT ... ON DATABASE` miravam um banco inexistente. Corrigido ao reexecutar o mesmo script com o nome certo.

14.4 O certificado TLS: verify-full sem raiz não verifica nada

`DB_SSLMODE=verify-full` já estava previsto desde a v3.0 para banco remoto (seção 13.8), mas nunca tinha sido testado contra um banco de verdade. Na primeira tentativa:

```
root certificate file "/opt/render/.postgresql/root.crt" does not exist
Either provide the file, use the system's trusted roots with sslrootcert=system,
or change sslmode to disable server certificate verification.
```

Adicionar `sslrootcert="system"` (para usar os certificados confiáveis do próprio sistema operacional) trocou o erro por outro:

```
connection failed: ... SSL error: certificate verify failed
```

Diagnóstico (fora do Render, direto pelo terminal, sem tocar em nenhuma senha):

```
openssl s_client -connect ep-blue-thunder-b643ndyv.c-2.sa-east-1.aws.neon.tech:5432 \
  -servername ep-blue-thunder-b643ndyv.c-2.sa-east-1.aws.neon.tech -starttls postgres -showcerts

depth=3 C=US, O=Internet Security Research Group, CN=ISRG Root X1
depth=2 C=US, O=ISRG, CN=Root YR
depth=1 C=US, O=Let's Encrypt, CN=YR1
depth=0 CN=*.c-2.sa-east-1.aws.neon.tech
```

A cadeia do Neon está correta e validou sem erro nesta máquina (`Verify return code: 0 (ok)`). A causa não era o certificado do Neon: é que `psycopg[binary]` traz **sua própria cópia** do OpenSSL/libpq dentro do pacote Python, compilada em outra máquina — o caminho padrão que ela usa para "os certificados do sistema" nem sempre é o mesmo caminho real do contêiner onde o pacote acaba rodando (no caso, o do Render). `sslrootcert=system` depende dessa coincidência, que aqui não aconteceu.

Correção: incluir no próprio projeto uma cópia da raiz pública e estável da Let's Encrypt (`ISRG Root X1`, autoassinada, válida até 2035) e apontar `sslrootcert` diretamente para esse arquivo — sem depender de adivinhar onde o sistema guarda as suas raízes.

Antes de publicar a correção	Depois
`sslrootcert="system"` — depende do contêiner	`sslrootcert=` caminho fixo para `deploy/isrg-root-x1.pem` (arquivo do projeto)

Conferido antes de publicar, com o certificado real do Neon e essa raiz exata (para os dois endereços — com e sem `-pooler`):

```
openssl s_client ... -CAfile deploy/isrg-root-x1.pem -verify_return_error
Verify return code: 0 (ok)
```

A verificação completa do certificado (identidade do servidor, não só a criptografia) segue ativa — nada foi enfraquecido para "fazer funcionar". Os 88 testes locais continuam passando (o ajuste não altera o modo local, que usa `DB_SSLMODE=prefer`, onde `sslrootcert` é ignorado).

14.5 Verificação real no ar

O que foi testado	Resultado
/api/saude	`{"banco": "ok", "ok": true}` — HTTP 200
Busca por "Bismarck"	2 navios: Bismarck e Tirpitz (mesma classe, `Classe Bismarck`)
Ficha completa (Bismarck)	Cronologia, comando, batalhas, armamento e situação atual carregados do banco, sem colunas internas
Contagem por marinha (filtros)	64 navios distribuídos nas 8 marinhas, batendo com o total (seção 13)

14.6 Limites do plano gratuito (esperado, não é defeito)

- **Render "hiberna"** a aplicação depois de ≈ 15 min sem acessos; o próximo pedido acorda o serviço, com alguns segundos a dezenas de segundos de atraso.
- **Neon suspende** o cômputo do banco de forma parecida depois de um tempo sem consultas; acorda sozinho na próxima conexão.
- **Reimplantação automática** a cada `git push` não disparou sozinha neste serviço (criado a partir do `Blueprint`); foi preciso usar **Manual Deploy** no painel do Render a cada correção — registrado aqui como observação operacional, não como falha de segurança.

Achados desta etapa (numeração segue a da seção 13)

#	Achado	Risco	Correção
A12	O SQL Editor do Neon (web) não executa `COPY ... FROM stdin`	Carga de dados falha silenciosamente na metade do script	Substituído por `INSERT INTO ... VALUES` (SQL padrão), gerado a partir do backup já conferido
A13	`sslrootcert="system"` não é confiável com o OpenSSL empacotado dentro do `psycopg[binary]`	`verify-full` falha em produção mesmo com o certificado do banco correto	Raiz `ISRG Root X1` fixa no projeto (`deploy/isrg-root-x1.pem`), conferida por `openssl` antes de publicar
A14	O Render não reimplanta sozinho a cada push neste tipo de serviço	Uma correção enviada ao GitHub pode continuar fora do ar até alguém notar	`Manual Deploy` confirmado depois de cada envio, como parte do processo (seção 14.6)

14.7 Segurança: o que ficou igual e o que ficou mais forte

O modelo de ameaças continua o da seção 13.2: a API só lê a view `v_navios` por um papel sem permissão de escrita, com validação estrita da entrada, limite de requisições e consultas sempre parametrizadas. O que muda na publicação:

- O tráfego entre o Render e o Neon agora é **TLS com verificação completa do certificado** (`verify-full`), algo que o modo local (rede interna da própria máquina) nunca precisou nem testou antes desta versão.
- O banco nunca fica acessível a partir de um endereço fixo controlado por este projeto: é o Neon quem administra rede, backups e o próprio PostgreSQL.
- A senha do papel `navios_leitura` usada no Neon é **diferente** da senha usada no banco local — gerada à parte, nunca reaproveitada entre os dois ambientes.
- Os cabeçalhos de segurança, a CSP, o limite de requisições e a recusa a usuários administradores (seção 13) valem exatamente como antes: o código da API não mudou, só o `host` do banco e a verificação do certificado.

14.8 O que ficou pendente

Item	Detalhe
Registrar royalnavy.com.br	Mesma pendência da seção 12.4 (compra é do titular); quando resolvida, pode apontar para a publicação em vez do endereço onrender.com
Site (GitHub Pages) ainda não aponta para a API publicada	`python build_static.py` com `RN_API_ORIGIN=https://royalnavy-consulta.onrender.com` ligaria o site estático à consulta ao vivo (recurso já existente desde a seção 13.8)
Limitador de requisições em memória	Serve bem com 1 processo (a situação atual); se o tráfego justificar mais processos, precisa de Redis (seção 13.8)
Rotação periódica da senha do Neon	Mesmo procedimento de `db/aplicar.py --rotacionar-senha`, adaptado para apontar ao banco remoto (seção 13.6)

15. Tema claro “Mar em Paz” (v4.1)

A v4.1 redesenhou o modo claro como o **contraponto direto** do modo escuro: a mesma ideia de profundidade (descer da superfície até o leito do mar, com medidor de profundidade), mas em vez de guerra, **paz** — um comboio de navios de carga e de guerra rumo ao horizonte, águas claras com vida marinha, minas sendo inspecionadas, submarinos em patrulha e, no fim, **naufrágios transformados em recifes de coral**. O tema escuro não foi tocado. Esta seção registra o que foi construído, como, o teste de fluidez e a correção da barra de navegação que apareceu no caminho.

15.1 O conceito: dois temas, a mesma viagem

Trecho da página	Modo escuro — “Guerra em alto mar” (v2.0)	Modo claro — “Mar em Paz” (v4.1)
Hero	Batalha naval noturna (war-hero.js)	Comboio de cargueiros e navios de guerra em **perspectiva**, rumo ao horizonte (convoy-hero.js)
Ao descer	Água escurece, neve marinha, submarinos, clarões	Água clara e luminosa, bolhas, cardumes, golfinho; medidor `0 → 40 m` com 4 zonas
Faixa 1	Guerra submarina (torpedos, sonar, avarias)	**Zona de patrulha**: minas inspecionadas por um ROV, submarino com sonar, tubarões, tartaruga, águas-vivas
Faixa 2	Navios naufragados no escuro	**Jardins de Coral**: naufrágios cobertos de corais, peixes, arraia, anêmonas com peixe-palhaço
Rodapé	Leito escuro	Areia clara com cáusticas (luz que ondula no fundo)
Arquivos	`dark.css`, `war-hero.js`, `deep-sea.js`	`light.css`, `convoy-hero.js`, `calm-sea.js`

Cada tema só existe enquanto está ativo: os canvases do outro ficam `display: none` e não consomem CPU. Os dois usam o mesmo controlador de hero (`RNHero`, seção 4.1) e o mesmo monitor de desempenho (`RNPerf`, seção 5).



Figura 30 — Hero do modo claro (1366×768): o comboio segue para o horizonte, com a costa de falésias e o farol à esquerda e o sol à direita.

15.2 Hero: um comboio em perspectiva (convoy-hero.js)

O hero é um canvas 2D que simula uma **câmera de verdade**, sem WebGL: cada navio é um conjunto de prismas 3D projetados na tela com a divisão pela profundidade, o que dá a convergência para um único **ponto de fuga** — a sensação de “indo embora” que foi pedida. A câmera fica a `CAM_H = 70` unidades acima da água e o mundo corre ao contrário, na velocidade `SPEED = 34`, entre `Z_NEAR = 30` e `Z_END = 9500`.

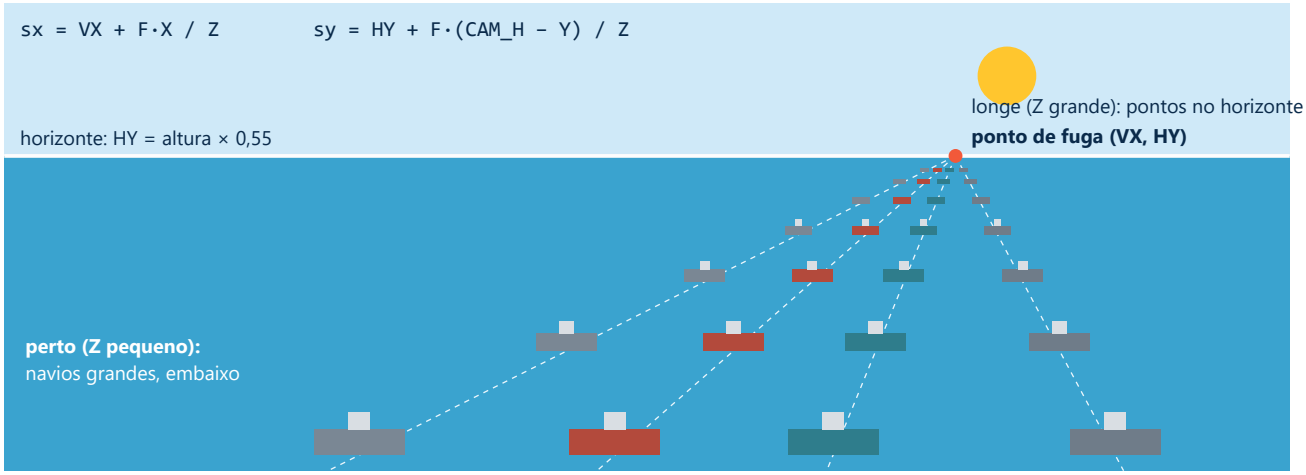


Figura 31 — Projeção do hero: as quatro pistas convergem para o ponto de fuga; o tamanho e a altura na tela de cada navio são proporcionais a 1/Z.

Constante	Valor	Função
`HZ`	0,55	Altura do horizonte como fração da tela
`VPX`	0,74	Posição horizontal do ponto de fuga (à direita, junto ao sol)
`SUN`	{0,78 · 0,24}	Posição do sol na tela; o brilho na água fica sob ele
`CAM_H`	70	Altura da câmera sobre o mar

Constante	Valor	Função
`Z_NEAR / Z_END`	30 / 9500	Onde o navio surge e onde some (já além da névoa)
`SPEED`	34	Velocidade do comboio (unidades por segundo, vezes `dt`)

Pistas e modelos

Pista (x)	Quem navega	Espaçamento
−560	Escoltas: destróier Type 45, fragata Type 23	2.600
−320 e −115	Mercantes: porta-contêineres, petroleiro, graneleiro, transportador de carros	1.300
+150	Fragata, porta-aviões classe Queen Elizabeth, destróier	2.300

Os modelos são **prismas convexos** (caixas, cunhas e troncos) montados à mão: o porta-contêineres tem duas pilhas por baia (nível de detalhe 2) mais quatro blocos intermediários (nível 1); os navios de guerra têm casaria, mastros e convés de voo. Por serem convexos, dá para descartar as faces de trás pela normal de cada aresta (*back-face culling*) e desenhar as restantes em **ordem do pintor** — da proa para a popa e, na mesma linha, de fora para dentro — sem precisar de z-buffer.

Técnica	Para que serve
Ordem do pintor + descarte de faces	Sem z-buffer: cada prisma é desenhado de trás para a frente; faces voltadas para longe e faces de borda (quase de perfil) nem são traçadas
Fusão de faces da mesma cor	Faces vizinhas com a mesma cor viram um só preenchimento (menos chamadas de desenho)
Névoa atmosférica (*haze*)	Cores dos navios distantes se misturam ao céu; os níveis de névoa são pré-calculados, sem misturar cores a cada quadro
LOD (nível de detalhe)	Menos de 4 px: um ponto (`speck`); menos de 11 px: modelo simplificado; menos de 34 px: detalhe 1; acima disso, detalhe 2
Esteiras de Kelvin	Cada navio deixa o “V” de espuma; também com LOD (menos detalhe ao longe)
Mar	14 ondulações retas (`SWELL_GAP = 180`, 3 faixas de transparência), marolas e o brilho do sol em retângulos finos
Cenário	Costa com falésias brancas e farol, helicóptero, gaivotas e nuvens; céu, costa e mar vêm de um fundo pré-renderizado (`buildBg`), refeito só ao redimensionar

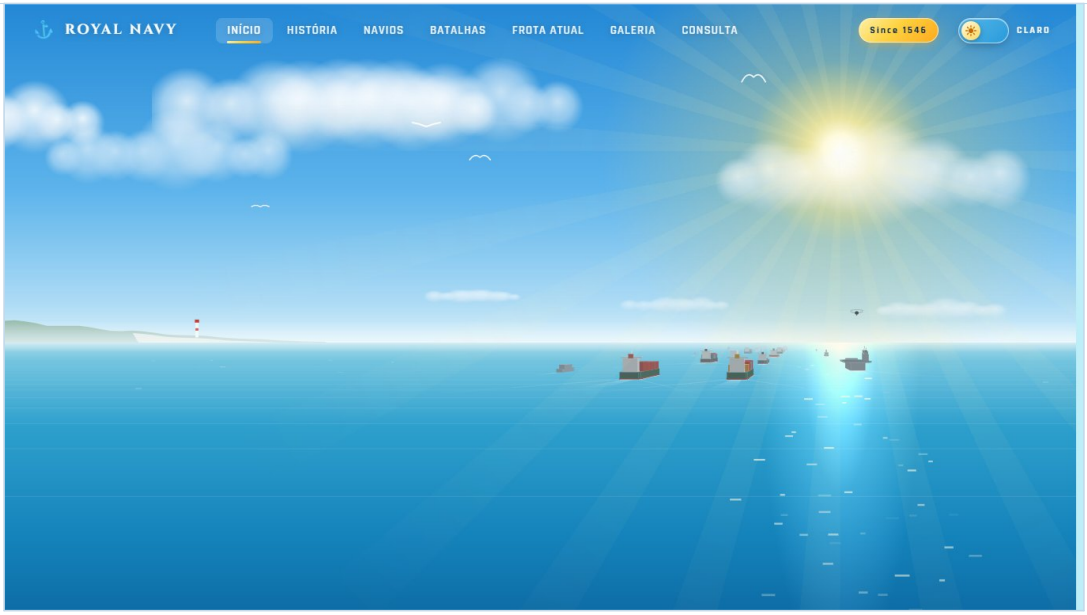


Figura 32 — O mesmo hero sem o texto (capturado com o conteúdo oculto): a perspectiva fica evidente; do mais perto ao mais longe, os navios diminuem até virar pontos no horizonte.

15.3 O mergulho: medidor de 0 a 40 m

Ao rolar a página, um único canvas fixo (`.calm-canvas`, desenhado em **meia resolução**, `SC = 0,5`) pinta a coluna d’água e muda a cor do céu claro para o turquesa e depois para o azul do recife (`STOPS`). O medidor `#calm-gauge` acompanha a rolagem e muda de zona:

Profundidade	Zona	Onde aparece
0 m	Superfície	Logo abaixo do hero (seção História)
2 m	Águas rasas	Início da descida
12 m	Mar aberto	Faixa “Zona de patrulha” (a Figura 34 foi tirada aos 18 m)
30 m	Recife de coral	Faixa “Jardins de Coral” e rodapé



Figura 33 — Logo abaixo do hero: raios de luz filtrados pela superfície, cardume e golfinho, e o medidor em “1 m — Superfície”.

15.4 Zona de patrulha: paz com elementos navais

Faixa própria (`patrol-canvas`), animada só enquanto visível. Junta o pacífico ao naval — como pedido:

- **Minas submarinas** (5), presas ao fundo por correntes que desaparecem em degradê (`drawChain`, 4 elos que perdem a opacidade); cada uma ganha uma etiqueta verde de *inspecionada* quando o robô passa.
- **ROV caça-minas** amarelo, com cone de luz e cordão umbilical até a superfície, que vai de mina em mina.
- **Submarino** que emite pings de sonar em anéis, mais um segundo ao longe; quilhas de navios passam na superfície.
- **Vida marinha:** tubarões, cardumes de fusilheiros, tartaruga, águas-vivas, bolhas.



Figura 34 — Zona de patrulha (18 m, “Mar aberto”): submarino, minas com etiqueta verde, o ROV com seu cone de luz, tubarões, tartaruga e águas-vivas.

15.5 Jardins de Coral: naufrágios viraram recifes

A última faixa (`reef-canvas`) mostra o casco de um galeão e de navios de guerra no leito, cobertos de **colônias de coral** (cérebro, leque, esponjas, anêmonas, ouriços e estrelas-do-mar). Peixes-cirurgião orbitam o galeão, há cardumes, um tubarão, uma arraia sobre a areia e peixes-palhaço nas anêmonas. O piso de areia tem **cáusticas** — o desenho de luz que a superfície projeta no fundo — geradas uma única vez como um ladrilho (ruído de Worley) e só deslocadas a cada quadro.



Figura 35 — Jardins de Coral (37 m): o galeão e os cascos naufragados, ainda legíveis sob os corais; areia com cáusticas.

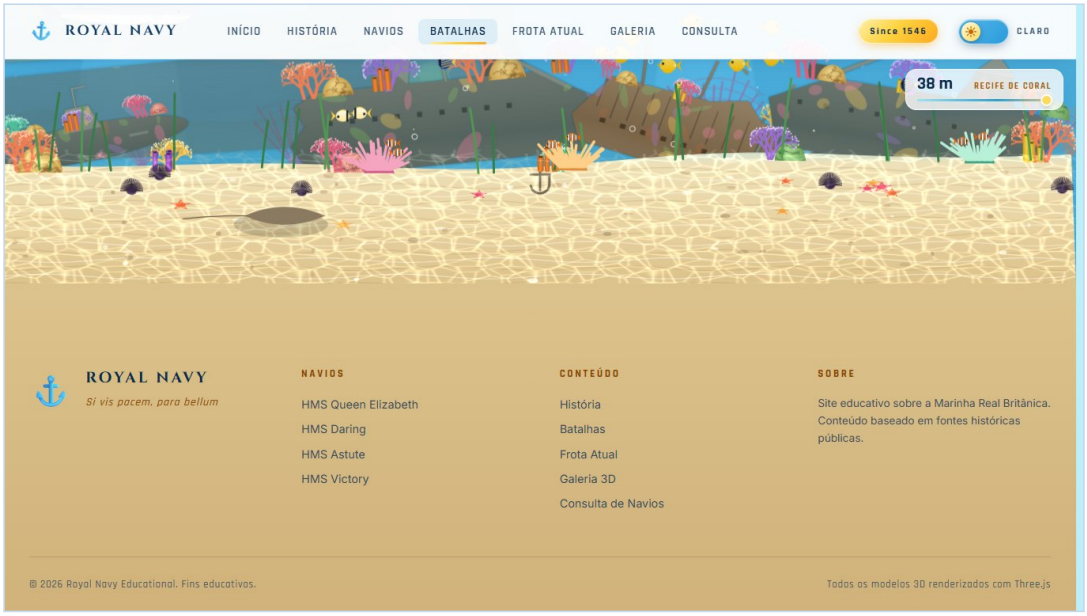


Figura 36 — Rodapé em areia clara (#dcc28c → #d0b27a), continuação do leito do recife.

15.6 Teste de fluidez

Como foi medido. Edge (Chromium) controlado por DevTools, com GPU real (`--use-angle=d3d11`), janela 1366×768, monitor de 120 Hz. Para cada um de sete pontos da página, o script rola até lá, espera 3 s e registra **6 s** de intervalos entre quadros (`requestAnimationFrame`); em paralelo lê o tempo de tarefa da thread principal (`Performance.getMetrics`). Antes de medir há **15 s de aquecimento** (carga dos sprites e compilação do JavaScript). São 3 rodadas por tema, alternando claro e escuro; a tabela mostra a **mediana** das rodadas (o pior quadro é o maior das três). Como o monitor é de 120 Hz, o teto é ≈ 120 fps; 60 fps já seria fluido.

Ponto da página (claro)	FPS	ms / quadro	p95 (ms)	Pior quadro (ms)	Quadros > 20 ms	CPU da thread principal	Nível RNPerf
hero	104,0	9,6	16,6	25,6	0	55 %	2 (alto)
mergulho	113,3	8,8	16,5	24,6	0	52 %	2 (alto)
navios (WebGL)	119,2	8,4	8,5	17,2	0	75 %	2 (alto)
patrulha	120,2	8,3	8,5	8,9	0	41 %	2 (alto)
batalhas	120,2	8,3	8,5	16,6	0	30 %	2 (alto)
recife	105,7	9,5	16,7	25,3	1	74 %	2 (alto)
rodapé	120,2	8,3	8,5	16,6	0	34 %	2 (alto)

Tabela — Modo claro em 1366×768, mediana de 3 rodadas de 6 s cada (≈ 700 quadros por ponto). Em todos os pontos, no máximo 1 quadro passou de 20 ms (mediana) e o nível de qualidade ficou sempre no máximo.

Ponto	Claro: FPS	Claro: p95	Claro: CPU	Escuro: FPS	Escuro: p95	Escuro: CPU
hero (claro: comboio · escuro: batalha)	104,0	16,6	55 %	86,7	16,9	57 %
mergulho	113,3	16,5	52 %	93,1	16,7	57 %
navios (WebGL)	119,2	8,5	75 %	118,4	8,5	75 %
batalhas	120,2	8,5	30 %	119,7	8,5	41 %
rodapé	120,2	8,5	34 %	120,1	8,5	54 %

Tabela — Mesmos pontos nos dois temas. Só os pontos que existem nos dois temas são comparados (as faixas de patrulha e de recife são exclusivas do claro). O tema escuro, já publicado na v2.0, também rodou sem problemas (nível máximo quase sempre), mas com mais quadros acima de 20 ms no hero, onde a batalha tem muitas partículas.

Ponto (viewport de celular 375×812)	FPS	p95 (ms)	CPU
hero (comboio)	107,0	16,7	54 %
mergulho (superfície)	108,4	16,6	60 %
navios (WebGL)	120,2	8,5	39 %
patrulha	120,1	8,6	32 %
batalhas	120,0	8,7	25 %
recife	120,1	8,5	37 %
rodapé (areia)	120,1	8,5	24 %

Tabela — Modo claro com o viewport emulado de celular (uma rodada). **Atenção:** foi o mesmo computador com GPU dedicada, só com a janela e o toque emulados — isto não substitui um teste em um aparelho real.

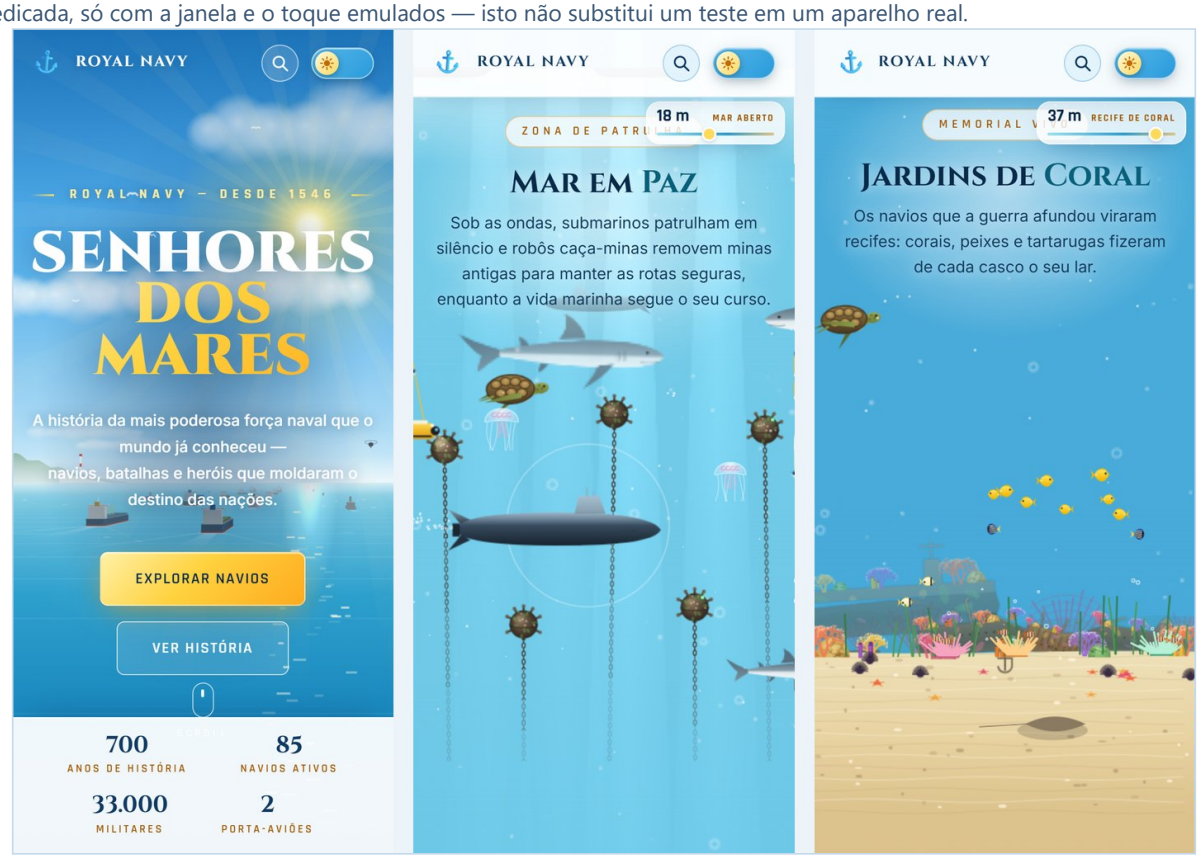


Figura 37 — Modo claro no celular (375×812): hero, zona de patrulha e recife.

O que o teste e o desenvolvimento revelaram (e como foi resolvido)

#	Achado	Correção
L1	Sem aquecimento, a primeira medição após carregar mostrou travadas de 100–200 ms e o RNPerf caindo ao nível 0 (o pior nível), em **ambos** os temas	Fenômeno dos primeiros segundos (carga de sprites e compilação). O protocolo passou a incluir 15 s de aquecimento; o efeito **não foi investigado a fundo** (seção 15.9)
L2	Faixas de luz ("raios") em cada faixa do mergulho dobraram o custo de desenho e deixavam uma **emenda visível** entre as faixas	Removidas; o efeito de luz ficou no canvas fixo único (comentário no código explica)
L3	Erro de precedência de operadores no agrupamento das ondulações em 3 faixas de transparência	Corrigido com `Math.min(2, Math.floor(...)) !== band`
L4	As correntes das minas terminavam de forma abrupta	`drawChain`: 4 elos que perdem a opacidade até sumir
L5	Naufrágios pouco legíveis sobre a água clara	Casco mais escuro, contorno (sombra por `destination-over`) e menos corais, porém maiores
L6	Benchmark sintético reutilizando o mesmo canvas tinha um custo fixo de 8,3 ms em `getImageData`	Canvas novo a cada quadro no teste; o site em si não usa leitura de pixels
L7	Com a aba oculta ou o painel escondido o navegador pausa o `requestAnimationFrame` — números enganosos	Medição com o Edge controlado por DevTools, janela visível; RNPerf usa `!document.hidden`
L8	O RNPerf só estava ativo no modo escuro	`syncPerf` em theme.js agora liga o monitor nos **dois** temas

Mecanismos de desempenho usados (resumo): sprites e fundos **pré-renderizados**; níveis de névoa e cáusticas **pré-calculados**; canvas fixo em meia resolução; cada faixa é ligada e desligada por `IntersectionObserver` (`band()`); pools de bolhas de tamanho fixo; LOD nos navios; DPR limitado; movimento por `dt`; `prefers-reduced-motion` desenha um quadro só. A suíte de 88 testes (API/segurança) continuou passando; ela não cobre o front-end, por isso a verificação visual e as medições acima.

15.7 Correção da barra de navegação

Problema (antigo, herdado da v2.0): com a janela entre ≈ 770 e ≈ 1040 px de largura a barra do topo não cabia: os links, a etiqueta "Since 1546" e o botão de tema brigavam por espaço e o **botão de tema saía da tela**. Um primeiro ajuste até 1100 px ainda deixava a faixa de 1101 a ≈ 1260 px quebrada; o limite subiu para 1280 px.

Regra	Efeito
`.nav-link { white-space: nowrap }`	Rótulos como "Frota Atual" nunca quebram em duas linhas
`@media (max-width: 1280px)`	Esconde o texto do botão de tema e a etiqueta "Since 1546"; barra com 1,25 rem de margem; links com `padding: .4rem .55rem` e espaçamento de letras menor; `.nav-links` com `min-width: 0` e centralizado
`@media (max-width: 960px)`	Marca menor (0,95 rem), links com `padding: .4rem` e fonte de 0,84 rem
`@media (max-width: 768px)`	Botão de tema empurrado para a direita (`margin-left: auto`); abaixo disso a barra de links já some (pendência do menu para celular, seção 10)

Verificado com as fontes carregadas (`document.fonts.status = loaded`), medindo as caixas dos elementos no Edge:

Largura da janela	Marca	Links	Botão de tema	Sobreposição?
800 px	20 – 181	197 – 690	706 – 770	Não
900 px	20 – 181	197 – 790	806 – 870	Não
1024 px	20 – 208	224 – 914	930 – 994	Não

Uma captura anterior, feita antes de as fontes carregarem, parecia mostrar a marca sobre o “Início” a 800 px. Era o texto ainda na fonte reserva — a medição com as fontes carregadas não encontrou sobreposição.



Figura 38 — Barra do topo: 1100 px (em cima) e 800 px nos modos claro e escuro; a marca, os links e o botão de tema cabem sem se sobrepor.

15.8 Arquivos e guia de manutenção do tema claro

Arquivo	Linhas	Papel
`static/js/convoy-hero.js`	661	Hero: comboio em perspectiva (registra a cena `light` no RNHero)
`static/js/calm-sea.js`	1.337	Mergulho: medidor, canvas fixo, faixas de patrulha e de recife, sprites
`static/css/light.css`	194	Tokens e estilos do tema claro (fundo, títulos com halo, cartões de vidro, rodapé de areia)
`static/js/main.js`	−269	Removida a antiga cena clara (`registerSunnyScene`, 263 linhas); fallback de cor até a cena registrar
`static/js/theme.js`	±4	RNPerf passa a atuar nos dois temas
`templates/index.html`	+31	Link do CSS, canvas e medidor, faixas “patrulha” e “recife”, dois scripts novos (CSP inalterada)

Quero...	Edite...
Mudar o ritmo ou a posição do comboio	`SPEED`, `LANES`, `HZ`, `VPX` em convoy-hero.js
Adicionar um tipo de navio ao comboio	Uma função em `MODELS` (prismas) e uma entrada na pista desejada
Mudar a cor da água por profundidade	`STOPS` em calm-sea.js
Alterar zonas ou profundidade máxima	`Gauge` e `MAX_M` (40) em calm-sea.js
Mudar o que vive no recife	função `reef()` e os sprites (peixes, corais) em calm-sea.js
Mudar cores do tema claro	tokens no início de `light.css`
Publicar a mudança	`python build_static.py` (atualiza `docs/`), commit e push

15.9 Limites e o que ficou pendente

- **Uma máquina só:** os números vêm de um PC com GPU dedicada e monitor de 120 Hz. Não há medição em celular real nem em computador modesto; é a melhor evidência disponível, não uma garantia.
- **Travadas logo após carregar:** vistas em uma rodada sem aquecimento (L1) nos dois temas. Se existirem em uso real, o visitante as notaria nos primeiros segundos; vale investigar (provável: construção de sprites e compilação do JavaScript no primeiro quadro).
- **Menu para celular:** continua pendente (seção 10); a correção da barra tratou só de larguras de tablet/notebook.
- **Modo de movimento reduzido:** o hero claro é desenhado uma vez; as faixas do mergulho continuam dependendo do navegador honrar `prefers-reduced-motion` (não foi testado em aparelho real).

16. Glossário

Termo	Significado
Canvas 2D	Elemento HTML em que o JavaScript desenha pixels diretamente (usado no hero).
WebGL	API do navegador para desenhar gráficos 3D na GPU. O Three.js é uma biblioteca que a simplifica.
Three.js	Biblioteca JavaScript de 3D: cena, câmera, luzes, geometrias e materiais.
Draw call	Uma ordem de desenho enviada à GPU. Muitas por frame = mais custo de CPU.
InstancedMesh	Objeto Three.js que desenha várias cópias da mesma peça em uma única draw call.
DPR (device pixel ratio)	Razão entre pixels físicos e CSS. Em escala de 125%, DPR = 1,25.
FPS	Quadros por segundo; 60 é o alvo de fluidez em telas comuns.
requestAnimationFrame (rAF)	Agenda um desenho sincronizado com a taxa de atualização da tela.
dt (delta time)	Tempo real entre dois frames; usado para o movimento não depender do monitor.
IntersectionObserver	API que avisa quando um elemento entra ou sai da tela, sem ler layout a cada scroll.
ResizeObserver	API que avisa quando um elemento muda de tamanho.
Compositor	Etapa do navegador que combina camadas na GPU. transform e opacity animam nele sem repintar.
Repaint / Layout	Repintar pixels / recalcular posições. Ambos são caros quando ocorrem em todo frame.
Especificidade CSS	Regra que decide qual declaração vence quando várias se aplicam ao mesmo elemento.
backdrop-filter	Efeito de desfoque do que está atrás de um elemento; caro sobre conteúdo animado.
SRI	Subresource Integrity: hash que o navegador confere para garantir que um script externo não foi alterado.
defer	Atributo do script que o baixa em paralelo e executa após o HTML, sem bloquear a página.
Debugger do Werkzeug	Console interativo do Flask em modo debug; permite executar código no servidor.
prefers-reduced-motion	Preferência do sistema por menos animação; o site a respeita.
inert	Atributo HTML que remove um elemento do foco, cliques e leitores de tela.
GitHub Pages	Hospedagem gratuita de sites estáticos a partir de um repositório do GitHub.
Site estático	Conjunto de arquivos (HTML, CSS, JS) entregues como estão, sem processamento no servidor.
localhost / 127.0.0.1	Endereço da própria máquina. Não é acessível por outras pessoas.
CSP	Content-Security-Policy: regra que diz ao navegador de quais origens a página pode carregar recursos.
HTTPS / TLS	Conexão criptografada entre navegador e servidor; o Pages a fornece automaticamente.
DNS	Sistema que traduz nomes (royalnavy.com.br) em endereços IP.
Registro A / AAAA	Registros DNS que ligam um nome a endereços IPv4 / IPv6.
Registro CNAME	Registro DNS que faz um nome (www) apontar para outro nome (alansouzadev7.github.io).
Propagação de DNS	Tempo para as mudanças de DNS chegarem a todos os servidores (minutos a horas).

Termo	Significado
RDAP	Protocolo de consulta a dados de registro de domínios; usado para checar a disponibilidade.
NXDOMAIN	Resposta DNS “domínio inexistente”.
Enforce HTTPS	Opção do Pages que redireciona todo acesso HTTP para HTTPS.
data-theme	Atributo do <html> (light ou dark) que ativa os estilos e cenas do modo escuro.
View Transitions API	API do navegador que fotografa a página antes e depois de uma mudança e permite animar a passagem entre as duas imagens.
clip-path	Propriedade CSS que recorta a área visível de um elemento; aqui, um círculo que cresce a partir do botão.
Web Animations API	element.animate(): anima propriedades por JavaScript, inclusive pseudo-elementos, usando o compositor.
Sprite	Imagem pequena pré-desenhada e reutilizada (brilho, fumaça, silhueta de navio).
Pool de objetos	Conjunto fixo de objetos reaproveitados (partículas, bolhas), para não criar objetos durante a animação.
Mistura aditiva	Modo de desenho em que as cores se somam (globalCompositeOperation = 'lighter'); usado em fogo e brilhos.
Meia resolução	Canvas desenhado com metade da largura e da altura (¼ dos pixels) e ampliado pelo navegador.
RNPerf	Monitor de tempo de quadro do site que reduz partículas e efeitos (nos dois temas, desde a v4.1) em máquinas lentas; níveis 2 (alto), 1 (médio) e 0 (baixo).
HUD	Elemento fixo sobre a tela (aqui, o medidor de profundidade).
Zonas oceânicas	Camadas do mar por profundidade: epipelágica (até ≈ 200 m), mesopelágica (até ≈ 1.000 m), batipelágica (até ≈ 4.000 m) e abissal; o medidor do site usa esses limites.
Mediana / p95	Valor central de uma série / valor abaixo do qual ficam 95% das medições.
WCAG	Diretrizes de acessibilidade web; o nível AA pede contraste mínimo de 4,5:1 para texto comum.
Tag (git)	Marca permanente em um commit; a v2.0.0 e a v3.0.0 são tags.
SQL injection	Ataque em que texto malicioso digitado pelo usuário vira parte de um comando SQL. Evita-se com consultas parametrizadas e validação.
Consulta parametrizada	O texto do usuário viaja separado do SQL, como parâmetro, e nunca é interpretado como comando.
XSS (cross-site scripting)	Ataque que faz a página executar código inserido em um texto. Evita-se exibindo dados com textContent e com uma CSP restritiva.
Papel (role) do PostgreSQL	Conta do banco com permissões próprias; a API usa um papel que só lê uma view.
View	Consulta salva que aparece como uma tabela virtual; aqui, mostra só os campos públicos e esconde a tabela original.
Privilégio mínimo	Princípio de dar a cada conta apenas o que ela precisa: a API só lê, nunca escreve.
Trigrama (pg_trgm)	Pedaço de 3 letras; comparar trigramas permite achar textos parecidos (busca com erro de digitação).
unaccent	Extensão do PostgreSQL que remove acentos para comparar textos.
Limite de requisições (rate limit)	Teto de pedidos por IP e por minuto; responde 429 quando excedido.

Termo	Significado
Pool de conexões	Conjunto pequeno de conexões com o banco reaproveitadas entre requisições.
CORS	Regra do navegador que decide se um site pode ler respostas de outra origem; aqui só por lista exata.
HSTS	Cabeçalho que manda o navegador usar sempre HTTPS naquele site.
DNS rebinding	Ataque que faz um site externo acessar serviços locais (localhost) pelo navegador da vítima; barrado validando o cabeçalho Host.
CVE	Identificador público de uma vulnerabilidade de segurança conhecida (ex.: CVE-2026-27205).
Bandit · pip-audit	Ferramentas que procuram falhas de segurança no código Python e vulnerabilidades nas dependências.
LGPD	Lei Geral de Proteção de Dados: exige cuidado com dados pessoais, inclusive endereços IP e textos pesquisados em registros de acesso.
Hardening	Endurecimento: reduzir a superfície de ataque e as permissões de um sistema.
PaaS (Platform as a Service)	Serviço que roda a aplicação sem o usuário administrar servidor, sistema operacional ou rede; aqui, o Render.
Banco de dados gerenciado	PostgreSQL operado por um provedor (aqui, o Neon): backups, rede e o próprio motor ficam por conta dele.
Spin down / autosuspend	O provedor desliga a aplicação ou o banco depois de um tempo sem uso, para economizar recursos no plano gratuito; volta sozinho no próximo acesso, com alguns segundos de atraso.
Cadeia de certificados	Sequência de assinaturas entre o certificado de um servidor e uma raiz confiável; o cliente só precisa confiar na raiz para validar toda a cadeia.
verify-full	Modo mais rigoroso de TLS do PostgreSQL: confere a cadeia do certificado *e* se o nome do servidor bate com o do certificado.
sslrootcert	Parâmetro de conexão que diz onde estão as raízes confiáveis para validar o certificado do servidor; pode ser um arquivo, ou a palavra especial “system”.
ISRG Root X1	Raiz de confiança pública mantida pela Let's Encrypt, autoassinada, usada por milhões de sites; válida até 2035.
Blueprint (Render)	Arquivo (aqui, deploy/render.yaml) que descreve um serviço do Render em código, para criá-lo e reproduzi-lo sem preencher formulários.
Ponto de fuga	Ponto do horizonte para onde convergem as linhas paralelas que se afastam da câmera; é o que dá a sensação de profundidade no hero claro.
Projeção em perspectiva	Transformar coordenadas 3D em 2D dividindo pela distância (Z): quanto mais longe, menor e mais perto do horizonte na tela.
Ordem do pintor	Desenhar de trás para a frente, de modo que o que está perto cubra o que está longe, sem precisar de z-buffer.
Descarte de faces traseiras (back-face culling)	Não desenhar as faces de um sólido que estão viradas para longe da câmera.
LOD (nível de detalhe)	Usar modelos mais simples quando o objeto é pequeno na tela (navios distantes viram pontos).
Esteira de Kelvin	O “V” de ondas que um navio deixa atrás de si ao se deslocar na água.
Cáusticas	Desenho de luz ondulante que a superfície da água projeta no fundo; aqui, um ladrilho pré-calculado com ruído de Worley.

Termo	Significado
Ruído de Worley	Padrão de células irregulares calculado a partir da distância até pontos aleatórios; usado para as cáusticas.
ROV	Veículo submarino operado remotamente; no site, o robô amarelo que inspeciona as minas.
Mediana / rodada	Em cada medição de desempenho, o valor central entre as rodadas repetidas; reduz o efeito de uma rodada atípica.

— Fim do documento —

17. Revisão de segurança e robustez (v4.2)

Esta seção documenta a revisão de segurança feita em 30 de setembro de 2026 sobre o site já publicado (v4.1) e as quatro melhorias que saíram dela, identificadas aqui pelas letras **A** a **D**. O método foi de **leitura do código, execução de testes e sondagem do ambiente publicado**; nada foi testado contra sistemas de terceiros. Todos os números abaixo foram medidos ou extraídos do código e do repositório nesta revisão.

> **Em uma frase.** A base de segurança da v3.0/v4.0 estava sólida: nenhuma dependência vulnerável, nenhum segredo no histórico, nenhum achado da análise estática. O que a revisão encontrou foram **lacunas de verificação e de robustez** — a suíte de 88 testes não estava rodando sem banco, a consulta dava **503** depois de o banco ficar ocioso e a política de conteúdo (CSP) ainda aceitava estilos inline — e todas foram corrigidas e cobertas por testes (agora **99**).

17.1 Escopo e verificações feitas

Verificação	Resultado
Dependências (OSV, 7 pacotes fixados)	Flask 3.1.3, Werkzeug 3.1.8, psycopg 3.3.6, psycopg-pool 3.3.3, Flask-Limiter 4.1.1, python-dotenv 1.2.3 e gunicorn 23.0.0: **nenhuma vulnerabilidade conhecida**.
Análise estática (Bandit)	Sem achados em app.py, consulta_api.py, config_env.py, db/ e build_static.py.
Segredos no histórico do Git	17 commits examinados (URIs de conexão com senha, tokens, chaves privadas, DB_PASSWORD=): apenas o texto-modelo *troque-esta-senha* do .env.example. O único arquivo sensível versionado é o certificado raiz público (ISRG Root X1).
Scripts externos	O Three.js r128 (cdnjs) carrega com SRI (sha512) e crossorigin. Nenhum script inline nem evento inline nos modelos.
Produção (Render)	CSP com frame-ancestors 'none', HSTS (1 ano, includeSubDomains), nosniff, X-Frame-Options DENY, COOP/CORP same-origin, Referrer-Policy e Permissions-Policy. HTTP redireciona (301) para HTTPS; Host forjado é recusado (403 na borda); um payload clássico de SQL injection foi barrado pela WAF do provedor antes de chegar ao aplicativo.
Testes automáticos	88 testes existentes — **mas todos ignorados (skipped)** neste computador por falta do banco local (achado A).

Limitação: o pip-audit não conseguiu consultar o PyPI neste computador (erro de certificado SSL, provavelmente de antivírus ou proxy que intercepta o tráfego). A consulta foi feita diretamente à base OSV, a mesma fonte de avisos usada pelo pip-audit.

17.2 Achado A — a suíte de testes não estava sendo exercitada

Problema. Os 88 testes de funcionamento e segurança dependem de um PostgreSQL com a tabela `navios_historicos`. Sem ele, a fixture `exige_banco` ignora todos: o resultado era `88 skipped`, que parece verde mas não prova nada. Além disso, o repositório não trazia o DDL nem dados para montar esse banco.

Solução. Um banco **descartável** em Docker (PostgreSQL 16, escutando só em 127.0.0.1, senha de administrador aleatória e descartada ao final) e um novo script, `db/seed_teste.py`, que cria a tabela de 17 colunas e insere **84 navios sintéticos** (nomes e cascos históricos; datas, comandantes e armamento são placeholders de teste, marcados como tal na coluna de observações). O script só roda em localhost, só cria a tabela se ela não existir e só insere se estiver vazia — nunca altera dados existentes. Os segredos do papel de leitura foram gravados em arquivo temporário (variável `RN_ENV_FILE`), sem tocar em `~/.royalnavy`.

Etapa	Resultado
db/aplicar.py (extensões, view e papel)	Executou os dois SQL e a verificação interna: **11 de 11 [OK]** (somente leitura, statement_timeout 3 s; sem INSERT, UPDATE, DELETE, CREATE, leitura de arquivos ou virar superusuário).
pytest (88 testes originais)	**88 passaram** em cerca de 4 s (antes: 88 ignorados).

Etapa	Resultado
pytest após as correções B e C	**99 passaram** (11 testes novos).

Como reproduzir (PowerShell, na pasta do projeto, com o Docker em execução):

```
docker run -d --name rn-pg-teste -e POSTGRES_PASSWORD=<senha-aleatoria> `
    -p 127.0.0.1:55432:5432 postgres:16-alpine
$env:PGHOST="127.0.0.1"; $env:PGPORT="55432"; $env:PGPASSWORD="<senha-aleatoria>"
$env:RN_ENV_FILE="$env:TEMP\rn-teste.env"      # segredos de teste fora do ~/.royalnavy
python db/seed_teste.py        # tabela + 84 navios sinteticos
python db/aplicar.py           # extensoes, view v_navios, papel navios_leitura e verificacao
python -m pytest -q            # 99 passed
docker rm -f rn-pg-teste       # descarta o banco
```

17.3 Achado B — o 503 intermitente da consulta (cold start do banco)

Sintoma. Na sondagem de produção, `/api/saude` respondeu **503** uma vez e, segundos depois, voltou a responder 200. **Causa provável:** o Neon suspende o banco por inatividade e o pooler derruba conexões ociosas; o pool da aplicação entregava uma conexão já morta, e o tempo de conexão de 3 s era curto para o banco acordar. O mecanismo foi reproduzido localmente (abaixo); a produção não foi alterada durante a revisão.

Reprodução local. Com o banco de teste, uma requisição abriu a conexão do pool; o servidor foi instruído a encerrar essa sessão (`pg_terminate_backend`, o mesmo efeito da suspensão) e a requisição seguinte foi feita:

Versão do código	Antes de derrubar a conexão	Depois de derrubar a conexão
v4.1 (antes)	/api/saude → 200	**/api/saude → 503** (log: *discarding closed connection*)
v4.2 (depois)	/api/saude → 200	**/api/saude → 200** e /api/navios/busca → 200

Correção (consulta_api.py):

- `check=ConnectionPool.check_connection`: o pool testa a conexão antes de entregá-la e descarta as mortas em vez de repassar o erro.
- Tempos configuráveis, sempre limitados a 1–15 s (o gunicorn encerra a requisição em 20 s): `DB_CONNECT_TIMEOUT` (padrão 3 → **8 s**), `DB_POOL_TIMEOUT` (3 → **5 s**) e abertura do pool com o tempo de conexão + 2 s (4 → **10 s**).
- Continua falhando rápido e de forma genérica quando o banco está realmente fora do ar (503 sem detalhes para o cliente; detalhes só no log).

Testes novos (7). Seis casos do limite de tempo (ausente, válido, zero, negativo, enorme, texto inválido) e um que confirma que o pool é criado com `check`, com os tempos esperados e ainda com `default_transaction_read_only=on`. A primeira requisição após uma suspensão longa do Neon ainda pode levar alguns segundos; o que mudou é que ela deixa de falhar.

17.4 Achado C — front-end e política de segurança de conteúdo (CSP)

Inventário dos pontos onde texto vira HTML. A página de consulta (`consulta.js`) já usava somente `textContent`, validava o id como inteiro positivo, cancelava buscas antigas (AbortController) e não enviava cookies. Restavam dois `innerHTML` em `main.js` (modal do navio, com dados estáticos e *escapeHtml*) e seis atributos `style="--fill:N"` nas barras da frota, que obrigavam a CSP a manter `'unsafe-inline'` em `style-src`.

Ponto	Antes (v4.1)	Depois (v4.2)
Modal do navio (main.js)	2 × innerHTML com escapeHtml	DOM criado com createElement + textContent + replaceChildren; helper `criaTexto`; escapeHtml removida
Barras da frota (index.html)	6 × style="--fill:N"	6 × data-fill="N" e 6 seletores em style.css (1, 0,9, 0,8, 0,75, 0,6, 0,4)

Ponto	Antes (v4.1)	Depois (v4.2)
CSP style-src	'self' **'unsafe-inline'** https://fonts.googleapis.com	'self' https://fonts.googleapis.com
CSP em 3 lugares	app.py e *meta* das 2 páginas	Idênticas (já havia um teste que compara cabeçalho e meta) e docs/ regenerado

Validação em navegador real (servidor local com o banco de teste; as duas páginas; temas claro e escuro):

- **0 eventos** `securitypolicyviolation`, 0 erros de console e 0 elementos com atributo `style` em uma página que antes tinha 6.
- As seis barras chegam ao valor final correto (`scaleX` = 1, 0,8, 0,6, 0,75, 0,9 e 0,4). Como o painel de testes roda com a página oculta, o IntersectionObserver não dispara lá; por isso a classe `is-visible` foi aplicada à mão e a transição desligada só para ler o valor final.
- O modal abre com 10 especificações e 6 itens de armamento (mesmo conteúdo de antes).
- Payload na URL — `/consulta?q=<img src=x onerror=alert(1)>&id=60`: nenhuma tag *img* foi injetada; o texto aparece apenas como valor do campo; a ficha 60 carregou normalmente.

Testes novos (4) que impedem a regressão: a CSP não pode conter `unsafe-inline` nem `unsafe-eval`; os modelos não podem ter `<style>`, `style=`, eventos on…=, `<script>` sem `src` nem `javascript:` (2 casos, um por página), e todo script externo precisa de `integrity=`; nenhum arquivo em `static/js` pode usar `innerHTML`, `outerHTML`, `insertAdjacentHTML`, `document.write`, `eval` nem `new Function`.

17.5 Achado D — hostname do banco fora do repositório

O `deploy/render.yaml` (repositório público) continha o hostname do pooler do Neon. Ele não é um segredo — o acesso exige a senha do papel `navios_leitura`, que só lê a view — mas identifica a instância do banco a quem quiser atacá-la. Agora `DB_HOST` usa `sync: false` e o valor fica somente no painel do Render (aba Environment), como já ocorria com `DB_PASSWORD` e `ALLOWED_HOSTS`.

> **Atenção (ação sua).** Antes do próximo *Sync* do blueprint, confirme no painel do Render que a variável `DB_HOST` continua preenchida (o serviço já em execução mantém o valor atual). O hostname permanece nos commits antigos (ce2a480 e 51137b2); reescrever o histórico seria destrutivo e sem ganho real, porque não há segredo a proteger. Se quiser a mesma discrição para o futuro, basta trocar a senha do papel de leitura, o que o `db/aplicar.py --rotacionar-senha` faz.

17.6 Riscos residuais e próximos passos

Item	Situação	Sugestão
Limite de requisições em memória	Contador zera a cada reinício/suspensão do Render (1 worker).	Aceitável neste porte; usar Redis se houver mais de um worker.
GitHub Pages (docs/)	Não permite cabeçalhos; a CSP vai só por *meta*, sem frame-ancestors.	Sem correção possível no Pages; o Render já entrega os cabeçalhos completos.
Trusted Types	Não adotado. Agora que não há innerHTML, seria viável.	Avaliar `require-trusted-types-for 'script'` e testar com o Three.js.
Google Fonts	CSS externo (não aceita SRI) e permite conexão a fonts.gstatic.com.	Hospedar as fontes localmente zera a dependência e melhora a privacidade.
pip-audit neste PC	Falha por erro de certificado SSL local.	Investigar o antivírus/proxy; a OSV foi usada como substituta.
Monitoramento da consulta	/api/saude é profundo (acorda o banco).	Se houver monitor de disponibilidade, tolerar a primeira resposta lenta.

17.7 Resumo da versão 4.2.0

Versão	Conteúdo
4.2.0	Revisão de segurança e robustez: suíte executada de ponta a ponta (99 testes) com banco descartável e *db/seed_teste.py*; pool com verificação de conexão e tempos configuráveis (fim do 503 após inatividade); CSP sem 'unsafe-inline', sem innerHTML e sem style inline; hostname do banco retirado do repositório; README e este documento atualizados.

Arquivos alterados: `consulta_api.py`, `app.py`, `templates/index.html`, `templates/consulta.html`, `static/js/main.js`, `static/css/style.css`, `deploy/render.yaml`, `tests/test_consulta.py`, `README.md` e `docs/` (regenerado); arquivo novo: `db/seed_teste.py`.

Termos novos deste capítulo

Termo	Significado
Cold start	Demora da primeira requisição depois de o banco (ou o serviço) ter sido suspenso por inatividade.
Seed de teste	Conjunto de dados sintéticos carregado num banco descartável só para rodar os testes.
SRI (Subresource Integrity)	Hash no atributo *integrity* que faz o navegador recusar um script externo adulterado.
Trusted Types	Recurso do navegador que só aceita HTML/JS vindos de funções autorizadas, bloqueando quase todo XSS pelo DOM.
OSV	Base pública de vulnerabilidades em pacotes (Open Source Vulnerabilities), consultada por API.

18. Sugestões de navios e painel do administrador (v4.3)

A versão 4.3 (20 de outubro de 2026) abre o acervo à colaboração do público: qualquer visitante pode sugerir um navio que sentiu falta na consulta. O site confere se o navio já existe, registra os que faltam numa fila **em análise** e um **painel restrito** permite ao administrador completar as informações, aprovar ou rejeitar. Todos os números desta seção foram medidos nos testes automáticos, no navegador e no próprio site publicado.

Em uma frase. O visitante sugere; o site compara com o acervo **só lendo o banco**; as sugestões ficam num **arquivo JSON num repositório privado do GitHub** (não no PostgreSQL); o administrador decide no painel e, ao aprovar, recebe o comando SQL pronto para incluir o navio no banco. 157 testes passando, 0 achados do Bandit, fluxo verificado em produção.

18.1 O que o visitante vê

A página `/sugerir` pede o nome do navio (obrigatório) e, se a pessoa quiser, marinha, guerra e uma fonte. Nenhum dado pessoal é pedido. A resposta depende do que está no acervo e na fila:

O visitante digita	O site responde
Um navio do acervo, em qualquer grafia: USS Arizona, arizona, ARIZONA, BB-39, KMS Bismarck, blücher	**"Navio já existente"**, com link para a ficha na consulta. A comparação ignora acentos, maiúsculas e prefixos (USS, HMS, SMS, KMS, HMAS, HMCS, HMNZS, IJN, HNLMS, ORP) e também aceita a designação do casco.
Um nome parecido com um do acervo (ex.: Arizonna)	**"Você quis dizer...?"** com até 3 candidatos (similaridade de trigramas ≥ 0,4). Só registra se a pessoa confirmar que é outro navio.
Um navio que não está no acervo	**"Sugestão registrada"**: entra na fila com a situação *em análise*.
Um navio já sugerido	**"Já está em análise"** (soma +1 pedido, sem duplicar) ou "já avaliada", se a equipe já aprovou ou rejeitou.

Depois do envio o servidor redireciona para a página de resultado (padrão *POST → redirect → GET*): recarregar a página não reenvia o formulário. A página é acessada pela consulta — no texto de ajuda e quando a busca não encontra nada, já com o nome preenchido — e pelas seções de contato. O menu principal não ganhou um 9º item para não reabrir o problema da barra de navegação corrigido na v4.1. Na versão estática (GitHub Pages) a página apenas avisa que precisa do servidor.

18.2 Decisões de projeto

Decisão	Motivo
Sugestões fora do banco	O PostgreSQL continua **somente leitura** para o site (papel navios_leitura, sem nenhuma tabela ou papel novo no Neon). Assim, nenhuma falha no formulário público pode alterar o acervo.
Arquivo JSON num repositório privado do GitHub	O disco do Render gratuito é apagado a cada reinício, e disco persistente só existe nos planos pagos. A API do GitHub grava o arquivo de forma persistente e gratuita; cada sugestão vira um commit (histórico auditável). Repositório **privado e separado**: as anotações não ficam públicas e o token não alcança o código do site.
Token *fine-grained* com permissão mínima	Acesso a um único repositório, só *Contents: Read and write*, com data de validade. Fica apenas nas variáveis de ambiente do Render.
Sem notificação por e-mail	Previsto no início e descartado pelo responsável do projeto. Além disso, o Render gratuito bloqueia as portas SMTP (25, 465 e 587) desde setembro de 2025. O painel destaca as sugestões novas.
Painel no próprio site	Escolhido em vez de trabalhar só no SQL Editor do Neon. Por ser a única área com login, recebeu as proteções da seção 18.4.

18.3 Como funciona por dentro

- **Comparação com o acervo** (`sugestoes.py`): três consultas parametrizadas pelo mesmo pool e papel da consulta. A chave do nome é calculada no próprio banco (`rn_norm` — minúsculas e sem acento — e uma expressão regular que retira os prefixos), para que a normalização seja idêntica à da busca. Depois, igualdade exata com o nome ou com a designação do casco e, se não houver, `similarity()` do pg_trgm para os parecidos.
- **Armazenamento**: classe `ArmazemGitHub` (API REST *contents*, só biblioteca padrão do Python, tempo limite de 8 s) e `ArmazemArquivo` para desenvolvimento e testes. Leitura com cache de 20 s; gravação "ler → alterar → gravar" com o identificador da versão do arquivo (sha); se outra requisição gravou antes (HTTP 409/422), o site relê e tenta de novo, até 3 vezes.
- **Sem armazenamento configurado**, a página avisa que o registro está indisponível — nunca grava num lugar que se apaga sozinho.
- **Limites**: no máximo 500 sugestões em análise e 3.000 no total, para proteger o arquivo contra enxurradas.

18.4 Painel do administrador e segurança

O painel (`/admin`) lista as sugestões (novas destacadas, contagem de pedidos e filtros por situação) e permite completar os 15 campos do acervo, salvar, **aprovar**, **rejeitar** ou **reabrir**. Ao aprovar, ele exibe um comando `INSERT ... WHERE NOT EXISTS` para colar no SQL Editor do Neon — o site nunca escreve no banco. Testado num PostgreSQL real: a primeira execução insere 1 linha, a segunda insere 0, e aspas, aspas duplas e barra invertida chegam intactas.

Camada	O que foi feito
Senha	Só o **hash** (scrypt) fica no ambiente (`ADMIN_PASSWORD_HASH`), gerado por `ferramentas/gerar_hash_admin.py` sem exibir a senha (mínimo 14 caracteres). Sem a variável, o painel não existe (404).
Sessão	Cookie assinado (`SECRET_KEY`), HttpOnly, SameSite=Strict e Secure; sai sozinho após 20 min sem uso e no máximo 2 h; a sessão é trocada no login (contra fixação de sessão).
Força bruta	5 tentativas a cada 15 min por IP **e** trava geral após 20 erros em 15 min, contra ataque vindo de muitos IPs.
CSRF	Token aleatório na sessão e em todo formulário (comparação em tempo constante) e recusa de cabeçalho Origin de outro site ou nulo. Vale também para a sugestão e para o contato.
Robôs	Campo-isca invisível (quem preenche recebe "sucesso" e nada é gravado) e limites por IP: 6 sugestões por minuto e 40 por hora; contato, 5 por minuto.
Entrada e saída	Mesma lista de caracteres permitidos da busca; fonte só com endereço http(s) válido; tudo que volta à página é escapado; respostas do painel com no-store e noindex.
Contato corrigido	A página /contato (06/10/2026) não tinha CSRF, campo-isca nem limite de envios; passou a ter os três.

18.5 Como foi verificado

Verificação	Resultado
Testes automáticos	**157 passando** (99 anteriores + 58 novos), contra o banco descartável em Docker. A API do GitHub é simulada: criação do arquivo, atualização, conflito com nova tentativa e falha de rede sem vazar o token.
Regras de front-end	Os testes de "sem estilo, script ou evento inline" e "CSP do cabeçalho igual à da meta" passaram a cobrir **todos** os modelos, inclusive os do painel.
Análise estática	Bandit sem achados. O único alerta (montagem de SQL em texto) é o comando exibido ao administrador: o site não o executa e cada valor tem as aspas dobradas — justificado no código.
Navegador	Página e painel nos temas claro e escuro e no celular (375 px, sem rolagem horizontal); 0 violações de CSP; 0 atributos style.
Produção (Render)	"USS Arizona" → Navio já existente; "HMS Victory" → Você quis dizer HMS Victorious? → confirmado → **registrado no repositório privado**; "Victory" → já está em análise (2 pedidos); envio sem token → 403; /admin → tela de login; login do administrador conferido pelo responsável.

Bug encontrado no navegador (e não pelos testes)

O login do painel respondia **403**. O painel usava `Referrer-Policy: no-referrer`, e, por especificação, com essa política o navegador envia `Origin: null` em todo POST — que a checagem de CSRF recusa, corretamente. A política passou a ser `same-origin` (o endereço continua sem vazar para outros sites) e um teste de regressão garante que o painel não volte a usar `no-referrer`. O cliente de testes do Flask não envia Origin, por isso só o teste no navegador revelou o problema.

18.6 Configuração da publicação

Variáveis novas no Render (aba Environment), todas fora do Git:

Variável	Conteúdo
SUGESTOES_GITHUB_REPO	dono/repositório dos dados (o repositório privado de sugestões)
SUGESTOES_GITHUB_TOKEN	token fine-grained (Contents: Read and write, só nesse repositório, com validade)
ADMIN_PASSWORD_HASH	hash scrypt gerado por ferramentas/gerar_hash_admin.py (nunca o botão "Generate", que criaria um valor aleatório sem senha)
SECRET_KEY	64+ caracteres aleatórios (aqui o "Generate" do Render serve)

O `deploy/render.yaml` lista essas variáveis com `sync: false` (e `generateValue` para a SECRET_KEY). O deploy automático do Render não disparou com o envio para a main; foi preciso o Manual Deploy, como já registrado na seção 14 (v4.0).

Cuidados operacionais. (1) Nunca mostrar valores de tokens ou senhas em capturas de tela; se acontecer, use *Regenerate token* no GitHub e atualize a variável no Render. (2) O token tem data de validade: anote-a e regenere antes, senão a página passa a responder "Não foi possível registrar a sugestão agora". (3) Guarde a senha do painel num gerenciador de senhas: o hash não permite recuperá-la.

18.7 Limites e próximos passos

Item	Situação	Sugestão
Login do painel	Só senha (sem segundo fator).	Acrescentar código de 6 dígitos por aplicativo autenticador (2FA).
Arquivo único	A API contents só entrega arquivos de até 1 MB por esta rota (milhares de sugestões).	Arquivar as avaliadas por ano se a fila crescer muito.
Dependência do GitHub	Se a API estiver fora do ar, o registro falha com aviso (nada se perde silenciosamente).	Aceitável; o status aparece em githubstatus.com.
Inclusão no acervo	Manual: o administrador cola o SQL no Neon.	Intencional: mantém o banco somente leitura para o site.
Notificação	Nenhuma (decisão do responsável).	O painel destaca as novas; o repositório também registra cada sugestão como commit.

18.8 Resumo da versão 4.3.0

Versão	Conteúdo
4.3.0	Página “Sugerir navio” com comparação ao acervo (exato, parecidos e prefixos), fila em arquivo JSON num repositório privado do GitHub, painel do administrador com login seguro e geração do SQL de inclusão, proteções CSRF/campo-isca/limites também no contato; 157 testes; README e este documento atualizados.

Arquivos novos: `sugestoes.py`, `protecao.py`, `templates/sugerir.html`, `templates/admin_*.html`, `static/css/sugerir.css`, `ferramentas/gerar_hash_admin.py`, `tests/test_sugestoes.py`. Alterados: `app.py`, `build_static.py`, modelos e CSS da consulta e do contato, `consulta.js`, `deploy/render.yaml`, `.env.example`, `README.md` e `docs/`.

Termos novos deste capítulo

Termo	Significado
CSRF	Ataque em que outro site faz o navegador da vítima enviar um formulário ao nosso; barrado pelo token secreto em cada formulário.
Campo-isca (honeypot)	Campo escondido de pessoas que só robôs preenchem; denuncia o envio automático.
Token fine-grained	Chave de acesso do GitHub limitada a repositórios e permissões escolhidos, com validade.
Hash (scrypt)	Transformação de mão única da senha; permite conferir a senha digitada sem guardá-la.
Fixação de sessão	Ataque que reaproveita uma sessão criada antes do login; evitado trocando a sessão ao entrar.
POST → redirect → GET	Após enviar um formulário, o servidor redireciona para uma página comum; recarregar não reenvia os dados.

19. PDFs dos navios para download (v4.4)

A versão 4.4 (20 de outubro de 2026) permite **baixar as informações dos navios em PDF**: a ficha completa de um navio ou o resultado de uma busca, gerados na hora pelo próprio servidor. Os números desta seção foram medidos nos testes automáticos, nos PDFs gerados e no navegador.

> **Em uma frase.** Dois botões novos na consulta geram PDFs em cerca de 60 a 110 ms, com os mesmos dados, a mesma validação e o mesmo papel somente-leitura da consulta; o texto do banco é escapado antes de entrar no PDF. 186 testes passando (29 novos), 0 achados do Bandit.

19.1 O que o visitante vê

Botão na consulta	Rota	Conteúdo
Baixar ficha (PDF)	`GET /api/navios/<id>/pdf`	Ficha completa (A4 retrato): marinha, tipo, guerra, casco, classe, cronologia com datas no formato brasileiro, comando, batalhas, armamento, situação atual, observações e outros navios da classe.
Baixar resultados (PDF)	`GET /api/navios/busca/pdf`	A busca que está na tela (A4 paisagem), com os filtros usados e uma tabela de nome, casco, classe, marinha, tipo e guerra; até 200 navios por PDF (acima disso, o PDF avisa para refinar a busca).

O download usa o cabeçalho *Content-Disposition: attachment*, com nomes como `navio-1-uss-arizona.pdf` e `busca-estados-unidos.pdf`. O botão da lista só aparece quando há resultados e carrega os mesmos critérios da busca exibida; o servidor valida tudo de novo. Na versão estática (GitHub Pages) a consulta, e portanto os PDFs, continuam indisponíveis.

19.2 Decisões de projeto

Decisão	Motivo
Gerar no próprio Flask (Python)	Chegou a ser proposto um serviço separado em PHP; o responsável do projeto preferiu manter tudo em Python. Assim os PDFs reaproveitam a leitura, a validação e o limite de requisições que a consulta já tem, sem um segundo serviço no Render.
ReportLab, em memória	Mesma biblioteca usada nesta documentação; gera o arquivo direto na memória do servidor, sem gravar nada em disco.
Fontes padrão do PDF (Helvetica)	Não exigem arquivos de fonte no servidor e deixam a geração rápida. Cobrem o alfabeto latino (cp1252): um caractere fora dele vira a letra sem o sinal (“Sōryū” → “Soryu”).
Até 200 navios por PDF de busca	Mantém o arquivo e o tempo de geração pequenos; o PDF informa o total e pede para refinar a busca.
Pillow 13.0.0	O ReportLab depende do Pillow; a versão 12.3.0 tem 9 avisos de vulnerabilidade na base OSV, a 13.0.0 nenhum. As três dependências novas ficaram fixadas em requirements.txt e requirements-lock.txt.

19.3 Segurança

Camada	O que foi feito
Dados	Lidos pelo papel `navios_leitura` na view `v_navios`, com as mesmas consultas parametrizadas da ficha e da busca (a ficha em JSON e a ficha em PDF usam a mesma função).
Entrada	A busca em PDF passa pela mesma validação estrita (caracteres, tamanhos, parâmetros conhecidos): parâmetros inválidos dão 400; navio inexistente dá 404; só GET.
Abuso	Gerar PDF custa mais CPU que responder JSON: limite de 10 fichas e 6 buscas em PDF por minuto por IP, além do limite geral da API.
Conteúdo	Todo texto do banco é escapado antes de entrar no layout (o ReportLab interpretaria marcações como <b>); caracteres de controle são removidos e cada campo é limitado a 4.000 caracteres.
Arquivo	Nome do arquivo só com [a-z0-9-] (nada de aspas, barras ou acentos no cabeçalho HTTP); respostas com `no-store` e `nosniff`.

19.4 Como foi verificado

Verificação	Resultado
Testes automáticos	**186 passando** (157 anteriores + 29 novos em tests/test_pdf.py), contra o banco descartável em Docker. Os testes leem o **texto** dos PDFs gerados (pypdf): nome, casco, classe, datas, filtros e total, e não só a existência do arquivo.
Casos de borda	Navio inexistente (5 casos), parâmetros proibidos, métodos não GET, 6 entradas inválidas na busca, limite por minuto, limite de 200 linhas, dados hostis (marcações, caracteres de controle, texto de 9.000 caracteres) e 6 nomes de arquivo perigosos.
Desempenho	Ficha do USS Arizona: 108 ms na primeira chamada (inclui carregar a biblioteca), 1 página, 2,8 KB. Busca da 2ª Guerra Mundial (59 navios): 59 ms, 3 páginas, 8,6 KB. Um teste garante menos de 1 s por ficha.
Navegador	Os dois botões aparecem na consulta e baixam o PDF com o nome correto; 0 violações de CSP.
Análise estática	Bandit sem achados. Um alerta sobre xml.sax.saxutils.escape (falso positivo: a função só troca &, < e >) foi eliminado trocando-a por html.escape.

19.5 Limites e próximos passos

Item	Situação	Sugestão
Alfabetos não latinos	Transliterados (“Sōryū” → “Soryu”; ideogramas viram “?”).	Embutir uma fonte TTF com Unicode completo (ex.: DejaVu Sans).
Catálogo completo	Não implementado: o PDF de busca vai até 200 navios.	Gerar o catálogo uma vez e guardar em cache.
Imagens	Os PDFs são só texto e tabelas.	Incluir a silhueta do navio quando houver imagens no acervo.
Cache	Cada PDF é gerado a cada pedido (rápido, e protegido pelo limite por IP).	Cache curto se o volume crescer.

19.6 Resumo da versão 4.4.0

Versão	Conteúdo
4.4.0	PDFs para download da ficha de cada navio e do resultado da busca, gerados em memória pelo Flask com o ReportLab; mesma leitura somente-leitura, validação e limites da consulta; Pillow atualizado para 13.0.0; 186 testes; README e este documento atualizados.

Arquivos novos: `pdf_navios.py` e `tests/test_pdf.py`. Alterados: `consulta_api.py`, `templates/consulta.html`, `static/js/consulta.js`, `static/css/consulta.css`, `requirements*.txt`, `README.md` e `docs/`.

Termos novos deste capítulo

Termo	Significado
Content-Disposition	Cabeçalho HTTP que diz ao navegador para baixar a resposta como arquivo, com o nome indicado.
ReportLab	Biblioteca Python que monta PDFs (textos, tabelas, páginas) diretamente em código.
cp1252 (WinAnsi)	Conjunto de caracteres do alfabeto latino coberto pelas fontes padrão do PDF.
Transliteração	Troca de um caractere por outro equivalente que a fonte consegue desenhar (ō → o).